Web アプリ開発 学習資料

12 付録 — 出てくるコードとコマンドの一覧

00〜11 に出てくる 1414 個のコードの部品とコマンドを、機能ごとに、意味と初めて出てくる所つきで引く

12 付録: 出てくるコードとコマンドの一覧（機能ごと）

この一覧について

00〜11 の巻のコードに出てくる、**HTML のタグと属性・CSS の項目・TypeScript と React・Python と Django・Rust と Topcoat・SQL・書式・記号・設定ファイル・コマンドなど 1414 個**を、**59 の分類**に分けて、1 つずつ説明した一覧です。

- **本文との関係:** どの部品も、本文の中で**初めて出てくるコードのすぐ下**に、「**▶ ここで初めて出てくるコード**」の箱で同じ説明を載せています。この一覧は、それを機能ごとに並べ直した物です。
- **使い方:** 「この `useEffect` は何だっけ？」「`.filter()` は Django と TypeScript で何が違う？」と思ったら、下の「分類の一覧」から分類を選んで探してください。HTML 版（`webapp_learning\html`）なら、`Ctrl` + `F` で、コードの名前で探すのがいちばん速いです。
- **「初めて出てくる所」の列:** その部品が最初に出てくる巻と章です（`08 前編 3. 設定を読む` は、08 の前編の 3 章）。そこで、実際にどう使われているかを見られます。
- **文字や記号の書き方も入っています:** f 文字列の書式（`{amount:,.2f}`）・Rust の `{:<10}`・`\n` のような特別な文字・正規表現・日時の書式（`%Y-%m-%d`）・記号（`?.`・`??`・`&&`・`**kwargs`）・HTML と JSX の属性（`for`・`htmlFor`）・Django のテンプレートの `{% for %}`・`package.json` や `Cargo.toml` の設定・コマンドのオプションも、1 つずつ説明しています。
- **やりたいことから探す時は:** 次の 13 付録「逆引き」で、「一覧をしぼり込みたい」「数を 3 けたごとに区切りたい」のような、やりたいことからコードを探せます。本文に出てこない、合わせて覚えておくとよいコードも載せています。
- **同じ綴りでも、言語が違えば別の物**として、それぞれの言語の分類に載せています。例えば `.filter()` は、TypeScript では配列から条件に合う物を集めるメソッド、Django ではデータベースの行を条件でしぼるメソッドです。
- **分類の並び:** 学ぶ順（道具 → HTML・CSS → TypeScript・React → Python・Django → Rust・Topcoat）に並べています。「画面に出す」から後ろの分類は、主に 07〜10 の Rust・Topcoat・SQL・メールの部品です。
- **読者が自分で書く名前**（`Request`・`list_requests` など、練習アプリのコードで自分で作る型や関数の名前）は、作った所の本文で説明しているので、この一覧には入れていません。ただし、作るより前に先に出てくる物（`Outbox`・`<FormPage>` など）は、「練習アプリで作る物」の表に、意味を載せています。

> **ヒント:** 1 つの部品を覚えたら、その分類の表の、近くにある物も読んでみてください。例えば `Django` の表には、行を作る（`.create()`）・探す（`.filter()`・`.first()`）・数える（`.count()`）・保存する（`.save()`）が並んでいます。一緒に覚えると、使える場面がぐっと増えます。

分類の一覧

分類	数	分類	数
PowerShell	48	Vec（並び）	17
コマンド	2	HashMap（表）	2
Node.js	13	範囲	1
設定ファイル（JSON）	17	条件	9
Python の道具	5	くり返し	6
Django の道具	3	場合分け	8
psql	2	Option	10
Rust の道具	3	エラー処理	24
cargo のコマンド	10	関数	5
設定ファイル（TOML）	17	型とメソッド	29
Rust の部品（クレート）	13	モジュール	8
HTML	95	所有権と借用	7
CSS	40	イテレーター	17
コメント	3	ファイルとフォルダ	20
文字列の中の特別な書き方	10	入出力	3
正規表現	27	プロセス	5
日時の書式	7	設定	1
JavaScript・TypeScript	78	時間	25
TypeScript	85	スレッドと共有	7
React	69	非同期	7
練習アプリで作る物（TypeScript）	5	HTTP	12
Python	194	認証	10
Django	113	テスト	6
Django REST framework	15	データベース	29

分類	数	分類	数
練習アプリで作る物（Python）	1	SQL	74
画面に出す	9	JSON	8
変数	5	Topcoat	67
数	15	練習アプリで作る物（Rust）	34
文字	7	メール	19
文字列	33		

PowerShell

コード	意味と使い方	初めて出てくる所
`cd`	「change directory（フォルダを移る）」の略。`cd rust-study` で、今いるフォルダの中の `rust-study` へ移ります。`cd ..` は 1 つ上のフォルダへ、`cd ~` はホームフォルダ（`C:\Users\you`）へ戻ります。今いる場所は、行の頭の `PS C:\Users\you\rust-study>` の部分に表示されます。	00 6. 練習の環境を用意する
`#`	`#` から行の終わりまでは「コメント」（人が読むためのメモ）で、PowerShell は無視します。コマンドの横に、何をするかを日本語で書いておけます。コピーしてそのまま貼り付けても、問題ありません。	00 6. 練習の環境を用意する
`mkdir`	「make directory（フォルダを作る）」の略。`mkdir ~\Documents\my-practice\01` のように、途中のフォルダもまとめて作れます（`~` はホームフォルダ `C:\Users\you`）。もう同じ名前のフォルダがあると、エラーで知らせます。	01 2. HTML の基本
`.\`	`.\target\debug\practice.exe` の `.\` は、「今いるフォルダの中の」という意味です。PowerShell は、安全のため、今いるフォルダのプログラムでも `.\` を付けないと動かしません。	04 1. 準備: 仮想環境を作って、部品を入れる
`Copy-Item`	ファイルやフォルダをコピーするコマンド。`Copy-Item 元 先`。最後の `.` は「今いるフォルダへ」という意味です。	05 2. プロジェクトとアプリを作る
`param`	スクリプトが受け取る引数を決める書き方。`param([string[]] $ManageArgs)` は、スクリプトに渡した値を `$ManageArgs` に入れます。	06 前編 2. 開発の準備

コード	意味と使い方	初めて出てくる所
`Join-Path`	フォルダの場所とファイル名を、`\` でつないだ場所を作るコマンド。`Join-Path $PSScriptRoot "dev.env"` は「このスクリプトのフォルダの `dev.env` 」。	06 前編 2. 開発の準備
`if`	PowerShell の条件の書き方。`if (-not (Test-Path $envFile)) { … }` は「ファイルが無ければ」。比べる時は `-eq`・`-ne`・`-not` などを使います。	06 前編 2. 開発の準備
`-not`	「〜でない」（`True` と `False` を逆にする）。`-not (Test-Path $file)` は「そのファイルが無いなら」。`!` と書いても同じです。	06 前編 2. 開発の準備
`Test-Path`	ファイルやフォルダがあるかを調べるコマンド。あれば `True`、無ければ `False`。	06 前編 2. 開発の準備
`throw`	エラーを起こして、そこで止める命令。`throw "dev.env がありません"` のように、説明の文を付けます。`try … catch` の中なら `catch` へ、無ければスクリプトがそこで止まります。	06 前編 2. 開発の準備
`foreach`	PowerShell のくり返し。`foreach ($line in Get-Content $envFile) { … }` は、ファイルの 1 行ずつを `$line` に入れて、中を動かします。	06 前編 2. 開発の準備
`-match`	正規表現（文字の形の決まり）に合うかの比べ方。`$line -match '^\s*#'` は「空白のあと `#` で始まる行（コメント）か」。合った部分は `$Matches` で取り出せます。	06 前編 2. 開発の準備
`-or`	「または」（どちらか一方でも `True` なら `True`）。`$LASTEXITCODE -ne 0 -or $out.Count -eq 0` は「失敗したか、出力が空か」。	06 前編 2. 開発の準備
`-notmatch`	`-match` の逆で、正規表現に合わないかの比べ方。`$line -notmatch '='` は「`=` を含まない行か」。	06 前編 2. 開発の準備
`continue`	くり返し（`foreach` など）の中で、今の 1 回分の残りを飛ばして、次へ進む命令。`if (…) { continue }` で、合わない行を飛ばします。	06 前編 2. 開発の準備
`-split`	文字列を、区切り（正規表現）で分けて並びにする演算子。`"a=b" -split '=', 2` は `"a"` と `"b"`（`2` は「多くても 2 つに分ける」）。	06 前編 2. 開発の準備
`Set-Item`	項目の値を設定するコマンド。`Set-Item -Path "Env:名前" -Value 値` で、この PowerShell の窓の中だけの環境変数を設定します。	06 前編 2. 開発の準備
`Set-Item -Path`	`-Path "Env:名前"` で、どの項目（ここでは環境変数）を設定するかを指定します。	06 前編 2. 開発の準備

コード	意味と使い方	初めて出てくる所
`Set-Item -Value`	`-Value 値` で、設定する値を指定します。	06 前編 2. 開発の準備
`Push-Location`	今いるフォルダを覚えてから、指定したフォルダへ移るコマンド。あとで `Pop-Location` を打つと、覚えたフォルダへ戻れます。	06 前編 2. 開発の準備
`try`	`try { … } finally { … }` で、途中で失敗しても、`finally` の中（元のフォルダへ戻る など）を必ず動かす書き方。	06 前編 2. 開発の準備
`finally`	`try { … } finally { … }` の `finally` の所は、`try` の中がうまくいっても、途中で失敗しても、必ず最後に動きます。元のフォルダへ戻る・記録を閉じる、などの後片づけに使います。	06 前編 2. 開発の準備
`Remove-Item`	ファイルやフォルダを消すコマンド（短い呼び名は `rm`）。**ごみ箱に入らず、すぐ消えます。**フォルダを中身ごと消す時は `-Recurse` を付けます。消す前に、名前をよく確かめます。	06 前編 2. 開発の準備
`Remove-Item -Path`	`Remove-Item` で、消す物の場所を決めるオプション。`Remove-Item -Path "Env:NAME"` は、環境変数 `NAME` を消します（`Env:` は環境変数の場所）。	06 前編 2. 開発の準備
`Remove-Item -ErrorAction`	`Remove-Item` で、エラーが起きた時の動きを決めるオプション。`-ErrorAction SilentlyContinue` は「無くてもエラーを出さずに次へ進む」。	06 前編 2. 開発の準備
`Pop-Location`	`Push-Location` で覚えたフォルダへ戻るコマンド。	06 前編 2. 開発の準備
`exit`	スクリプトを、決めた終了コードで終えるコマンド。`exit $code` は、manage.py の終了コードをそのまま返します。	06 前編 2. 開発の準備
`'…'`	`'…'`（シングルクォート）で囲んだ文字は、書いたとおりの文字で、中の `$` や `` ` `` は特別な意味になりません。正規表現（`'^\s*#'`）や、そのまま渡したい文字に使います。中に `'` を入れる時は `''` と 2 つ書きます。	06 前編 2. 開発の準備
`"…$変数…"`	`"…"`（ダブルクォート）で囲んだ文字の中の `$名前` は、変数の中身に置きかわります。`"こんにちは、$name さん"` → `こんにちは、太郎 さん`。そのまま `$` を出したい時は、`'…'`（シングルクォート）で囲みます。	06 前編 2. 開発の準備
`$( … )`	`"…$($t.Port)…"` の `$( )` は、「中の式やコマンドの結果を、文字の中に入れる」書き方です。`"$x.Name"` のように書くと `$x` だけが置きかわってしまうので、`.` の付いた物や計算は `$( )` で囲みます。	06 前編 2. 開発の準備

コード	意味と使い方	初めて出てくる所
`$true`	`$true` は「はい（真）」を表す値です。`-Recurse:$true` のように、スイッチに値を渡す時にも使います。反対は `$false` です。	06 前編 2. 開発の準備
`$名前`	`$os = …` のように `$` で始まる名前は「変数」（値に付けた名前）です。`=` の右の結果を覚えておき、あとで `$os` と書けば使えます。名前の大文字・小文字は区別しません。	06 前編 2. 開発の準備
`$ErrorActionPreference`	`$ErrorActionPreference = "Stop"` は、「エラーが起きたら、そこでスクリプトを止める」という設定です。付けないと、エラーが起きても次の行へ進んでしまいます（`.exe` のコマンドの失敗は止まらないので、`$LASTEXITCODE` も調べます）。	06 前編 2. 開発の準備
`$PSScriptRoot`	`$PSScriptRoot` は、「今動いているスクリプト（`.ps1`）が置いてあるフォルダ」の場所が入った、特別な変数です。今いるフォルダがどこでも、スクリプトの隣のファイルを `Join-Path $PSScriptRoot "dev.env"` で見つけられます。	06 前編 2. 開発の準備
`$LASTEXITCODE`	`$LASTEXITCODE` は、最後に動かしたプログラム（`cargo`・`git`・`psql` など、`.exe` のコマンド）の「終了コード」が入る、特別な変数です。`0` なら成功、`0` 以外なら失敗です。`if ($LASTEXITCODE -ne 0) { … }` で、失敗した時の処理を書けます。	06 前編 2. 開発の準備
`[Parameter()]`	`param( )` の中の `[Parameter(ValueFromRemainingArguments = $true)]` は、引数の受け取り方を決める印です。この指定なら、名前を付けずに渡した残りの引数を、全部その変数に受け取ります。`[Parameter(Mandatory)]` なら「必ず渡す」です。	06 前編 2. 開発の準備
`[string[]]`	`[string[]]` は「文字列の並び（配列）」という型です。	06 前編 2. 開発の準備
`&`	`& "C:\Program Files\…\vswhere.exe"` の先頭の `&` は、「この文字（や変数）が指すプログラムを動かす」という印です。場所に空白が入っていて `"` で囲んだ時や、変数に入れた場所を動かす時に必要です。	06 前編 2. 開発の準備
`@( … )`	`@( … )` は「並び（配列）」を作る書き方です。`@("a", "b")` で 2 つの並び、`@()` で空の並び。中のコマンドの結果が 1 つでも、必ず並びとして扱えます。	06 前編 2. 開発の準備
`@名前`	`コマンド @CurlArgs` のように、`$` の代わりに `@` を付けて変数を渡すと、変数に入れた並び（や表）の中身を、引数としてばらして渡せます（スプラッティング）。	06 前編 2. 開発の準備

コード	意味と使い方	初めて出てくる所
`param( … )`	スクリプトの最初に書く `param([string]$Base = "http://…")` は、「このスクリプトが受け取る引数」の決まりです。`.\smoke.ps1 -Base http://…` のように、名前を付けて渡せます。`=` の右は、渡さなかった時の値です。	06 前編 2. 開発の準備
`+=`	`$x += 1` は `$x = $x + 1` の短い形です。並びに `$names += "山田"` と書くと、後ろに 1 つ足した並びになります。	06 前編 2. 開発の準備
`.\dev.ps1`	今いるフォルダの `dev.ps1`（この資料で作った、設定を読んでから manage.py を動かすスクリプト）を動かす書き方。`.\dev.ps1 migrate` のように、後ろに manage.py の命令を書きます。	06 前編 2. 開発の準備
`.\dev_serve.ps1`	今いるフォルダの `dev_serve.ps1`（静的ファイルを集めてから、waitress でアプリを動かすスクリプト）を動かす書き方。	06 後編 1. 開発中と、ビルドした後
`$env:名前`	`$env:GREETING` のように `$env:` を付けた名前は「環境変数」（プログラムに外から渡す設定）です。`$env:GREETING = "おはよう"` で設定すると、その PowerShell の窓から動かすプログラム（`cargo run` など）が読めます。窓を閉じると消えます。	06 後編 1. 開発中と、ビルドした後
`</code> | 行の最後の `` `` （バッククォート。`Shift` + `@` で打ちます）は、「次の行へ続く」という印です。長いコマンドを、何行かに分けて書けます。 `` ` `` の後ろに空白があると続かないので、注意します。	06 後編 1. 開発中と、ビルドした後	
`;`	`;` は、1 行にコマンドを 2 つ以上書く時の区切りです。`cargo fmt --check; $LASTEXITCODE` は、左を動かしてから右を動かします（左が失敗しても、右は動きます）。	09 8. 手元で、アプリから送ってみる

コマンド

コード	意味と使い方	初めて出てくる所
`code`	VS Code を動かすコマンド。`code .` で今のフォルダを開き、`code --version` で版を確かめます。	00 6. 練習の環境を用意する

コード	意味と使い方	初めて出てくる所
`code --version`	`code --version` は、VS Code の版を表示します。	00 6. 練習の環境を用意する

Node.js

コード	意味と使い方	初めて出てくる所
`node`	Node.js（パソコンの上で JavaScript を動かす道具）を動かすコマンド。`node probe.mjs` で、そのファイルを動かします。	00 6. 練習の環境を用意する
`node --version`	`node --version` は、入っている Node.js の版（`v24.…`）を表示します。	00 6. 練習の環境を用意する
`npm`	Node.js の部品を入れる・動かす道具のコマンド。`npm --version` で版を確かめ、`npm install`（部品を入れる）・`npm run dev`（`package.json` の `scripts` を動かす）のように使います。PowerShell で「スクリプトの実行が無効」と出る時は `npm.cmd` と打ちます。	00 6. 練習の環境を用意する
`npm --version`	`npm --version` は、Node.js の部品を入れる道具 npm の版を表示します。	00 6. 練習の環境を用意する
`npm install`	`package.json` に書いてある部品（TypeScript など）を、まとめて入れるコマンド。入った部品は `node_modules` フォルダに入ります（Git には入れません）。最初の 1 回だけ打ちます。`npm install 部品の名前` なら、その部品を足します。	02 1. TypeScript とは
`npx tsc`	TypeScript の型の確かめ（`tsc`）を動かすコマンド。何も出なければ、型のまちがいはありません。まちがいがあると、ファイル名・行・何がおかしいかを表示します。`npx` は、入れた部品のコマンドを動かす道具です。	02 1. TypeScript とは
`.writeHead()`	Node.js の練習用サーバーで、答えの番号（200 など）とヘッダーを書き始めるメソッド。	02 7. サーバーとのやり取り
`.end()`	Node.js のサーバーで、答え（レスポンス）の本文を送って、答えを終えるメソッド。`res.end(JSON.stringify(データ))`。これを呼ばないと、ブラウザーはずっと待ち続けます。	02 7. サーバーとのやり取り
`.listen()`	Node.js のサーバーを、決めた番号で待ち受け始めるメソッド。番号を 0 にすると、空いている番号を自動で選びます。	02 7. サーバーとのやり取り
`createServer()`	`createServer((req, res) => { … })` は、Node.js（`node:http`）で、HTTP のサーバーを作る関数です。	02 7. サーバーとのやり取り

コード	意味と使い方	初めて出てくる所
`npm run`	`package.json` の `scripts` に書いたコマンドを動かすコマンド。`npm run dev` は開発用のサーバーを起動し、`npm run build` は本番用のファイルを作ります。止める時は `Ctrl` + `C` です。	03 1. 動かしてみる
`.argv`	`process.argv` は、プログラムを動かした時の引数の並びです。0 番目は `node` の場所、1 番目はスクリプトの場所、2 番目からが渡した引数です。	04 5. 社員の CSV を読み込む
`sleep()`	`import { setTimeout as sleep } from "node:timers/promises";` で、`setTimeout` を `sleep` という名前で読み込んでいます。`await sleep(1000)` で、1 秒待ちます。	11 B. サンプル集

設定ファイル（JSON）

コード	意味と使い方	初めて出てくる所
`"name":`	`package.json` の `"name"` は、このプロジェクト（パッケージ）の名前です。	02 1. TypeScript とは
`"private":`	`"private": true` は、「このパッケージは npm に公開しない」という印です。間違って公開するのを防ぎます。	02 1. TypeScript とは
`"type":`	`"type": "module"` は、「`.js` のファイルを、新しい書き方（ES モジュール。`import`・`export`）として読む」という設定です。	02 1. TypeScript とは
`"scripts":`	`"scripts"` には、`npm run 名前` で動かすコマンドを書きます。`"check": "tsc"` なら、`npm run check` で `tsc`（型の確かめ）が動きます。	02 1. TypeScript とは
`"check":`	`"scripts"` の中の `"check": "tsc"` は、`npm run check` という名前で、`tsc`（TypeScript の型の確かめ）を動かす登録です。名前は自分で決めます。	02 1. TypeScript とは
`"devDependencies":`	`"devDependencies"` には、開発の時だけ使う部品と、その版を書きます。`npm install` で入ります。	02 1. TypeScript とは
`"@types/node":`	`"@types/node"` は、Node.js の機能（`node:http` など）の型の説明の部品です。これが無いと、TypeScript が Node.js の関数の型を知りません。	02 1. TypeScript とは
`"typescript":`	`"typescript": "7.0.2"` は、TypeScript（型を確かめる道具 `tsc`）の部品と、その版です。	02 1. TypeScript とは

コード	意味と使い方	初めて出てくる所
`"compilerOptions":`	`tsconfig.json` の `"compilerOptions"` には、TypeScript の確かめ方の設定を書きます。	02 1. TypeScript とは
`"strict":`	`"strict": true` は、TypeScript の確かめを、いちばんきびしくする設定です。`null` の取り違えなど、間違いを多く見つけてくれます。	02 1. TypeScript とは
`"target":`	`"target": "es2023"` は、どの版の JavaScript の書き方を前提にするかです。	02 1. TypeScript とは
`"module":`	`"module": "nodenext"` は、`import` の読み方を、今の Node.js と同じにする設定です。	02 1. TypeScript とは
`"noEmit":`	`"noEmit": true` は、`tsc` に、型の確かめだけをさせ、JavaScript のファイルは書き出させない設定です（動かす時は Node.js が直接 `.ts` を読みます）。	02 1. TypeScript とは
`"allowImportingTsExtensions":`	`"allowImportingTsExtensions": true` は、`import … from "./types.ts"` のように、`.ts` を付けて読み込むことを許す設定です。	02 1. TypeScript とは
`"erasableSyntaxOnly":`	`"erasableSyntaxOnly": true` は、「型を消すだけで JavaScript になる書き方」だけを許す設定です。Node.js が `.ts` をそのまま動かせる書き方に、そろえられます。	02 1. TypeScript とは
`"types":`	`"types": ["node"]` は、使う型の説明の部品（ここでは `@types/node`）を選ぶ設定です。	02 1. TypeScript とは
`"include":`	`"include": ["src/*.ts"]` は、確かめるファイルの範囲です。	02 1. TypeScript とは

Python の道具

コード	意味と使い方	初めて出てくる所
`py`	Windows の Python を、版を選んで動かすコマンド（Python Launcher）。`py --version` で版を確かめ、`py -3.14 -m venv .venv` のように、版を指定して仮想環境を作ります。	00 6. 練習の環境を用意する
`py --version`	`py --version` は、`py`（Windows の Python の起動係）が使う Python の版を表示します。	00 6. 練習の環境を用意する
`py -m`	`py -m venv .venv` の `-m` は、「Python の部品（モジュール）を、道具として動かす」指定です。`-m venv` で仮想環境を作り、`-m pip` で部品を入れます。	04 1. 準備: 仮想環境を作って、部品を入れる

コード	意味と使い方	初めて出てくる所
`.\.venv\Scripts\python.exe`	仮想環境（`.venv`）の中の Python を、場所を書いて動かす書き方。`.\.venv\Scripts\python.exe ファイル.py` でプログラムを動かし、`.\.venv\Scripts\python.exe -m pip install -r requirements.txt` で、その仮想環境に部品を入れます。仮想環境を「入る」操作（Activate）をしなくても、確実にその Python が使われます。	04 1. 準備: 仮想環境を作って、部品を入れる
`python`	Python を動かすコマンド。`python -c "…"` は、`"…"` の中の Python のコードを 1 回だけ動かします。	09 2. SMTP の会話を見る

Django の道具

コード	意味と使い方	初めて出てくる所
`.\.venv\Scripts\django-admin.exe`	仮想環境に入った Django の道具を動かす書き方。`django-admin.exe startproject config .` で、今いるフォルダに Django のプロジェクト（`manage.py` と `config` フォルダ）を作ります。	05 2. プロジェクトとアプリを作る
`.add_argument()`	manage.py の自分のコマンドで、受け取る引数を決めるメソッド（`argparse` の仕組み）。	06 前編 9. 社員の CSV の取り込み
`.SUCCESS()`	`self.style.SUCCESS(文)` は、manage.py のコマンドの出力を、成功の色（緑）にする関数です。	06 前編 9. 社員の CSV の取り込み

psql

コード	意味と使い方	初めて出てくる所
`\password`	psql の `\password 名前` は、その利用者（ロール）のパスワードを変える命令です。打つとパスワードを 2 回聞かれ、画面には表示されません（コマンドの履歴にも残りません）。	06 前編 2. 開発の準備
`\echo`	psql の `\echo '文字'` は、文字を画面に表示する命令です。SQL のファイルの中で、区切りの説明を出す時に使います。	10 8. SQL インジェクション

Rust の道具

コード	意味と使い方	初めて出てくる所
`rustc`	Rust のコンパイラー（翻訳する係）。人が書いた `.rs` のファイルを、パソコンが直接動かせる実行ファイル（`.exe`）に翻訳します。ふだんは直接打たず、`cargo` が中で呼んでくれます。`rustc --version` で版を確かめられます。	07 1. 準備
`rustc --version`	`rustc --version` は、入っている Rust のコンパイラーの版（`rustc 1.99.1 …`）を表示します。入れた直後に、ちゃんと入ったかを確かめる時に使います。	07 1. 準備
`cargo`	Rust のプロジェクトをまとめて世話する道具。`cargo new`（作る）・`cargo run`（翻訳して動かす）・`cargo build`（翻訳だけ）・`cargo test`（テスト）・`cargo add`（部品を足す）のように、後ろに「何をするか」を付けて使います。`cargo --version` で版を確かめられます。	07 1. 準備

cargo のコマンド

コード	意味と使い方	初めて出てくる所
`cargo --version`	`cargo --version` は、Cargo の版を表示します。	07 1. 準備
`cargo install`	Rust で作られた道具（コマンド）を、パソコンに入れるコマンド。`cargo install topcoat-cli --version 0.11.0 --locked` は、Topcoat の道具 `topcoat` を、決まった版で入れます。	07 1. 準備
`cargo install --version`	`cargo install topcoat-cli --version 0.11.0` の `--version` は、入れる道具の版を指定します。付けないと、いちばん新しい版を入れます。	07 1. 準備
`cargo install --locked`	`--locked` は、道具を作る時に、道具に付いている `Cargo.lock` の版のまま作る指定です。部品の新しい版のせいで作れない、ということを防げます。	07 1. 準備
`cargo run`	プログラムを翻訳（コンパイル）して、すぐ動かします。`Cargo.toml` があるフォルダで打ちます。プログラムが変わっていなければ翻訳を省くので、2 回目からは速くなります。`cargo run -- 値` のように `--` の後ろに書いた値は、プログラムへの引数（外から渡す値）になります。	07 5. Topcoat の最小のアプリ

コード	意味と使い方	初めて出てくる所
`cargo build`	プログラムを翻訳（コンパイル）して実行ファイルを作るだけで、動かしはしません。でき上がった物は `target\debug\` に入ります。`cargo build --release` だと、速く動く本番用の物が `target\release\` にできます。	07 6. async と、会社のサーバーへの持ち込み
`cargo build --release`	`cargo build --release` は、時間をかけて最適化した、速い本番用の実行ファイルを作ります（動かしはしません）。できた物は `target\release` に置かれます。	07 6. async と、会社のサーバーへの持ち込み
`cargo build --locked`	`--locked` は、`Cargo.lock` に書いてある部品の版のまま作り、`Cargo.lock` を書きかえない指定です。版がずれていたらエラーで止まるので、いつ作っても同じ物ができます。本番用やテストの時に付けます。	07 6. async と、会社のサーバーへの持ち込み
`cargo run --`	`cargo run -- Rust "Web アプリ"` の `--` は、「ここから後ろは、Cargo ではなく、動かすプログラムに渡す」という区切りです。後ろの値は、プログラムの中で `std::env::args()` で受け取れます。	08 前編 2. 開発の準備
`cargo run -- --to`	`cargo run -- send-test-mail --to 自分のアドレス` の `--to` は、練習アプリのコマンドに、テストのメールを送る宛先を渡すオプションです。	09 8. 手元で、アプリから送ってみる

設定ファイル（TOML）

コード	意味と使い方	初めて出てくる所
`[package]`	`Cargo.toml` の `[package]` の下には、このプロジェクト（パッケージ）自身のこと（名前・版・Rust の版の決まり）を書きます。`[ ]` で囲んだ行は「ここから、この組の設定」という見出し（テーブル）です。	07 5. Topcoat の最小のアプリ
`name =`	`name = "practice"` は、パッケージの名前です。`cargo run` で作られる実行ファイルの名前（`practice.exe`）にもなります。	07 5. Topcoat の最小のアプリ
`version =`	`version = "0.1.0"` は、このパッケージの版です。直したら数を上げます（「大きな変更.機能の追加.直し」の 3 つの数）。	07 5. Topcoat の最小のアプリ
`edition =`	`edition = "2024"` は、「Rust の書き方の決まりの、どの年の版を使うか」です。2015・2018・2021・2024 があり、新しく作る時は `cargo new` が最新を書きます。	07 5. Topcoat の最小のアプリ
`#`	TOML（`Cargo.toml` などの設定ファイルの書き方）では、`#` から行の終わりまでがコメントです。	07 5. Topcoat の最小のアプリ

コード	意味と使い方	初めて出てくる所
`[dependencies]`	`[dependencies]` の下には、このプロジェクトが使う部品（クレート）を、`名前 = "版"` の形で 1 行ずつ書きます。`cargo add 名前` を打つと、ここに書き足されます。	07 5. Topcoat の最小のアプリ
`"1.0"`	部品の版の `"1.0.229"` は、「1.0.229 以上で、2.0.0 より前」を表します（新しい直しは自動で使われてよい）。`"=1.0.229"` のように `=` を付けると、「ぴったりこの版だけ」になります。実際に使う版は、`Cargo.lock` に記録されます。	07 5. Topcoat の最小のアプリ
`{ キー = 値 }`	`{ version = "1.0", features = ["derive"] }` のように `{ }` で囲むと、1 行の中に、いくつもの設定をまとめて書けます（インラインテーブル）。	07 5. Topcoat の最小のアプリ
`version =`	部品の `{ version = "1.51.1", … }` の `version` は、使う部品の版です。	07 5. Topcoat の最小のアプリ
`features =`	部品の `features = ["macros", "rt-multi-thread"]` は、その部品の、どの機能（feature）を ON にして使うかです。部品は、使わない機能を OFF にして小さくできるようになっています。	07 5. Topcoat の最小のアプリ
`default-features =`	`default-features = false` は、部品の「既定で ON の機能」を全部 OFF にする指定です。そのうえで、要る機能だけを `features = […]` で選びます。	07 5. Topcoat の最小のアプリ
`[profile.dev]`	`[profile.dev]` は、ふだんの `cargo build`・`cargo run`（debug）の時の、作り方の設定です。	07 5. Topcoat の最小のアプリ
`debug =`	`debug = "line-tables-only"` は、実行ファイルに入れる「調べるための情報」を、エラーの行番号が分かる分だけにする設定です。翻訳が速くなり、`target` フォルダも小さくなります。	07 5. Topcoat の最小のアプリ
`[profile.dev.package."*"]`	`[profile.dev.package."*"]` は、「自分のコード以外の、全部の部品」の作り方の設定です。部品だけ最適化（`opt-level = 2` など）すると、自分のコードの翻訳は速いまま、動きを速くできます。	07 5. Topcoat の最小のアプリ
`rust-version =`	`rust-version = "1.99"` は、「このパッケージを作るのに要る、いちばん古い Rust の版」です。古い Rust で作ろうとすると、分かりやすいエラーで止まります。	08 前編 2. 開発の準備
`description =`	`description = "…"` は、パッケージの短い説明です。	08 前編 2. 開発の準備
`publish =`	`publish = false` は、「このパッケージは crates.io（みんなで部品を共有する所）に公開しない」という印です。間違って公開するのを防ぎます。	08 前編 2. 開発の準備

Rust の部品（クレート）

コード	意味と使い方	初めて出てくる所
`serde`	`serde` は、Rust の値を、JSON などの形に変えたり（シリアライズ）、JSON から Rust の値を作ったり（デシリアライズ）するための部品です。`features = ["derive"]` で、`#[derive(Serialize, Deserialize)]` が使えるようになります。	07 5. Topcoat の最小のアプリ
`tokio`	`tokio` は、Rust で非同期（待ち時間の間に、ほかの仕事を進める）のプログラムを動かすための部品です。たくさんの人の要求を、同時にさばく Web のサーバーの土台になります。	07 5. Topcoat の最小のアプリ
`topcoat`	`topcoat` は、Rust の Web アプリを作るための部品（フレームワーク）です。画面（`view!`）・URL ごとの処理（`#[page]`）・セッションなどを用意しています。	07 5. Topcoat の最小のアプリ
`anyhow`	`anyhow` は、いろいろな種類のエラーを、1 つの型（`anyhow::Error`）でまとめて扱い、説明を書き足せる（`.context(…)`）部品です。アプリの一番上の、エラーを表示する所で使います。	08 前編 2. 開発の準備
`argon2`	`argon2` は、パスワードを、元に戻せない形（ハッシュ）にして保存するための部品です。わざと時間とメモリを使う計算なので、盗まれても、元のパスワードを当てにくくなります。	08 前編 2. 開発の準備
`csv`	`csv` は、CSV（`,` で区切った表の文字）のファイルを読み書きする部品です。	08 前編 2. 開発の準備
`dotenvy`	`dotenvy` は、`.env` のファイルに書いた `名前=値` を読み、環境変数にする部品です。開発の時の設定に使います。	08 前編 2. 開発の準備
`getrandom`	`getrandom` は、OS から、予想できない安全な乱数を取ってくる部品です。セッションの合言葉（トークン）を作る時に使います。	08 前編 2. 開発の準備
`hex`	`hex` は、バイトの並びを、16 進数の文字（`a3f0…`）に変えたり戻したりする部品です。	08 前編 2. 開発の準備
`serde_json`	`serde_json` は、`serde` と組み合わせて、JSON を読み書きする部品です。	08 前編 2. 開発の準備
`sha2`	`sha2` は、SHA-256 などの、データの指紋（ハッシュ）を計算する部品です。	08 前編 2. 開発の準備
`sqlx`	`sqlx` は、PostgreSQL などのデータベースに、Rust からつないで SQL を送る部品です。	08 前編 2. 開発の準備

コード	意味と使い方	初めて出てくる所
`time`	`time` は、日付・時刻を扱う部品です（期限の計算や、`2026-11-05` の形への変換など）。	08 前編 2. 開発の準備

HTML

コード	意味と使い方	初めて出てくる所
`<html>`	HTML の文書全体を包むタグ。`<html lang="ja">` の `lang="ja"` は「日本語の文書」という印で、読み上げソフトや翻訳に使われます。中に `<head>`（ページの情報）と `<body>`（画面に出す中身）を入れます。	01 2. HTML の基本
`<head>`	ページの「情報」を書く所。題名（`<title>`）・文字コード（`<meta charset="utf-8">`）・CSS の読み込み（`<link>`）などを書きます。ここに書いた物は、画面の本文には出ません。	01 2. HTML の基本
`<meta>`	ページの情報を書くタグ。`<meta charset="utf-8">` は文字コード（日本語が化けないように必ず書く）、`<meta name="viewport" content="width=device-width, initial-scale=1">` はスマホでも画面の幅に合わせて表示させる決まり文句です。閉じタグはありません。	01 2. HTML の基本
`<title>`	ブラウザーのタブに出る、ページの題名のタグ。`<head>` の中に書きます。	01 2. HTML の基本
`<body>`	画面に出す中身を全部入れるタグ。見出し・段落・フォームなどは、全部この中に書きます。	01 2. HTML の基本
`<h1>`	いちばん大きな見出し（heading 1）のタグ。数が大きいほど小さな見出しになります（`<h2>` 〜 `<h6>`）。`<h1>` は、1 ページに 1 つにします。	01 2. HTML の基本
`<p>`	段落（paragraph）のタグ。ひとまとまりの文を囲みます。段落と段落の間には、自動で少し間が空きます。	01 2. HTML の基本
`<!DOCTYPE html>`	`<!DOCTYPE html>` は、「これは今の HTML（HTML5）で書いたページです」という、ページのいちばん最初に書く宣言です。大文字でも小文字（`<!doctype html>`）でもかまいません。	01 2. HTML の基本
`lang=`	`<html lang="ja">` の `lang` は、ページの言葉（`ja` は日本語、`en` は英語）です。読み上げソフトの読み方や、ブラウザーの翻訳の判断に使われます。	01 2. HTML の基本

コード	意味と使い方	初めて出てくる所
`charset=`	`<meta charset="utf-8">` の `charset` は、ページの文字の符号（エンコーディング）です。いつも `utf-8` にしておくと、日本語が文字化けしません。	01 2. HTML の基本
`class=`	`class` は、タグに付ける「グループの名前」です。CSS で `.note { … }` と書くと、`class="note"` の物全部に、同じ見た目が付きます。`class="note done"` のように、空白で区切っていくつも付けられます。	01 2. HTML の基本
`<link>`	ほかのファイルとのつながりを書くタグ。`<link rel="stylesheet" href="style.css">` で、CSS のファイルを読み込みます。`<head>` の中に書き、閉じタグはありません。	01 3. フォーム: 依頼の入力画面
`<header>`	ページの上の帯（アプリの名前・メニューなど）を表すタグ。見た目は CSS で決めます。意味の付いたタグを使うと、読み上げソフトなどにも構造が伝わります。	01 3. フォーム: 依頼の入力画面
`<strong>`	「大事」という意味を付けるタグ。ふつうは太字で表示されます。	01 3. フォーム: 依頼の入力画面
`<nav>`	メニュー（ほかのページへのリンクのまとまり）を表すタグ。	01 3. フォーム: 依頼の入力画面
`<a>`	リンクのタグ。`<a href="URL">文字</a>` で、押すとその URL へ移ります。	01 3. フォーム: 依頼の入力画面
`<main>`	ページの「いちばん大事な中身」を表すタグ。ヘッダーやメニューを除いた本体を囲みます。	01 3. フォーム: 依頼の入力画面
`<form>`	入力欄とボタンをまとめて、送る（サーバーへ届ける）ためのタグ。`action` に送り先、`method="post"` に送り方を書きます。	01 3. フォーム: 依頼の入力画面
`<div>`	特別な意味を持たない「箱」のタグ。CSS で見た目や並べ方を決めるために、ほかのタグをまとめる時に使います。	01 3. フォーム: 依頼の入力画面
`<label>`	入力欄の名前を表すタグ。`<label for="title">件名</label>` の `for` と、入力欄の `id` を同じにすると、名前を押しても入力欄が選ばれ、読み上げソフトにも伝わります。	01 3. フォーム: 依頼の入力画面
`<input>`	1 行の入力欄のタグ。`type` で種類（`text`・`email`・`password`・`hidden` など）を、`name` で送る時の名前を決めます。閉じタグはありません。	01 3. フォーム: 依頼の入力画面
`<textarea>`	何行も書ける入力欄のタグ。`<textarea name="detail" rows="4"></textarea>` の `rows` は、見えている行の数です。最初から入れておく文字は、タグの間に書きます。	01 3. フォーム: 依頼の入力画面

コード	意味と使い方	初めて出てくる所
`<select>`	選ぶ欄（プルダウン）のタグ。中に `<option value="値">表示する文字</option>` を並べます。`selected` を付けた物が、最初に選ばれます。	01 3. フォーム: 依頼の入力画面
`<option>`	`<select>`（選ぶ欄）の中の、選べる 1 つ。`<option value="new">新しく作る</option>` の `value` が送られる値、間の文字が画面に出る文字です。`selected` で最初に選ばれます。	01 3. フォーム: 依頼の入力画面
`<button>`	ボタンのタグ。フォームの中の `<button type="submit">送る</button>` は、押すとフォームを送ります。`type="button"` にすると、送らない、ふつうのボタンになります。	01 3. フォーム: 依頼の入力画面
`name=`	`name` は、入力欄（`<input>`・`<textarea>`・`<select>`）の名前です。フォームを送ると、`title=牛乳` のように、この名前と入れた値の組がサーバーに届きます。`<meta name="viewport" …>` の `name` は、設定の種類の名前です。	01 3. フォーム: 依頼の入力画面
`content=`	`<meta name="viewport" content="width=device-width, initial-scale=1">` の `content` は、その設定の値です。この `viewport` の設定は、「スマホでも、画面の幅に合わせて表示する」という意味で、いつも書いておきます。	01 3. フォーム: 依頼の入力画面
`rel=`	`<link rel="stylesheet" …>` の `rel` は、つなぐファイルが「何か」です。`stylesheet` は CSS のファイル、`icon` はタブの小さな絵です。	01 3. フォーム: 依頼の入力画面
`href=`	`href` は、リンク先や、読み込むファイルの場所（URL）です。`<a href="/notes">一覧</a>` は、押すと `/notes` へ移るリンクです。	01 3. フォーム: 依頼の入力画面
`action=`	`<form action="/notes">` の `action` は、フォームの送り先の URL です。書かなければ、今のページに送ります。	01 3. フォーム: 依頼の入力画面
`method=`	`<form method="post">` の `method` は、フォームの送り方です。`post` はデータを送って何かを変える時（登録・削除）、`get` は検索のように、URL の `?q=…` に入れて送る時に使います。	01 3. フォーム: 依頼の入力画面
`enctype=`	`<form enctype="multipart/form-data">` の `enctype` は、フォームの送り方の形です。ファイルを送るフォームでは、必ずこの `multipart/form-data` にします。	01 3. フォーム: 依頼の入力画面
`for=`	`<label for="q">` の `for` は、この説明の文字が、どの入力欄（`id="q"`）のための物かを結び付けます。文字を押しても、その欄に入力できるようになり、読み上げソフトも欄の名前を読めます。	01 3. フォーム: 依頼の入力画面
`id=`	`id` は、ページの中で 1 つだけの名前です。`<label for="title">` で欄と結び付けたり、`#title` のリンクでその場所へ飛んだりできます。同じ `id` を 2 つ付けてはいけません。	01 3. フォーム: 依頼の入力画面

コード	意味と使い方	初めて出てくる所
`type=`	`<input type="…">` の `type` は、入力欄の種類です。`text`（文字）・`email`・`password`（伏せ字）・`number`・`date`・`checkbox`（チェック）・`file`（ファイル）・`search` などがあります。`<button type="submit">` は、押すとフォームを送るボタンです。	01 3. フォーム: 依頼の入力画面
`maxlength=`	`maxlength="100"` は、入力できる文字の数の上限です。それより長くは打てません（サーバー側でも、必ず確かめます）。	01 3. フォーム: 依頼の入力画面
`required`	`required` は、「この欄は必ず入れる」という印です。値は書かず、名前だけで ON になります。空のまま送ろうとすると、ブラウザーが止めます。	01 3. フォーム: 依頼の入力画面
`placeholder=`	`placeholder` は、入力欄が空の時に、うすく見せる例の文字です。入力の説明の代わりにはならないので、`<label>` も付けます。	01 3. フォーム: 依頼の入力画面
`rows=`	`<textarea rows="6">` の `rows` は、見える行の数です。	01 3. フォーム: 依頼の入力画面
`accept=`	`<input type="file" accept=".jpg,.png,.pdf">` の `accept` は、選べるファイルの種類です。ファイルを選ぶ画面で、ほかの種類が見えにくくなります（ブラウザーの手助けなので、サーバー側でも必ず確かめます）。	01 3. フォーム: 依頼の入力画面
`multiple`	`<input type="file" multiple>` の `multiple` は、ファイルを一度にいくつも選べるようにする印です。	01 3. フォーム: 依頼の入力画面
`aria-*=`	`aria-label="…"`・`aria-current="page"` のように `aria-` で始まる属性は、読み上げソフトなどの手助けの道具に、見た目では分からないことを伝える印です。`aria-label` は名前、`aria-current="page"` は「今開いているページ」、`aria-invalid="true"` は「入力に間違いがある」です。	01 3. フォーム: 依頼の入力画面
`value=`	`value` は、入力欄の最初の値や、選び肢（`<option>`）を選んだ時に送られる値です。	01 3. フォーム: 依頼の入力画面
`<table>`	表のタグ。中に `<thead>`（見出しの行）と `<tbody>`（中身の行）を入れ、行は `<tr>`、ますは `<th>`（見出し）か `<td>`（中身）で作ります。	01 4. 表: 依頼の一覧
`<thead>`	表の見出しの行をまとめるタグ。	01 4. 表: 依頼の一覧
`<tr>`	表の 1 行（table row）のタグ。中に `<th>` や `<td>` を並べます。	01 4. 表: 依頼の一覧
`<th>`	表の見出しのます（table header）のタグ。ふつうは太字で表示されます。	01 4. 表: 依頼の一覧

コード	意味と使い方	初めて出てくる所
`<tbody>`	表の中身の行をまとめるタグ。	01 4. 表: 依頼の一覧
`<td>`	表の中身のます（table data）のタグ。	01 4. 表: 依頼の一覧
`<span>`	文の一部分を囲む、特別な意味の無いタグ。CSS で、その部分だけ色を変える時などに使います（`<div>` と違い、改行しません）。	01 4. 表: 依頼の一覧
`<input>`	1 行の入力欄のタグ。`type` で種類（`text`・`email`・`password`・`hidden` など）を、`name` で送る時の名前を決めます。閉じタグはありません。	03 4. 一覧の画面: 読み込みと、しぼり込み
`<form>`	入力欄とボタンをまとめて、送る（サーバーへ届ける）ためのタグ。`action` に送り先、`method="post"` に送り方を書きます。	03 5. 入力フォーム
`<label>`	入力欄の名前を表すタグ。`<label for="title">件名</label>` の `for` と、入力欄の `id` を同じにすると、名前を押しても入力欄が選ばれ、読み上げソフトにも伝わります。	03 5. 入力フォーム
`<main>`	ページの「いちばん大事な中身」を表すタグ。ヘッダーやメニューを除いた本体を囲みます。	03 7. 画面の切りかえと、部品の組み立て
`colspan=`	`<td colspan="4">` の `colspan` は、表の 1 つのマスを、横に何列分つなげるかです。	05 7. HTML の画面
`<h2>`	2 番目の大きさの見出しのタグ。	06 前編 6. 1 回限りの承認の URL
`<svg>`	図（線や四角）を描くタグ。中に `<rect>`（四角）・`<line>`（線）などを並べます。	06 前編 6. 1 回限りの承認の URL
`<rect>`	SVG の中で、四角を描くタグ。`x`・`y`（左上の位置）・`width`・`height`（大きさ）で決めます。	06 前編 6. 1 回限りの承認の URL
`viewBox=`	`<svg viewBox="0 0 400 300">` の `viewBox` は、SVG の絵の中の座標の範囲です（左上の x・y と、幅・高さ）。表示の大きさが変わっても、絵はこの範囲に合わせて伸び縮みします。	06 前編 6. 1 回限りの承認の URL
`x=`	SVG の `x`・`y` は、図形（`<rect>`・`<text>`）を置く位置です。左上が 0 で、右と下へ行くほど大きくなります。	06 前編 6. 1 回限りの承認の URL

コード	意味と使い方	初めて出てくる所
`y=`	SVG の `y` は、図形を置く縦の位置です（上が 0）。	06 前編 6. 1 回限りの承認の URL
`width=`	`width` は、幅です（SVG の `<rect>` なら四角の幅）。	06 前編 6. 1 回限りの承認の URL
`height=`	`height` は、高さです。	06 前編 6. 1 回限りの承認の URL
`<text>`	SVG（線や図形で描く絵）の中に、文字を書くタグ。`<text x="110" y="18" textAnchor="end">ラベル</text>` は、決めた位置に、右そろえで文字を書きます。	06 後編 7. グラフ
`DOCTYPE`	`<!DOCTYPE html>` は、「この文書は HTML5 です」とブラウザーに伝える、HTML の先頭の決まった 1 行です。	07 5. Topcoat の最小のアプリ
`required=`	`required` は、「この欄は必ず入れる」という印です。空のまま送ろうとすると、ブラウザーが止めて知らせます（サーバー側でも、必ず確かめます）。	07 5. Topcoat の最小のアプリ
`<ul>`	順番の無い並び（点の付いた並び）のタグ。中に `<li>` を並べます。	08 前編 7. 1 回限りの承認の URL
`<li>`	並び（`<ul>`・`<ol>`）の 1 つのタグ。	08 前編 7. 1 回限りの承認の URL
`role=`	`role="status"` は、その部分の役わりを、読み上げソフトに伝える印です。`status` は「お知らせ」で、変わった時に読み上げられます。`role="alert"` は、すぐ知らせる大事なお知らせです。	08 前編 7. 1 回限りの承認の URL
`<small>`	`<small>` は、補足の小さな文字（注意書き・おまけの情報）を表すタグです。	08 後編 1. 画面の並び
`<line>`	`<line>` は、SVG（線や図形で描く絵）の中で、直線を引くタグです。`x1`・`y1`（始まり）と `x2`・`y2`（終わり）の位置を書きます。	08 後編 2. 画面の部品と、XSS を防ぐ仕組み
`x1=`	SVG の `<line>` の `x1`・`y1` は線の始まりの位置、`x2`・`y2` は終わりの位置です。	08 後編 2. 画面の部品と、XSS を防ぐ仕組み

コード	意味と使い方	初めて出てくる所
`y1=`	SVG の `<line>` の `y1` は、線の始まりの縦の位置です。	08 後編 2. 画面の部品と、XSS を防ぐ仕組み
`x2=`	SVG の `<line>` の `x2` は、線の終わりの横の位置です。	08 後編 2. 画面の部品と、XSS を防ぐ仕組み
`y2=`	SVG の `<line>` の `y2` は、線の終わりの縦の位置です。	08 後編 2. 画面の部品と、XSS を防ぐ仕組み
`<text>`	SVG（線や図形で描く絵）の中に、文字を書くタグ。`<text x="110" y="18" textAnchor="end">ラベル</text>` は、決めた位置に、右そろえで文字を書きます。	08 後編 2. 画面の部品と、XSS を防ぐ仕組み
`autocomplete=`	`autocomplete="email"` は、ブラウザーに、「この欄にはメールアドレスを入れる」と伝え、覚えている値を出してもらう印です。`username`・`current-password`・`new-password` などがあります。	08 後編 3. ログインの画面
`inputmode=`	`inputmode="numeric"` は、スマホで入力する時に、数字のキーボードを出してもらう印です。	08 後編 3. ログインの画面
`selected=`	`<option selected>` の `selected` は、その選び肢を、最初から選んだ状態にする印です。	08 後編 5. 入力フォーム
`multiple=`	`multiple` は、ファイル（や選び肢）を一度にいくつも選べるようにする印です。	08 後編 5. 入力フォーム
`<script>`	`<script src="drawing.js"></script>` は、ページで動かす JavaScript のファイルを読み込むタグです。	08 後編 5. 入力フォーム
`src=`	`src` は、読み込むファイル（画像・スクリプト）の場所です。	08 後編 5. 入力フォーム
`style=`	`style="display:inline"` は、そのタグだけに、直接 CSS を書く属性です。いくつもの所で使う見た目は、CSS のファイルに書きます。	08 後編 7. 依頼の詳しい画面と、ボタンの操作
`<`	`<` は、`<` という文字そのものを表す書き方（文字参照）です。HTML では `<` がタグの始まりの印なので、文字として見せたい時は `<` と書きます。`>` は `>`、`&` は `&`、`"` は `"` です。入力された文字をこの形に変えておくと、`<script>` を書かれても、ただの文字として表示されます（エスケープ）。	10 6. XSS: 画面に、スクリプトを動かされる

コード	意味と使い方	初めて出てくる所
`>`	`>` は、`>` という文字そのものを表す書き方（文字参照）です。	10 6. XSS: 画面に、スクリプトを動かされる
`<style>`	HTML の中に、直接 CSS を書くタグ。ふつうは `<head>` の中に書きます（大きくなったら、別の CSS のファイルに分けます）。	11 B. サンプル集
`<em>`	「強めたい」という意味のタグ。ふつうは斜めの字で表示されます。	11 B. サンプル集
` `	改行のタグ。閉じタグはありません。段落を分けたい時は、` ` を並べるより `<p>` を分けます。	11 B. サンプル集
`<!-- -->`	HTML では、`<!--` から `-->` までがコメントで、画面には出ません。	11 B. サンプル集
`min=`	`<input type="number" min="1" max="99">` の `min` は、入れられる一番小さな数です。	11 B. サンプル集
`max=`	`max` は、入れられる一番大きな数です。	11 B. サンプル集
`selected`	`selected` は、その選び肢（`<option>`）を、最初から選んだ状態にする印です。	11 B. サンプル集
`cols=`	`<textarea cols="40">` の `cols` は、見える幅（だいたいの文字数）です。	11 B. サンプル集
`checked`	`checked` は、チェックボックスが最初からチェックされた状態にする印です。値は書かず、名前だけで ON になります。	11 B. サンプル集
`<b>`	文字を太くするタグ（特別な意味は無く、見た目だけ）。「大事」という意味を付ける時は `<strong>` を使います。	11 B. サンプル集

CSS

コード	意味と使い方	初めて出てくる所
`navy:`	この資料の CSS で決めた色の変数（`--navy`：濃紺。文字や線の色）。`:root { --navy: #1e3a5f; }` のように、名前の頭を `--` にして決め、`color: var(--navy);` のように使います。色を 1 か所で決めておくと、あとで変える時に 1 か所直すだけで済みます。	01 5. CSS の基本
`blue:`	この資料の CSS で決めた色の変数（`--blue`：青。ボタンやリンクの色）。`var(--blue)` で使います。	01 5. CSS の基本
`color:`	文字の色を決める項目。`color: var(--navy);` や `color: #eb5757;` のように書きます。	01 5. CSS の基本

コード	意味と使い方	初めて出てくる所
`:root`	CSS の `:root` は、ページ全体（いちばん外側の `<html>`）を選ぶ書き方です。ページのどこからでも使う色などの変数（`--color-text`）を、ここに書きます。	01 5. CSS の基本
`--名前`	CSS の `--color-text: #0f172a;` のように `--` で始まる名前は、自分で決める変数（カスタムプロパティ）です。`color: var(--color-text);` で使います。色を 1 か所で決めておけば、変える時も 1 か所で済みます。	01 5. CSS の基本
`#rrggbb`	CSS の `#0f172a` のような `#` と 6 けた（または 3 けた）の 16 進数は、色の書き方です。2 けたずつ、赤・緑・青の強さ（00〜ff）を表します。`#ffffff` は白、`#000000` は黒です。	01 5. CSS の基本
`var()`	CSS の `var(--color-text)` は、`--` で始まる変数の値を使う書き方です。	01 5. CSS の基本
`display:`	箱の並べ方を決める項目。`display: flex;` は「中身を横一列に並べる」、`display: grid;` は「ます目に並べる」、`display: none;` は「表示しない」です。	01 7. 並べ方: flex と grid
`align-items:`	`display: flex` の中身を、上下（横に並べた時の縦）のどこにそろえるかを決める項目。`center` で真ん中にそろえます。	01 7. 並べ方: flex と grid
`justify-content:`	`display: flex` の中身を、並ぶ向きにどう配るかを決める項目。`space-between` は「両端に寄せて、間を均等に空ける」。	01 7. 並べ方: flex と grid
`/* */`	CSS では、`/*` から `*/` までがコメントです。	01 7. 並べ方: flex と grid
`.クラス名`	CSS の `.note { … }` のように `.` で始まる名前は、「`class="note"` の付いたタグ全部」を選ぶ書き方（クラスセレクター）です。`{ }` の中の見た目が、その全部に付きます。	01 7. 並べ方: flex と grid
`grid-template-columns:`	`display: grid` の、列の幅を決める項目。`160px 1fr` は「1 列目は 160px、2 列目は残り全部」。`fr` は「残りを分ける割合」です。	01 7. 並べ方: flex と grid
`gap:`	`flex` や `grid` の中身どうしの間の幅を決める項目。`gap: 12px 16px;` は「行の間 12px、列の間 16px」。	01 7. 並べ方: flex と grid
`数px`	CSS の `16px` の `px` は、画面の点（ピクセル）の数で表す大きさの単位です。	01 7. 並べ方: flex と grid
`数fr`	CSS のグリッドの `1fr` は、「残りの場所を分ける割合」の単位です。`grid-template-columns: 1fr 2fr;` なら、残りを 1 対 2 で分けた 2 列になります。	01 7. 並べ方: flex と grid

コード	意味と使い方	初めて出てくる所
`max-width:`	`@media (max-width: 640px) { … }` は、「画面の幅が 640px 以下の時だけ、中の決まりを使う」という書き方（スマホ向けの見た目）。箱に書く `max-width: 760px;` は、「幅の上限」です。	01 8. 画面の幅に合わせる
`margin:`	箱の外側の余白を決める項目。`margin: 12px;` は上下左右に 12px。2 つ書くと「上下 左右」、4 つ書くと「上 右 下 左」の順です。	01 8. 画面の幅に合わせる
`padding:`	箱の内側の余白（枠と中身の間）を決める項目。書き方は `margin` と同じです。	01 8. 画面の幅に合わせる
`flex-direction:`	`display: flex` の中身を並べる向きを決める項目。`row` は横（最初の値）、`column` は縦に並べます。	01 8. 画面の幅に合わせる
`@media`	CSS の `@media (max-width: 640px) { … }` は、「画面の幅が 640px 以下の時だけ」の見た目を書く所です（メディアクエリ）。スマホでは 1 列にする、などに使います。	01 8. 画面の幅に合わせる
`(max-width)`	`@media` の `(max-width: 640px)` は、「画面の幅が 640px 以下」という条件です。`(min-width: …)` なら「以上」、`(prefers-color-scheme: dark)` なら「暗い色の設定の時」です。	01 8. 画面の幅に合わせる
`minmax()`	CSS の `minmax(0, 1fr)` は、グリッドの列の幅を「いちばん小さくて 0、いちばん大きくて残りの 1 つ分」の範囲で決める書き方です。中身が長くても、列がはみ出さなくなります。	01 8. 画面の幅に合わせる
`green:`	この資料の CSS で決めた色の変数（`--green`：緑。完了・成功の色）。	11 B. サンプル集
`orange:`	この資料の CSS で決めた色の変数（`--orange`：オレンジ。差し戻しなどの注意の色）。	11 B. サンプル集
`red:`	この資料の CSS で決めた色の変数（`--red`：赤。エラーや、消すボタンの色）。	11 B. サンプル集
`gray:`	この資料の CSS で決めた色の変数（`--gray`：薄い灰色。背景の色）。	11 B. サンプル集
`box-sizing:`	CSS で、箱の幅に枠線と内側の余白を含めるかを決める項目。`box-sizing: border-box;` にすると、`width` の中に `padding` と `border` が含まれ、大きさの計算が分かりやすくなります。	11 B. サンプル集
`font-family:`	CSS で、文字の書体（フォント）を決める項目。`font-family: sans-serif;` は「飾りの無い書体」。	11 B. サンプル集
`background:`	CSS で、背景（色や画像）を決める項目。`background: var(--color-bg);` は、色の変数の色にします。	11 B. サンプル集

コード	意味と使い方	初めて出てくる所
`border-radius:`	CSS で、箱の角の丸みを決める項目。	11 B. サンプル集
`text-align:`	CSS で、文字の左右のそろえ方を決める項目。`center` で中央。	11 B. サンプル集
`font-size:`	CSS で、文字の大きさを決める項目。	11 B. サンプル集
`border:`	CSS で、箱の枠線（太さ・線の種類・色）をまとめて決める項目。`border: 1px solid #e2e8f0;` 。	11 B. サンプル集
`cursor:`	CSS で、マウスを乗せた時の矢印の形を決める項目。`pointer` は、押せる物の「指の形」です。	11 B. サンプル集
`btn:`	`.btn:hover` は、この資料で決めたボタンの class（ `btn` ）の、マウスを乗せている時の見た目です（ `:hover` は乗せている時の状態）。	11 B. サンプル集
`opacity:`	透明さを決める項目。`1` で不透明、`0` で見えない。`0.85` は少しだけ透ける（ボタンにマウスを乗せた時の合図など）。	11 B. サンプル集
`*`	CSS の `*` は、「全部のタグ」を選ぶ書き方です。`* { box-sizing: border-box; }` は、全部のタグの幅の数え方をそろえます。	11 B. サンプル集
`A B`	CSS の `.note-list li` のように、空白で区切って並べると、「`.note-list` の**中にある** `li` 」を選びます（子孫セレクター）。	11 B. サンプル集
`:hover`	CSS の `button:hover` の `:hover` は、「マウスが乗っている間だけ」の見た目です。`:focus` は「キーボードで選ばれている間」、`:disabled` は「使えない時」です。	11 B. サンプル集

コメント

コード	意味と使い方	初めて出てくる所
`//!`	`//!` で始まるコメントは、そのファイル（モジュール）全体の説明書です。ファイルのいちばん上に書きます。`///` （すぐ下の関数や型の説明）と使い分けます。	07 2. 依頼の状態を enum で書く
`///`	`///` で始まるコメントは、すぐ下の関数や型の「説明書」（ドキュメントコメント）です。`cargo doc` で説明のページになり、VS Code では、名前にマウスを乗せると表示されます。ふつうのコメント（ `//` ）とちがい、何の説明かが決まっています。	07 2. 依頼の状態を enum で書く
`//`	`//` から行の終わりまでは「コメント」（人が読むためのメモ）で、プログラムとしては無視されます。`// OS の種類を表示する` のように、何をしているかを日本語で書いておくと、あとで読み返す時に助かります。	07 2. 依頼の状態を enum で書く

文字列の中の特別な書き方

コード	意味と使い方	初めて出てくる所
`\n`	文字列の中の `\n` は「改行」を表す書き方です。`\`（バックスラッシュ。日本語のキーボードやフォントでは `¥` に見えることがあります）と `n` の 2 文字で、改行 1 つを表します。`"1行目\n2行目"` を表示すると 2 行になります。	02 4. 配列の道具: 絞る・変える・まとめる
`\\`	文字列の中に `\` そのもの（1 つ）を入れたい時は、`\\` と 2 つ書きます。`\` は、`\n` のような特別な書き方の始まりの印だからです。Windows の場所を `"C:\\Users"` と書くのも、この理由です（生の文字列 `r"C:\Users"` なら 1 つで済みます）。	05 6. ORM: Python でデータベースを操作する
`\n`	文字列の中の `\n` は「改行」を表す書き方です。`\`（バックスラッシュ。日本語のキーボードやフォントでは `¥` に見えることがあります）と `n` の 2 文字で、改行 1 つを表します。`"1行目\n2行目"` を表示すると 2 行になります。	06 前編 5. 状態を変える処理を 1 か所に集める
`\x…`	`\xff` のように `\x` の後ろに 16 進数を 2 けた書くと、その番号の 1 バイト（文字列なら 1 文字）を表します。`b"\xef\xbb\xbf"` は、UTF-8 のファイルの先頭に付くことがある、BOM という 3 バイトです。	06 前編 7. 図面と添付ファイルの確かめ
`\r`	文字列の中の `\r` は「行の頭に戻る」（CR）という、目に見えない文字です。Windows のファイルや HTTP の通信では、行の終わりを `\r\n`（CR と LF の 2 文字）で表します。	06 前編 7. 図面と添付ファイルの確かめ
`\n`	文字列の中の `\n` は「改行」を表す書き方です。`\`（バックスラッシュ。日本語のキーボードやフォントでは `¥` に見えることがあります）と `n` の 2 文字で、改行 1 つを表します。`"1行目\n2行目"` を表示すると 2 行になります。	08 前編 6. 状態を変える処理
`\r`	文字列の中の `\r` は「行の頭に戻る」（CR）という、目に見えない文字です。Windows のファイルや HTTP の通信では、行の終わりを `\r\n`（CR と LF の 2 文字）で表します。	08 前編 8. 社員の CSV と、管理のコマンド
`\x…`	`\xff` のように `\x` の後ろに 16 進数を 2 けた書くと、その番号の 1 バイト（文字列なら 1 文字）を表します。`b"\xef\xbb\xbf"` は、UTF-8 のファイルの先頭に付くことがある、BOM という 3 バイトです。	08 前編 8. 社員の CSV と、管理のコマンド
`\"`	文字列の中に `"` そのものを入れたい時は、`\"` と書きます。`"\"Rust\""` は、`"Rust"` という `"` 付きの文字になります。`\` を付けないと、そこで文字列が終わったと思われます。	08 前編 8. 社員の CSV と、管理のコマンド

コード	意味と使い方	初めて出てくる所
`\\`	文字列の中に `\` そのもの（1 つ）を入れたい時は、`\\` と 2 つ書きます。`\` は、`\n` のような特別な書き方の始まりの印だからです。Windows の場所を `"C:\\Users"` と書くのも、この理由です（生の文字列 `r"C:\Users"` なら 1 つで済みます）。	08 後編 8. 添付ファイル

正規表現

コード	意味と使い方	初めて出てくる所
`正規表現`	正規表現は、「こういう形の文字」を表すための、小さな書き方の決まりです。`^\s*#` は「行の頭から、空白がいくつかあって、`#` が来る」という形を表します。`-match`（PowerShell）・`re.match`（Python）・`/…/`（JavaScript）・`Regex::new`（Rust）などで、文字がその形に合うかを調べたり、合う所を取り出したりします。記号ごとの意味は、この一覧の「正規表現」の表にまとめています。	03 4. 一覧の画面: 読み込みと、しぼり込み
`正規表現`	正規表現は、「こういう形の文字」を表すための、小さな書き方の決まりです。`^\s*#` は「行の頭から、空白がいくつかあって、`#` が来る」という形を表します。`-match`（PowerShell）・`re.match`（Python）・`/…/`（JavaScript）・`Regex::new`（Rust）などで、文字がその形に合うかを調べたり、合う所を取り出したりします。記号ごとの意味は、この一覧の「正規表現」の表にまとめています。	04 5. 社員の CSV を読み込む
`\d`	正規表現の `\d` は、「数字 1 つ（0〜9）」を表します。	04 5. 社員の CSV を読み込む
`{n,m}`	正規表現の `{6}` は「すぐ前の物が、ちょうど 6 回」、`{8,12}` は「8 回から 12 回」を表します。`\d{6}` は「数字 6 けた」。	04 5. 社員の CSV を読み込む
`正規表現`	正規表現は、「こういう形の文字」を表すための、小さな書き方の決まりです。`^\s*#` は「行の頭から、空白がいくつかあって、`#` が来る」という形を表します。`-match`（PowerShell）・`re.match`（Python）・`/…/`（JavaScript）・`Regex::new`（Rust）などで、文字がその形に合うかを調べたり、合う所を取り出したりします。記号ごとの意味は、この一覧の「正規表現」の表にまとめています。	06 前編 2. 開発の準備
`\s`	正規表現の `\s` は、「空白 1 つ」（スペース・タブ・改行など）を表します。`\s*` は「空白が 0 個以上」。	06 前編 2. 開発の準備
`*`	正規表現の `*` は、「すぐ前の物が、0 回以上くり返す」を表します。`\s*` は「空白が 0 個以上」、`.*` は「何でも、何文字でも」。	06 前編 2. 開発の準備

コード	意味と使い方	初めて出てくる所
`^`	正規表現の `^` は、「行（文字）の頭」を表します。`^#` は「`#` で始まる」。	06 前編 2. 開発の準備
`\.`	正規表現の `\.` のように、記号の前に `\` を付けると、その記号を特別な意味にせず、文字そのものとして表します。`\.` は `.`、`\/` は `/` です。	06 前編 3. 設定
`[…]`	正規表現の `[ ]` は、「中に書いた文字の、どれか 1 つ」を表します。`[0-9a-zA-Z_-]` は「数字・英字・`_`・`-` のどれか」、`[.-]` は「`.` か `-`」。`-` で範囲を書けます。	06 前編 3. 設定
`+`	正規表現の `+` は、「すぐ前の物が、1 回以上くり返す」を表します。`\d+` は「数字が 1 つ以上」。	06 前編 3. 設定
`.`	正規表現の `.` は、「どんな 1 文字でも」を表します。`.` という文字そのものを表したい時は、`\.` と書きます。	06 前編 3. 設定
`^`	正規表現の `^` は、「行（文字）の頭」を表します。`^#` は「`#` で始まる」。	06 前編 3. 設定
`$`	正規表現の `$` は、「行（文字）の終わり」を表します。`\d+$` は「数字で終わる」。`^…$` と両方書くと、「全体がこの形か」を調べられます。	06 前編 3. 設定
`(?:…)`	正規表現の `(?: )` は、取り出さないまとまりです。`(?:^|; )` は「行の頭か、`;` のどちらか」をまとめるだけで、取り出す番号には数えません。	06 後編 3. サーバーとの窓口
`[^…]`	正規表現の `[^ ]` は、「中に書いた文字**以外**の 1 文字」を表します。`[^;]+` は「`;` 以外の文字が 1 つ以上」、`[^\x21-\x7E]` は「英数字と記号（ASCII の見える文字）以外」。	06 後編 3. サーバーとの窓口
`+`	正規表現の `+` は、「すぐ前の物が、1 回以上くり返す」を表します。`\d+` は「数字が 1 つ以上」。	06 後編 3. サーバーとの窓口
`^`	正規表現の `^` は、「行（文字）の頭」を表します。`^#` は「`#` で始まる」。	06 後編 3. サーバーとの窓口
`|`	正規表現の `|` は、「左か右のどちらか」を表します。`api/|admin/` は「`api/` か `admin/`」。	06 後編 3. サーバーとの窓口
`(…)`	正規表現の `( )` は、まとまり（グループ）を作ります。合った文字のうち、`( )` の中の部分だけを、あとから取り出せます（`m[1]`・`m.group(1)`・`$1` など）。`(…)?` のように、まとまりごとくり返しや省略もできます。	06 後編 3. サーバーとの窓口

コード	意味と使い方	初めて出てくる所
`(?=…)`	正規表現の `(?= )` は「この後ろに、こういう形が続く時だけ」、`(?! )` は「この後ろに、こういう形が続かない時だけ」を表します（先読み）。`^(?!api/|admin/).*$` は「`api/` や `admin/` で始まらない物、全部」。中の文字は、合わせた文字には入りません。	06 後編 3. サーバーとの窓口
`*`	正規表現の `*` は、「すぐ前の物が、0 回以上くり返す」を表します。`\s*` は「空白が 0 個以上」、`.*` は「何でも、何文字でも」。	06 後編 3. サーバーとの窓口
`|`	正規表現の `|` は、「左か右のどちらか」を表します。`api/|admin/` は「`api/` か `admin/`」。	06 後編 3. サーバーとの窓口
`\d`	正規表現の `\d` は、「数字 1 つ（0〜9）」を表します。	06 後編 4. ログインと、画面の切りかえ
`\.`	正規表現の `\.` のように、記号の前に `\` を付けると、その記号を特別な意味にせず、文字そのものとして表します。`\.` は `.`、`\/` は `/` です。	06 後編 4. ログインと、画面の切りかえ
`?`	正規表現の `?` は、「すぐ前の物が、あっても無くてもよい（0 回か 1 回）」を表します。`(\/edit)?` は「`/edit` があってもなくてもよい」。	06 後編 4. ログインと、画面の切りかえ
`$`	正規表現の `$` は、「行（文字）の終わり」を表します。`\d+$` は「数字で終わる」。`^…$` と両方書くと、「全体がこの形か」を調べられます。	06 後編 4. ログインと、画面の切りかえ

日時の書式

コード	意味と使い方	初めて出てくる所
`%Y`	日時の書式の `%Y` は、4 けたの年（ `2026` ）です。`strftime("%Y-%m-%d")` → `2026-11-05`。	04 6. 日本時間と、承認の URL の文字列
`%m`	日時の書式の `%m` は、2 けたの月（ `01` 〜 `12` ）です。	04 6. 日本時間と、承認の URL の文字列
`%d`	日時の書式の `%d` は、2 けたの日（ `01` 〜 `31` ）です。	04 6. 日本時間と、承認の URL の文字列

コード	意味と使い方	初めて出てくる所
`%H`	日時の書式の `%H` は、24 時間の形の時（ `00` 〜 `23` ）です。	04 6. 日本時間と、承認の URL の文字列
`%M`	日時の書式の `%M` （大文字）は、2 けたの分（ `00` 〜 `59` ）です。月の `%m` （小文字）と取り違えないようにします。	04 6. 日本時間と、承認の URL の文字列
`%S`	日時の書式の `%S` （大文字）は、2 けたの秒（ `00` 〜 `59` ）です。	09 2. SMTP の会話を見る
`%f`	日時の書式の `%f` は、秒より細かい部分（Python では 6 けたのマイクロ秒）です。同じ秒に作ったファイルの名前が、重ならないようにする時に使います。	09 2. SMTP の会話を見る

JavaScript・TypeScript

コード	意味と使い方	初めて出てくる所
`const`	JavaScript・TypeScript で、変えない値に名前を付ける書き方。 `const db = …;` 。あとで別の値を入れようとするとエラーになります。	02 2. 変数・配列・オブジェクト・関数
`let`	JavaScript・TypeScript で、あとで変える値に名前を付ける書き方。	02 2. 変数・配列・オブジェクト・関数
`return`	JavaScript・TypeScript で、関数の結果を返して、関数を終える命令。	02 2. 変数・配列・オブジェクト・関数
`.log()`	`console.log(…)` は、渡した値を画面（ターミナルやブラウザーの開発者ツール）に表示します。	02 2. 変数・配列・オブジェクト・関数
`.join()`	JavaScript・TypeScript で、配列の中身を区切り文字でつないで 1 つの文字列にするメソッド。 `["a", "b"].join(";")` は `"a;b"` 。	02 2. 変数・配列・オブジェクト・関数
`=>`	アロー関数（短く書ける関数）の矢印。 `(path) => `…${path}`` は「 `path` を受け取って文字列を返す関数」。	02 2. 変数・配列・オブジェクト・関数

コード	意味と使い方	初めて出てくる所
`//`	JavaScript・TypeScript では、`//` から行の終わりまでがコメントです。	02 2. 変数・配列・オブジェクト・関数
`${…}`	`` `…` ``（バッククォートで囲んだ文字。テンプレート文字列）の中の `${ }` には、値や式を入れられます。`` `エラー ${res.status}` `` は、`res.status` が 404 なら `エラー 404` になります。改行も、そのまま書けます。	02 2. 変数・配列・オブジェクト・関数
`条件 ? A : B`	`条件 ? A : B` は、「条件が合えば A、合わなければ B」になる式です（三項演算子）。`user.isAdmin ? "管理者" : "一般"`。`if` とちがい、値を返すので、そのまま変数や表示に使えます。	02 2. 変数・配列・オブジェクト・関数
`.length`	`.length` は、並びの数や、文字の数です。文字の場合は、UTF-16 という数え方なので、絵文字などは 2 と数えることがあります（`[...文字].length` なら、見た目の文字の数に近くなります）。	02 2. 変数・配列・オブジェクト・関数
`if`	JavaScript・TypeScript の条件。`if (条件) { … } else { … }`。条件は `( )` で囲みます。	02 3. 依頼の状態を型にする
`default`	`export default` は、そのファイルから「いちばんの物」を 1 つ外に出す書き方。読み込む側は `import 名前 from "./ファイル"` と、好きな名前で受け取れます。	02 3. 依頼の状態を型にする
`/* */`	JavaScript・TypeScript では、`/*` から `*/` までがコメントです。何行にもまたがって書けます。	02 3. 依頼の状態を型にする
`import`	ほかのファイルや部品から、関数などを読み込む書き方。`import { rmSync } from "node:fs";` は、Node.js のファイルの道具から `rmSync` を使えるようにします。	02 4. 配列の道具: 絞る・変える・まとめる
`from`	`import { 名前 } from "場所";` の一部で、「どこから読み込むか」を書きます。	02 4. 配列の道具: 絞る・変える・まとめる
`for`	くり返しの書き方。`for (const x of 配列) { … }` は、配列の中身を 1 つずつ `x` に入れてくり返します。	02 4. 配列の道具: 絞る・変える・まとめる
`.filter()`	配列から、条件に合う物だけを集めた新しい配列を作るメソッド。	02 4. 配列の道具: 絞る・変える・まとめる
`.map()`	配列の中身を 1 つずつ変換した、新しい配列を作るメソッド。	02 4. 配列の道具: 絞る・変える・まとめる

コード	意味と使い方	初めて出てくる所
`!==`	`a !== b` は、「a と b が同じではない」（型も含めて比べる）です。`===`（同じ）の反対です。	02 4. 配列の道具: 絞る・変える・まとめる
`as`	TypeScript で「この値は、この型として扱う」と伝える書き方（`値 as 型`）。	02 5. 図面を JSON で保存する
`new`	クラスから、新しい物を作る書き方。`new DatabaseSync("ba.sqlite")`・`new Date()`・`new Error("…")` のように使います。	02 5. 図面を JSON で保存する
`.parse()`	`JSON.parse(文字列)` は、JSON の文字列を、JavaScript のオブジェクトや配列に変えます。逆は `JSON.stringify(値)` です。	02 5. 図面を JSON で保存する
`&&`	`a && b` は、「a と b の両方が当てはまる」（かつ）です。左が当てはまらなければ、右は調べません。	02 5. 図面を JSON で保存する
`||`	`a || b` は、「a か b のどちらか」（または）です。`el.innerText || el.getAttribute("name") || ""` のように、「左が空なら右を使う」という使い方もします（`0` や `""` も「空」とみなされるので、`null`・`undefined` だけを見たい時は `??` を使います）。	02 5. 図面を JSON で保存する
`Array`	`Array` は、並び（配列）の型です。`Array.isArray(値)` で、値が並びかどうかを調べられます。	02 5. 図面を JSON で保存する
`TextEncoder`	`new TextEncoder().encode(文字)` は、文字を UTF-8 のバイトの並びにします。`.length` で、バイトの数（大きさ）が分かります。	02 5. 図面を JSON で保存する
`else`	JavaScript・TypeScript で、`if` の条件に合わなかった時の所。`else if (条件)` で、条件をつなげられます。	02 7. サーバーとのやり取り
`await`	JavaScript・TypeScript で、時間のかかる処理（`Promise`）が終わるまで待って、結果を受け取る書き方。`async` の関数の中で使います。	02 7. サーバーとのやり取り
`async`	JavaScript・TypeScript で、中で `await` を使える関数にする印。結果は `Promise`（あとで届く約束）で返ります。	02 7. サーバーとのやり取り
`.close()`	Playwright で、ブラウザーを閉じるメソッド（`await browser.close()`）。	02 7. サーバーとのやり取り
`!`	`!res.ok` のように前に付けた `!` は、「〜ではない」です。`if (!res.ok)` は「うまくいかなかったら」。	02 7. サーバーとのやり取り

コード	意味と使い方	初めて出てくる所
`Promise`	`Promise` は、「あとで終わる処理」（サーバーの答えを待つ、時間を待つ）を表す物です。`await` を付けると、終わるまで待って、結果を受け取れます。`new Promise((resolve) => …)` で、自分で作ることもできます。	02 7. サーバーとのやり取り
`Error`	`new Error("説明")` は、エラーを作ります。`throw new Error(…)` でエラーを起こし、`try { … } catch (err) { … }` で受け止めます。`err.message` で説明の文を読めます。	02 7. サーバーとのやり取り
`fetch()`	`fetch(URL)` は、サーバーに HTTP の要求を送る関数です。`await fetch(…)` で答えを待ち、`res.ok`（うまくいったか）・`res.status`（状態コード）・`await res.json()`（JSON の中身）を読みます。	02 7. サーバーとのやり取り
`.ok`	`fetch` の答えの `.ok` は、状態コードが 200〜299（うまくいった）なら `true` です。	02 7. サーバーとのやり取り
`({ 名前 })`	関数の引数の `({ status, label })` は、渡されたオブジェクトから、`status` と `label` を取り出して受け取る書き方です。React の部品（コンポーネント）が props を受け取る時によく使います。	03 2. 部品
`const [ … ] =`	`const [a, b] = 並び;` は、並びの 0 番目を `a` に、1 番目を `b` に取り出す書き方です（分割代入）。React の `const [count, setCount] = useState(0);` もこれで、「今の値」と「値を変える関数」の 2 つを受け取っています。	03 3. props と state
`/…/`	JavaScript の `/…/` は、正規表現（文字の形を表す書き方）をそのまま書く形です。`/\d+/` は「数字が 1 つ以上」。`.match(/…/)`・`.test(…)`・`.split(/…/)` などで使います。記号ごとの意味は、この一覧の「正規表現」の表にあります。	03 4. 一覧の画面: 読み込みと、しぼり込み
`() =>`	`() => { … }` は、名前の無い関数（アロー関数）の書き方です。`()` の中が引数、`=>` の右が中身です。`(e) => setKeyword(e.target.value)` のように、1 つの式なら `{ }` を省けます。ほかの関数に「あとで呼んでもらう処理」を渡す時によく使います。	03 4. 一覧の画面: 読み込みと、しぼり込み
`.target`	イベント（`e`）の `.target` は、イベントが起きたタグ（入力欄など）です。`e.target.value` で、入力欄に今入っている文字を読めます。	03 4. 一覧の画面: 読み込みと、しぼり込み
`.trim()`	JavaScript・TypeScript で、文字列の前後の空白を取り除いたものを返すメソッド。	03 5. 入力フォーム
`.push()`	JavaScript・TypeScript で、配列の後ろに足すメソッド（元の配列が変わる）。	03 5. 入力フォーム

コード	意味と使い方	初めて出てくる所
`Set`	`new Set(並び)` は、同じ値を 1 つにまとめた集まりを作ります。`new Set(chosen).size !== chosen.length` は、「重なりを消すと数が減る」＝「同じ人が 2 回選ばれている」の確かめです。	03 5. 入力フォーム
`.size`	`Set` や `Map` の `.size` は、中の数です。	03 5. 入力フォーム
`.round()`	`Math.round(x)` は、小数を四捨五入して整数にする関数です。	03 6. 図面の画面
`Math`	`Math` は、計算の関数が集まった物です。`Math.round(3.6)` は四捨五入で `4`、`Math.max(1, 5)` は大きい方の `5` です。	03 6. 図面の画面
`.currentTarget`	イベントの `.currentTarget` は、イベントを受け取る処理（`onPointerDown` など）を付けたタグそのものです（`.target` は、中の、実際に押された物のことがあります）。	03 6. 図面の画面
`.clientX`	マウスやペンのイベントの `.clientX`・`.clientY` は、画面の左上からの位置（ピクセル）です。	03 6. 図面の画面
`.clientY`	`.clientY` は、画面の上からの位置（ピクセル）です。	03 6. 図面の画面
`defineConfig()`	`defineConfig({ … })` は、設定ファイル（Vite の `vite.config` や Playwright の `playwright.config`）で、設定を書くための関数です。書き間違いを、エディターが教えてくれるようになります。	06 後編 1. 開発中と、ビルドした後
`Object`	`Object.values(obj)` は、オブジェクトの値だけを並びにします。`Object.keys(obj)` はキーだけ、`Object.entries(obj)` は `[キー, 値]` の組の並びです。	06 後編 3. サーバーとの窓口
`String`	`String(値)` は、値を文字にします。`values.map(String)` は、並びの全部を文字にします。	06 後編 3. サーバーとの窓口
`FormData`	`FormData` は、フォームの中身（ファイルも含む）を、送れる形にまとめる入れ物です。`fetch` の `body` に入れて、ファイルを送ります。	06 後編 3. サーバーとの窓口
`decodeURIComponent()`	`decodeURIComponent("%E5%B1%B1")` は、URL やクッキーの中で `%XX` の形にした文字を、元の文字（山）に戻します。反対は `encodeURIComponent()` です。	06 後編 3. サーバーとの窓口
`.cookie`	`document.cookie` は、そのページのクッキーを、`名前=値; 名前=値` の形の文字で読みます（`HttpOnly` のクッキーは読めません）。	06 後編 3. サーバーとの窓口
`async () =>`	`async () => { … }` は、中で `await`（終わるまで待つ）を使える、名前の無い関数（アロー関数）です。	06 後編 4. ログインと、画面の切りかえ

コード	意味と使い方	初めて出てくる所
`Number`	`Number("42")` は、文字を数にします。数にできない時は `NaN`（数ではない）になります。	06 後編 4. ログインと、画面の切りかえ
`/…/g`	正規表現の後ろの `g` は、「合う所を全部」という指定（フラグ）です。`.replace(/a/g, "b")` は、全部の `a` を `b` にします（`g` が無いと、最初の 1 つだけ）。`i` は大文字・小文字を区別しない指定です。	06 後編 5. 依頼を出す
`setTimeout()`	`setTimeout(() => { … }, 3000)` は、3000 ミリ秒（3 秒）たってから、中の処理を 1 回動かします。返ってくる番号を `clearTimeout(番号)` に渡すと、取りやめにできます。	06 後編 5. 依頼を出す
`clearTimeout()`	`clearTimeout(timer)` は、`setTimeout` で予約した処理を取りやめにします。React では、部品が消える時の後片づけ（`useEffect` の `return () => …`）で呼びます。	06 後編 5. 依頼を出す
`Date`	`new Date(文字)` は、日時の値を作ります。`.toLocaleString("ja-JP")` で、日本の形の日時の文字になります。	06 後編 6. 依頼の詳細と、操作のボタン
`.createElementNS()`	名前空間（ここでは SVG）を決めて、新しい要素を作るメソッド。SVG の図形（`rect`・`line`）は、ふつうの `createElement` ではなく、これで作らないと表示されません。	08 後編 6. 図面
`.entries()`	`Object.entries(オブジェクト)` は、オブジェクトの「名前と値」の組を、`[名前, 値]` の配列の並びにします。`for (const [k, v] of Object.entries(attrs))` で、両方を同時に使えます。	08 後編 6. 図面
`.setAttribute()`	要素に、属性（`x="10"` など）を付けるメソッド。`node.setAttribute("x", "10")`。	08 後編 6. 図面
`.replaceChildren()`	要素の中身（子の要素）を、全部入れかえるメソッド。何も渡さないと、中身を空にします。	08 後編 6. 図面
`.setPointerCapture()`	押したポインター（マウスや指）の動きを、要素の外に出ても、この要素で受け取り続けるメソッド。図を描いている途中で、枠の外に出ても線が切れません。	08 後編 6. 図面
`.pop()`	JavaScript・TypeScript で、配列の最後の 1 つを取り出すメソッド（元の配列から消える）。	08 後編 6. 図面
`?.`	`a?.b` は、`a` が `null` や `undefined` なら、そこで止めて `undefined` にする書き方（オプショナルチェーン）。無いかもしれない物を、エラーにせずにたどれます。	08 後編 6. 図面

コード	意味と使い方	初めて出てくる所
`.value`	`.value` は、入力欄に今入っている文字や、クッキーなどの値です。	08 後編 6. 図面
`.textContent`	`.textContent` は、タグの中の文字です。入れた文字は、HTML としてではなく、ただの文字として表示されるので、安全です（`innerHTML` に入れると、タグとして動いてしまいます）。	08 後編 6. 図面
`.split()`	JavaScript・TypeScript で、文字列を区切りで分けて配列にするメソッド。`"a,b".split(",")` は `["a", "b"]`。正規表現（`/,/`）でも区切れます。	11 B. サンプル集
`.find()`	配列から、条件に合う最初の 1 つを返すメソッド。無ければ `undefined` です。その位置（番号）が欲しい時は `.findIndex()` を使います。	11 B. サンプル集
`const { … } =`	`const { name, email } = user;` は、オブジェクトの中の `name` と `email` を、同じ名前の変数に取り出す書き方です（分割代入）。	11 B. サンプル集
`.now()`	`performance.now()` は、ページ（プログラム）が始まってからの時間をミリ秒で返します。`Date.now()` は、1970 年 1 月 1 日からのミリ秒です。どちらも、かかった時間を計るのに使えます。	11 B. サンプル集
`SyntaxError`	`SyntaxError` は、書き方が正しくない時のエラーの種類です。`JSON.parse("壊れた文字")` は、`SyntaxError` を起こします。	11 B. サンプル集
`+=`	`sum += n` は `sum = sum + n` の短い形です。	11 B. サンプル集
`Map`	`new Map()` は、キーと値の組を覚えておく入れ物です。`.set(キー, 値)`・`.get(キー)`・`.has(キー)` で使います。	11 B. サンプル集
`Intl`	`Intl` は、国や言葉に合わせた表示（数の区切り・通貨・日付）の道具が集まった物です。`new Intl.NumberFormat("ja-JP").format(1234567)` → `1,234,567`。	11 B. サンプル集

TypeScript

コード	意味と使い方	初めて出てくる所
`function`	関数（ひとまとまりの処理）を作る書き方。`function greet(name: string): string { … }` は、「文字列の `name` を受け取って、文字列を返す関数 `greet`」です。	02 2. 変数・配列・オブジェクト・関数
`型[]`	`string[]` のように型の後ろに `[]` を付けると、「その型の並び（配列）」という型になります。`Array<string>` と書いても同じです。	02 2. 変数・配列・オブジェクト・関数

コード	意味と使い方	初めて出てくる所
`boolean`	`boolean` は、`true`（はい）か `false`（いいえ）の型です。	02 2. 変数・配列・オブジェクト・関数
`export`	「このファイルの外から使ってよい」という印。`export type RequestStatus = …` と書くと、ほかのファイルで `import` して使えます。	02 3. 依頼の状態を型にする
`type`	型に名前を付ける書き方。`type RequestStatus = "回覧中" | "回覧完了";` のように、決まった値だけを許す型や、オブジェクトの形を決められます。	02 3. 依頼の状態を型にする
`null`	「値が無い」ことを、わざと表す値。「できない操作なら `null` を返す」のように使います。	02 3. 依頼の状態を型にする
`switch`	値によって処理を分ける書き方。`switch (current) { case "回覧中": … }`。合う `case` から下を動かすので、それぞれの終わりに `return` や `break` を書きます。	02 3. 依頼の状態を型にする
`case`	`switch` の中の「この値の時は」の部分。	02 3. 依頼の状態を型にする
`never`	「ここには絶対に来ない」ことを表す型。`switch` の最後に `const unreachable: never = current;` と書いておくと、状態を足したのに `case` を書き忘れた時、`npx tsc` がエラーで教えてくれます。	02 3. 依頼の状態を型にする
`===`	「型も値も同じか」を比べる書き方。`==` は型を勝手にそろえてから比べて思わぬ結果になるので、ふつうは `===` を使います（違うかは `!==`）。	02 3. 依頼の状態を型にする
`A | B`	`type RequestStatus = "下書き" | "回覧中" | "承認済み";` の `|` は、「このどれか」を表す型です（ユニオン型）。決めた値以外を入れると、TypeScript がエラーで教えてくれます。`string | null` は「文字か、null」です。	02 3. 依頼の状態を型にする
`import type`	`import type { RequestStatus } from "./types.ts";` は、「型だけ」を読み込む書き方です。型は動く時には消えるので、動くプログラムには何も残りません。`import { useState, type FormEvent }` のように、1 つずつ `type` を付けることもできます。	02 3. 依頼の状態を型にする
`名前?:`	`note?: string;` のように名前の後ろに `?` を付けると、「この項目は、無くてもよい」という意味になります。無い時の値は `undefined` です。	02 3. 依頼の状態を型にする

コード	意味と使い方	初めて出てくる所
`of`	`for (const 名前 of 配列)` の一部で、「配列の中身を 1 つずつ」という意味です（ `in` を使うと、中身ではなく番号が出てくるので注意）。	02 4. 配列の道具: 絞る・変える・まとめる
`undefined`	「まだ値が入っていない」ことを表す値。 `?` の付いた無くてもよい項目が無い時や、配列の範囲の外を読んだ時に出てきます。	02 4. 配列の道具: 絞る・変える・まとめる
`.sort()`	配列を並べかえるメソッド。 `(a, b) => …` の比べる関数を渡し、マイナスなら `a` が先、プラスなら `b` が先になります。**元の配列を書きかえる**ので、 `[...requests].sort(…)` のように写してから並べかえることがよくあります（新しい配列を作る `toSorted` もあります）。	02 4. 配列の道具: 絞る・変える・まとめる
`.localeCompare()`	2 つの文字列を、辞書の順で比べるメソッド。 `a.localeCompare(b)` は、 `a` が先ならマイナス、同じなら 0、後ならプラス。 `sort` の中で、文字（日付の文字など）で並べる時に使います。	02 4. 配列の道具: 絞る・変える・まとめる
`??`	`a ?? b` は、 `a` が `null` か `undefined` なら `b` を使う書き方。 `(acc[key] ?? 0) + 1` は「まだ無ければ 0 から数える」。	02 4. 配列の道具: 絞る・変える・まとめる
`...`	スプレッド構文。 `[...requests]` は配列の中身を広げて、新しい配列（写し）を作ります。 `{ ...user, isAdmin: true }` は、オブジェクトを写して一部だけ変えます。	02 4. 配列の道具: 絞る・変える・まとめる
`unknown`	「何が入っているか分からない」ことを表す型。そのままでは使えず、 `typeof` などで確かめてから使います。外から来た JSON を受け取る時に使うと安全です。	02 5. 図面を JSON で保存する
`typeof`	値の種類を文字で返す書き方。 `typeof value === "number"` は「数か」を調べます。調べたあとの `if` の中では、TypeScript がその型として扱ってくれます。	02 5. 図面を JSON で保存する
`.isArray()`	`Array.isArray(値)` は、値が配列かどうかを調べる関数です。	02 5. 図面を JSON で保存する
`.stringify()`	`JSON.stringify(値)` は、オブジェクトや配列を、JSON の文字列にする関数です。保存したり、サーバーへ送ったりする時に使います。	02 5. 図面を JSON で保存する
`.encode()`	`new TextEncoder().encode(文字列)` は、文字列を UTF-8 のバイトの並びにするメソッド。 `.length` で、何バイトになるかが分かります（日本語は 1 文字 3 バイト）。	02 5. 図面を JSON で保存する
`Record`	`Record<RequestStatus, string>` は、「キーが `RequestStatus` の全部で、値が文字」のオブジェクトの型です。キーの書き忘れを、TypeScript が見つけてくれます。	02 5. 図面を JSON で保存する

コード	意味と使い方	初めて出てくる所
`void`	「何も返さない」ことを表す型。`Promise<void>` は「終わったことだけを知らせる約束」です。	02 7. サーバーとのやり取り
`throw`	エラーを起こして、処理を止める書き方。`throw new Error("サーバーのエラー")`。`try … catch` で受け止められます。	02 7. サーバーとのやり取り
`try`	失敗するかもしれない処理を `try { … }` で囲み、失敗した時の処理を `catch` に書く書き方。	02 7. サーバーとのやり取り
`catch`	`try` の中で失敗した時に動く所。`catch (e)` の `e` にエラーが入ります（何が来るか分からないので、使う前に確かめます）。	02 7. サーバーとのやり取り
`finally`	`try … catch` のあと、成功しても失敗しても最後に必ず動く所。片づけ（サーバーを閉じる など）を書きます。	02 7. サーバーとのやり取り
`.then()`	`Promise`（あとで届く約束）の結果が届いた時に動く関数を登録するメソッド。`await` を使えば、多くの場合は書かずに済みます。	02 7. サーバーとのやり取り
`.error()`	`console.error(…)` は、エラーの知らせを出す関数です（`console.log` のエラー版）。	02 7. サーバーとのやり取り
`名前<型>()`	`useState<RequestItem[] | null>(null)` や `res.json<T>()` のように、関数の名前の後ろの `< >` は、「どの型で扱うか」を指定する書き方です（ジェネリクス）。	02 7. サーバーとのやり取り
`型<型>`	`Promise<void>`・`Record<string, unknown>` のように、型の後ろの `< >` には、中に入る型を書きます。`Promise<void>` は「終わると何も返さない、あとで終わる処理」です。	02 7. サーバーとのやり取り
`AddressInfo`	`AddressInfo` は、Node.js のサーバーが待ち受けている場所（アドレスとポート番号）の型です。	02 7. サーバーとのやり取り
`.includes()`	文字列（配列）に、その文字（値）が含まれていれば `true` を返すメソッド。`r.title.includes(keyword)` は「件名に検索の言葉が入っているか」。	03 4. 一覧の画面: 読み込みと、しぼり込み
`<HTMLFormElement>`	`FormEvent<HTMLFormElement>` は、「`<form>` で起きた送信の出来事」の型。`< >` の中は、どのタグで起きたかを表します。	03 5. 入力フォーム
`.getBoundingClientRect()`	タグが、画面のどこに、どの大きさで表示されているかを返すメソッド。マウスの位置を、図の中の位置に直す時に使います。	03 6. 図面の画面
`.min()`	`Math.min(a, b)` は、小さい方を返す関数です（大きい方は `Math.max`）。	03 6. 図面の画面
`.abs()`	`Math.abs(x)` は、絶対値（マイナスを取った値）を返す関数です。	03 6. 図面の画面

コード	意味と使い方	初めて出てくる所
`<SVGSVGElement>`	`<svg>` のタグを表す型。`PointerEvent<SVGSVGElement>` は「`<svg>` の上で起きたマウスやペンの出来事」です。	03 6. 図面の画面
`.addEventListener()`	出来事（クリック・URL の `#` の変化など）が起きた時に動く関数を登録するメソッド。`window.addEventListener("hashchange", onChange)`。	03 7. 画面の切りかえと、部品の組み立て
`.removeEventListener()`	`addEventListener` で登録した関数を、外すメソッド。`useEffect` の片づけで呼んで、同じ関数が何度も登録されるのを防ぎます。	03 7. 画面の切りかえと、部品の組み立て
`.getElementById()`	`document.getElementById("root")` は、`id="root"` のタグを探して返す関数。React は、ここに画面を描きます。最後の `!` は「必ずある」と TypeScript に伝える印です。	03 7. 画面の切りかえと、部品の組み立て
`値!`	`document.getElementById("root")!` の最後の `!` は、「これは null ではないと、自分が保証する」と TypeScript に伝える印です（非 null 表明）。本当に null だと、動く時にエラーになるので、確かな時だけ使います。	03 7. 画面の切りかえと、部品の組み立て
`class`	クラス（データと、それを扱うメソッドをまとめた設計図）を作る書き方。`class ApiError extends Error { … }`。`new ApiError(…)` で作ります。	06 後編 3. サーバーとの窓口
`extends`	クラスを、ほかのクラスを元にして作る書き方。`class ApiError extends Error` は、標準の `Error` に、`status` などを足したエラーです。	06 後編 3. サーバーとの窓口
`this`	クラスの中で「この物自身」を表す言葉。`this.status = status;` は、自分の `status` に値を入れます。	06 後編 3. サーバーとの窓口
`instanceof`	値が、そのクラスから作られた物かを調べる書き方。`body instanceof FormData` は「ファイル付きのフォームの中身か」。	06 後編 3. サーバーとの窓口
`.match()`	文字列が正規表現に合う所を探すメソッド。見つかると、その情報（かっこの中の部分など）を配列で、見つからないと `null` を返します。	06 後編 3. サーバーとの窓口
`.flat()`	配列の中の配列を、1 つの配列に平らにするメソッド。	06 後編 3. サーバーとの窓口
`.catch()`	`Promise` が失敗した時の代わりの処理を登録するメソッド。`res.json().catch(() => ({}))` は「JSON として読めなければ空のオブジェクト」。	06 後編 3. サーバーとの窓口
`.append()`	`FormData` に項目を足すメソッド。`form.append("file", file)` で、ファイル付きのフォームを組み立てます。	06 後編 3. サーバーとの窓口

コード	意味と使い方	初めて出てくる所
`RequestInit`	`RequestInit` は、`fetch(URL, 設定)` の 2 つ目に渡す「設定」（`method`・`headers`・`body` など）の型です。	06 後編 3. サーバーとの窓口
`.test()`	正規表現が、文字列に合うかを `true`・`false` で返すメソッド。`/.(jpe?g|png|pdf)$/i.test(名前)` は「拡張子が JPG・PNG・PDF か」。	06 後編 5. 依頼を出す
`.has()`	`Set` や `Map` に、その値（キー）があれば `true` を返すメソッド。	06 後編 5. 依頼を出す
`.slice()`	配列や文字列の一部分を切り出した、新しい物を返すメソッド。`list.slice(0, 5)` は最初の 5 つ。	06 後編 5. 依頼を出す
`.toLocaleString()`	日時や数を、その国の書き方の文字列にするメソッド。`new Date(s).toLocaleString("ja-JP", { dateStyle: "short", timeStyle: "short" })` は `2026/11/02 8:00` の形です。	06 後編 6. 依頼の詳細と、操作のボタン
`.ceil()`	`Math.ceil(x)` は、小数を切り上げて整数にする関数です（大きさを KB で見せる時など）。	06 後編 6. 依頼の詳細と、操作のボタン
`.padStart()`	文字列の前を、決めた文字で埋めて、決めた長さにするメソッド。`"7".padStart(3, "0")` は `"007"`。	11 B. サンプル集
`.replaceAll()`	文字列の中の、合う所を全部置きかえた新しい文字列を返すメソッド（`replace` は最初の 1 つだけ）。	11 B. サンプル集
`.toUpperCase()`	英字を大文字にした文字列を返すメソッド。	11 B. サンプル集
`.endsWith()`	文字列が、その文字で終わっていれば `true` を返すメソッド。	11 B. サンプル集
`.isNaN()`	`Number.isNaN(値)` は、値が `NaN`（数ではない）かを調べる関数です。`Number("42円")` のように数にできなかった時に `NaN` になります。	11 B. サンプル集
`.parseInt()`	`Number.parseInt("42円", 10)` は、文字列の先頭の数の部分だけを整数として読む関数です（10 は 10 進数）。	11 B. サンプル集
`.toFixed()`	数を、小数点以下の桁を決めた文字列にするメソッド。`(1234.5678).toFixed(2)` は `"1234.57"`。	11 B. サンプル集
`.floor()`	`Math.floor(x)` は、小数を切り捨てて整数にする関数です（切り上げは `Math.ceil`、四捨五入は `Math.round`）。	11 B. サンプル集
`.findIndex()`	配列の中で、条件に合う最初の物の位置（番号）を返すメソッド。無ければ `-1` です。	11 B. サンプル集

コード	意味と使い方	初めて出てくる所
`.some()`	配列の中に、条件に合う物が 1 つでもあれば `true` を返すメソッド。	11 B. サンプル集
`.every()`	配列の中身が、全部条件に合えば `true` を返すメソッド。	11 B. サンプル集
`.reduce()`	配列を 1 つずつ見ながら、1 つの値（合計など）にまとめるメソッド。`a.reduce((合計, x) => 合計 + x.amount, 0)` の最後の `0` が、最初の値です。	11 B. サンプル集
`.toSorted()`	並べかえた**新しい**配列を返すメソッド（元の配列は変わらない）。React の state を並べかえる時に便利です。	11 B. サンプル集
`.forEach()`	配列の中身を 1 つずつ処理するメソッド（新しい配列は作らない）。`(x, i) => …` で、番号 `i` も受け取れます。	11 B. サンプル集
`interface`	オブジェクトの形に名前を付ける書き方（`type` とほぼ同じ）。`interface Approval { order: number; approver: string; }`。	11 B. サンプル集
`.keys()`	`Object.keys(オブジェクト)` は、オブジェクトの項目の名前を、配列で返す関数です（`Object.entries` は名前と値の組）。	11 B. サンプル集
`as const`	`as const` は、「この値は、中身も含めて変えない、決まった値」にする印です。`["下書き", "回覧中"] as const` の型は、ただの `string[]` ではなく、2 つの決まった文字の並びになります。	11 B. サンプル集
`.all()`	`Promise.all([…])` は、いくつかの `Promise`（時間のかかる処理）を同時に待って、全部の結果を配列で返す関数です。1 つでも失敗すると、全体が失敗します。	11 B. サンプル集
`.set()`	`Map` に「キー → 値」を入れる（上書きする）メソッド。	11 B. サンプル集
`.get()`	`Map` から、キーで値を取り出すメソッド。無ければ `undefined` です。	11 B. サンプル集
`.UTC()`	`Date.UTC(年, 月, 日, …)` は、世界標準時の日時を、1970 年からのミリ秒の数で返す関数です（月は 0 から数える）。`new Date(Date.UTC(…))` で日時を作ります。	11 B. サンプル集
`.toISOString()`	日時を、`2026-11-10T00:00:00.000Z` の形（世界標準時）の文字列にするメソッド。サーバーとのやり取りに使います。	11 B. サンプル集
`.toLocaleDateString()`	日付を、その国の書き方の文字列にするメソッド。`{ timeZone: "Asia/Tokyo" }` を付けて、日本時間で表示します。	11 B. サンプル集
`.getTime()`	日時を、1970 年からのミリ秒の数にするメソッド。2 つの日時の差を計算する時に使います。	11 B. サンプル集

コード	意味と使い方	初めて出てくる所
`.NumberFormat()`	`new Intl.NumberFormat("ja-JP", { style: "currency", currency: "JPY" })` は、数を「￥30,000」のような形にする係を作ります。	11 B. サンプル集
`.format()`	`Intl.NumberFormat` などの係で、数や日時を決めた形の文字列にするメソッド。	11 B. サンプル集

React

コード	意味と使い方	初めて出てくる所
`<span>`	文の一部分を囲む、特別な意味の無いタグ（JSX の中でも HTML と同じ）。状態の札など、一部だけ見た目を変える時に使います。	03 2. 部品
`className=`	JSX では、HTML の `class` の代わりに `className` と書きます（`class` は JavaScript の言葉として使われているからです）。`className="btn-primary"`。	03 2. 部品
`useState()`	React の部品に「覚えておく値」（state）を作る関数（フック）。`const [keyword, setKeyword] = useState("");` で、今の値 `keyword` と、値を変える関数 `setKeyword` ができます。`setKeyword(新しい値)` を呼ぶと、画面が描き直されます。	03 3. props と state
`useEffect()`	「画面を描いたあとに、することがある」時に使う関数（フック）。`useEffect(() => { … }, [見張る値])` で、データを取ってくる・イベントを登録する、などをします。`[ ]` の中の値が変わった時だけ、もう一度動きます。	03 4. 一覧の画面: 読み込みと、しぼり込み
`<p>`	段落のタグ（JSX の中でも HTML と同じ）。`return <p>読み込み中…</p>;` のように、部品が返す画面に使います。	03 4. 一覧の画面: 読み込みと、しぼり込み
`<section>`	ひとまとまりの部分（章）を表すタグ。	03 4. 一覧の画面: 読み込みと、しぼり込み
`<h1>`	いちばん大きな見出しのタグ（JSX の中でも HTML と同じ）。	03 4. 一覧の画面: 読み込みと、しぼり込み
`<table>`	表のタグ（JSX の中でも HTML と同じ）。	03 4. 一覧の画面: 読み込みと、しぼり込み

コード	意味と使い方	初めて出てくる所
`<thead>`	表の見出しの行をまとめるタグ。	03 4. 一覧の画面: 読み込みと、しぼり込み
`<tr>`	表の 1 行のタグ。	03 4. 一覧の画面: 読み込みと、しぼり込み
`<th>`	表の見出しのますのタグ。	03 4. 一覧の画面: 読み込みと、しぼり込み
`<tbody>`	表の中身の行をまとめるタグ。	03 4. 一覧の画面: 読み込みと、しぼり込み
`<td>`	表の中身のますのタグ。`<td>{r.id}</td>` のように、`{ }` の中に値を書けます。	03 4. 一覧の画面: 読み込みと、しぼり込み
`placeholder=`	`placeholder="件名でしぼる"` は、入力欄が空の時に、うすく見せる例の文字です。	03 4. 一覧の画面: 読み込みと、しぼり込み
`value=`	入力欄の `value={keyword}` は、「入力欄に表示する値は、この変数」という結び付けです。`onChange` で変数を変えると、表示も変わります（制御されたコンポーネント）。	03 4. 一覧の画面: 読み込みと、しぼり込み
`onChange=`	`onChange={(e) => setKeyword(e.target.value)}` は、入力欄の中身が変わるたびに動く処理です。	03 4. 一覧の画面: 読み込みと、しぼり込み
`key=`	`key={r.id}` は、並びを `map` で表示する時に、1 つずつに付ける「見分けの印」です。React は、これで、どれが増えた・消えた・動いたかを見分けます。並びの中で重ならない値（番号など）にします。	03 4. 一覧の画面: 読み込みと、しぼり込み
`.preventDefault()`	ブラウザーがふつうにする動き（フォームを送るとページを移動する など）を止めるメソッド。React でフォームを扱う時は、`onSubmit` の最初で `e.preventDefault()` を呼びます。	03 5. 入力フォーム
`<ul>`	順番の無い並び（点の付いた並び）のタグ。中に `<li>` を並べます。	03 5. 入力フォーム
`<li>`	並びの 1 つ（list item）のタグ。配列を `map` で `<li>` にする時は、`key` を付けます。	03 5. 入力フォーム

コード	意味と使い方	初めて出てくる所
`<div>`	特別な意味の無い箱のタグ（JSX の中でも HTML と同じ）。JSX では `class` ではなく `className` で CSS の class を付けます。	03 5. 入力フォーム
`<textarea>`	何行も書ける入力欄のタグ。React では `value` と `onChange` で中身を state と結び付けます。	03 5. 入力フォーム
`<select>`	選ぶ欄のタグ。React では `value` と `onChange` で、選んだ値を state と結び付けます。	03 5. 入力フォーム
`<option>`	`<select>` の中の選べる 1 つ（JSX でも HTML と同じ）。React では、選ばれている物を `<select value={…}>` の方で決めます。	03 5. 入力フォーム
`<button>`	ボタンのタグ。`onClick={() => …}` で、押した時にすることを決めます。	03 5. 入力フォーム
`{条件 && …}`	JSX の `{errors.length > 0 && <ul>…</ul>}` は、「条件が合う時だけ、右を表示する」書き方です。合わない時は、何も表示しません。	03 5. 入力フォーム
`onSubmit=`	`<form onSubmit={handleSubmit}>` は、フォームを送った時（送信ボタン・Enter）に動く処理です。中で `e.preventDefault()` を呼ぶと、ページの読み直しを止められます。	03 5. 入力フォーム
`role=`	`role="alert"` は、その部分の役わりを、読み上げソフトに伝える印です。`alert` は、すぐ知らせる大事なお知らせです。	03 5. 入力フォーム
`htmlFor=`	JSX では、`<label>` の `for` の代わりに `htmlFor` と書きます。	03 5. 入力フォーム
`id=`	`id` は、ページの中で 1 つだけの名前です。`<label htmlFor="title">` と組で使います。	03 5. 入力フォーム
`rows=`	`<textarea rows={3}>` の `rows` は、見える行の数です。	03 5. 入力フォーム
`type=`	`<input type="date">`・`<button type="submit">` の `type` は、入力欄やボタンの種類です。	03 5. 入力フォーム
`aria-*=`	`aria-label="…"` のように `aria-` で始まる属性は、読み上げソフトなどの手助けの道具に、見た目では分からないことを伝える印です。	03 5. 入力フォーム
`disabled=`	`disabled={sending}` は、`sending` が true の間、ボタンを押せなくします（2 回送るのを防ぎます）。	03 5. 入力フォーム
`<svg>`	図（線や四角）を描くタグ。中に `<rect>`（四角）・`<line>`（線）などを並べます。拡大しても線がぼやけません。	03 6. 図面の画面
`<h2>`	2 番目の大きさの見出しのタグ。	03 6. 図面の画面
`<pre>`	中の文字を、空白や改行もそのまま表示するタグ。JSON を見やすく表示する時に使います。	03 6. 図面の画面

コード	意味と使い方	初めて出てくる所
`viewBox=`	`<svg viewBox="0 0 400 300">` の `viewBox` は、SVG の絵の中の座標の範囲です。	03 6. 図面の画面
`onPointerDown=`	`onPointerDown` は、マウス・ペン・指で押した瞬間に動く処理です。`onPointerUp` は離した時です。	03 6. 図面の画面
`onPointerUp=`	`onPointerUp` は、マウス・ペン・指を離した時に動く処理です。	03 6. 図面の画面
`x=`	SVG の `x`・`y` は、図形を置く位置です。	03 6. 図面の画面
`y=`	SVG の `y` は、図形を置く縦の位置です。	03 6. 図面の画面
`width=`	`width` は、幅です。	03 6. 図面の画面
`height=`	`height` は、高さです。	03 6. 図面の画面
`onClick=`	`onClick={() => …}` は、押された時に動く処理です。	03 6. 図面の画面
`<header>`	ページの上の帯を表すタグ（JSX の中でも HTML と同じ）。	03 7. 画面の切りかえと、部品の組み立て
`<strong>`	「大事」という意味のタグ（太字）。	03 7. 画面の切りかえと、部品の組み立て
`<nav>`	メニューを表すタグ。	03 7. 画面の切りかえと、部品の組み立て
`<>…</>`	JSX の `<> … </>` は、名前の無い入れ物（フラグメント）です。部品は 1 つの物を返す決まりなので、いくつも並べたい時に、余分なタグを増やさずに包みます。	03 7. 画面の切りかえと、部品の組み立て
`href=`	`<a href="#/">` の `href` は、リンク先です。`#/` で始まる場所は、ページを読み直さずに、画面の中だけで切りかえるのに使っています。	03 7. 画面の切りかえと、部品の組み立て
`.render()`	`createRoot(要素).render(<App />)` は、React の部品（ここでは `<App />`）を、その要素の中に表示します。React の画面は、ここから始まります。	03 7. 画面の切りかえと、部品の組み立て
`<StrictMode>`	開発の時だけ、まちがいを見つけやすくするために、部品をわざと 2 回描いてみるなどの確かめをする、React の部品。本番の動きには影響しません。	03 7. 画面の切りかえと、部品の組み立て

コード	意味と使い方	初めて出てくる所
`createRoot()`	`createRoot(document.getElementById("root")!)` は、HTML の `<div id="root">` の中に、React の画面を描く場所を作る関数です。`.render(<App />)` で、いちばん外側の部品を描きます。	03 7. 画面の切りかえと、部品の組み立て
`react()`	`react()` は、Vite に、React（JSX）のファイルを扱えるようにする部品（プラグイン）です。	06 後編 1. 開発中と、ビルドした後
`useCallback()`	関数を覚えておき、描き直しのたびに作り直さないようにするフック。`useEffect` の見張る値に関数を入れる時などに使います。	06 後編 4. ログインと、画面の切りかえ
`maxLength=`	JSX の `maxLength={100}` は、入力できる文字の数の上限です（HTML の `maxlength`）。	06 後編 5. 依頼を出す
`accept=`	`accept=".jpg,.jpeg,.png,.pdf"` は、選べるファイルの種類です（サーバー側でも必ず確かめます）。	06 後編 5. 依頼を出す
`<ol>`	番号の付いた並びのタグ。中に `<li>` を並べます（回覧の順番など）。	06 後編 5. 依頼を出す
`<figure>`	図と、その説明をひとまとめにするタグ。	06 後編 7. グラフ
`<figcaption>`	`<figure>` の中の、図の説明のタグ。	06 後編 7. グラフ
`<g>`	SVG の中で、図の部品をまとめるタグ（group）。まとめて動かしたり、色を付けたりできます。	06 後編 7. グラフ
`transform=`	SVG の `transform="translate(0 30)"` は、図形を動かす（ずらす）指定です。	06 後編 7. グラフ
`textAnchor=`	SVG の `textAnchor="end"` は、文字を、置いた位置に右端をそろえて書く指定です（`middle` で真ん中）。	06 後編 7. グラフ
`rx=`	SVG の `<rect rx="4">` の `rx` は、四角の角を丸くする大きさです。	06 後編 7. グラフ
`<ol>`	番号の付いた並びのタグ。中に `<li>` を並べます（回覧の順番など）。	08 後編 7. 依頼の詳しい画面と、ボタンの操作
`<section>`	ひとまとまりの部分（章）を表すタグ。	11 B. サンプル集
`{/* */}`	JSX の中のコメントは、`{/* … */}` と書きます。	11 B. サンプル集
`checked=`	`checked={onlyOpen}` は、チェックボックスの ON・OFF を、変数と結び付けます。	11 B. サンプル集
`<h3>`	3 番目の大きさの見出しのタグ。	11 B. サンプル集

練習アプリで作る物（TypeScript）

コード	意味と使い方	初めて出てくる所
`.csrf()`	`api.ts` で作る関数の 1 つで、`/api/csrf/` を呼んで、CSRF の札（クッキー）を先に受け取ります。	06 後編 4. ログインと、画面の切りかえ
`.request()`	`api.ts` で作る関数の 1 つで、依頼 1 件のくわしい中身を問い合わせます。	06 後編 5. 依頼を出す
`.create()`	`api.ts` で作る関数の 1 つで、新しい依頼を送って作ります。作った依頼の番号（`id`）が返ります。	06 後編 5. 依頼を出す
`.upload()`	`api.ts` で作る関数の 1 つで、添付ファイルを 1 つ送ります。	06 後編 5. 依頼を出す
`.stats()`	`api.ts` で作る関数の 1 つで、集計（状態ごとの数など）を問い合わせます。	06 後編 7. グラフ

Python

コード	意味と使い方	初めて出てくる所
`True`	Python の「正しい（はい）」の値。反対は `False`。頭は大文字です（`true` ではありません）。	04 2. Python の書き方
`def`	関数を作る書き方。`def greet(name: str) -> str:` の次の行から、**字下げした所が関数の中身**です。Python は `{ }` の代わりに字下げで「どこまでが中身か」を表すので、字下げをそろえることがとても大事です。	04 2. Python の書き方
`return`	Python の関数の結果を返して、関数を終える命令。	04 2. Python の書き方
`for`	Python のくり返し。`for x in 並び:` の次の行から、字下げした所が中身です。`[式 for x in 並び]` の形（内包表記）で、新しいリストを 1 行で作ることもできます。	04 2. Python の書き方
`in`	`for x in 並び`（並びから 1 つずつ）と、`値 in 並び`（含まれているか）の 2 つの使い方があります。	04 2. Python の書き方
`if`	Python の条件。`if 条件:` の次の行から、字下げした所が中身です。`a if 条件 else b` の形で、1 行の中で値を選ぶこともできます。	04 2. Python の書き方

コード	意味と使い方	初めて出てくる所
`elif`	「そうでなくて、もし〜なら」（`else if` の短い形）。`if … elif … else` で、いくつもの場合に分けます。	04 2. Python の書き方
`else`	`if` の条件に合わなかった時の所。`if … else …` の 1 行の形でも使います。	04 2. Python の書き方
`.append()`	Python で、リストの後ろに 1 つ足すメソッド。	04 2. Python の書き方
`.get()`	辞書（`dict`）から、キーで値を取り出すメソッド。`d.get("port", 8080)` は、キーが無ければ 2 つ目の値（無ければ `None`）を返すので、`d["port"]` と違ってエラーになりません。	04 2. Python の書き方
`enumerate()`	並びを、番号つきで 1 つずつ取り出す関数。`for i, name in enumerate(approvers, start=1):` は、`i` に 1, 2, 3…、`name` に中身が入ります。	04 2. Python の書き方
`print()`	渡した値を画面に表示する関数。`print("a", 1)` のように、いくつも渡すと空白でつないで表示し、最後に改行します。	04 2. Python の書き方
`len()`	長さ（中身の数）を返す関数。`len("abc")` は 3、`len([1, 2])` は 2、`len(sys.argv)` は「渡された引数の数＋1」です。	04 2. Python の書き方
`#`	Python では、`#` から行の終わりまでがコメントです。	04 2. Python の書き方
`f"…{値}…"`	`f"…"`（頭に `f` を付けた文字）の中の `{ }` には、変数や式を書けて、その値が入ります（f 文字列）。`f"環境変数 {name} がありません"`。`{ }` の中の `:` の後ろには、`{amount:,}` のように書式も書けます。	04 2. Python の書き方
`[式 for 名前 in 並び]`	`[f"http://{host}" for host in ALLOWED_HOSTS]` は、「並びの 1 つずつから、新しい値を作って、並び（リスト）にする」書き方です（リスト内包表記）。後ろに `if 条件` を付けると、条件に合う物だけになります。	04 2. Python の書き方
`-> 型`	`def apply(self, action: str) -> None:` の `-> None` は、「この関数が返す物の型」の説明です（`None` は「何も返さない」）。	04 2. Python の書き方
`名前: 型`	`def apply(self, action: str)` の `: str` は、「この引数は文字」という型の説明（型ヒント）です。Python は動かす時には確かめませんが、エディターや mypy が間違いを教えてくれます。	04 2. Python の書き方
`名前: 型 =`	`app_name: str = "依頼・承認アプリ"` は、変数に型の説明を付けて、値を入れる書き方です。	04 2. Python の書き方

コード	意味と使い方	初めて出てくる所
`list[型]`	`list[str]` は「文字の並び」、`dict[str, int]` は「キーが文字で値が整数の辞書」という型です。	04 2. Python の書き方
`==`	`a == b` は、「a と b が等しいか」です。`=`（1 つ）は「値を入れる」です。	04 2. Python の書き方
`名前=値`	`open(path, encoding="utf-8")` の `encoding="utf-8"` のように、`名前=値` で渡す引数は「キーワード引数」です。順番に関係なく、名前で渡せます。	04 2. Python の書き方
`from`	`from パッケージ import 名前` の一部で、「どこから読み込むか」を書きます。`from pathlib import Path`。	04 3. 依頼の状態と、状態の移り変わり
`import`	ほかのモジュール（部品）を読み込む書き方。`import re` で正規表現の道具、`from pathlib import Path` で `Path` だけを使えるようにします。	04 3. 依頼の状態と、状態の移り変わり
`class`	クラス（データと、それを扱う関数をまとめた設計図）を作る書き方。`class Request:` の次から、字下げした所が中身です。	04 3. 依頼の状態と、状態の移り変わり
`None`	Python の「値が無い」ことを表す値。調べる時は `x is None` と書きます。	04 3. 依頼の状態と、状態の移り変わり
`not`	「〜ではない」。`if key not in TRANSITIONS:` は「辞書に無ければ」。	04 3. 依頼の状態と、状態の移り変わり
`raise`	エラー（例外）を起こして、処理を止める書き方。`raise TransitionError("…")`。呼んだ側は `try … except` で受け止められます。	04 3. 依頼の状態と、状態の移り変わり
`try`	Python で、失敗するかもしれない処理を `try:` の中に書き、失敗した時の処理を `except:` に書く書き方。	04 3. 依頼の状態と、状態の移り変わり
`except`	`try:` の中でエラーが起きた時に動く所。`except TransitionError as e:` は、その種類のエラーを `e` という名前で受け取ります。	04 3. 依頼の状態と、状態の移り変わり
`as`	Python で、別の名前を付ける書き方。`except Exception as error:`（起きたエラーを `error` で受け取る）・`import numpy as np`・`with open(…) as f:` のように使います。	04 3. 依頼の状態と、状態の移り変わり

コード	意味と使い方	初めて出てくる所
`field()`	`dataclasses` の部品で、dataclass の項目に、細かい決まりを付ける関数。`tags: list[str] = field(default_factory=list)` は「作るたびに、新しい空のリストを入れる」。`= []` と書くと、全部で同じリストを共有してしまうので、こう書きます。	04 3. 依頼の状態と、状態の移り変わり
`@dataclass`	クラスの上に付ける印（デコレーター）。項目（`title: str` など）を並べるだけで、作る時の処理（`__init__`）・表示の形・`==` の比べ方を自動で作ってくれます。	04 3. 依頼の状態と、状態の移り変わり
`"""…"""`	`"""…"""`（`"` を 3 つ）で囲んだ文字は、何行にもまたがって書けます。関数・クラス・ファイルのいちばん最初に書いた物は、その説明書（docstring）になり、`help()` やエディターで読めます。	04 3. 依頼の状態と、状態の移り変わり
`self`	クラスの中の関数（メソッド）の最初の引数 `self` は、「その関数を呼んだ、その物自身」です。`self.amount` で、自分の項目を読み書きします。呼ぶ時は、`self` は書きません（`r.apply("submit")`）。	04 3. 依頼の状態と、状態の移り変わり
`not in`	`key not in TRANSITIONS` は、「`key` が `TRANSITIONS` の中に無い」です。`in` の反対です。	04 3. 依頼の状態と、状態の移り変わり
`__name__`	`__name__` は、今のファイル（モジュール）の名前が入った、特別な変数です。ファイルを直接動かした時は `"__main__"` になります。`type(error).__name__` は、エラーの種類の名前です。	04 3. 依頼の状態と、状態の移り変わり
`__main__`	`if __name__ == "__main__":` の下は、「このファイルを直接動かした時だけ」動きます。ほかのファイルから `import` された時は、動きません。	04 3. 依頼の状態と、状態の移り変わり
`StrEnum`	`StrEnum`（`enum`）は、決まった値の一覧（列挙）を、文字として扱える形で作る型です。`class Status(StrEnum): DRAFT = "下書き"`。	04 3. 依頼の状態と、状態の移り変わり
`Exception`	`Exception` は、Python のふつうのエラー全部の、元になる型です。`class TransitionError(Exception):` で、自分のエラーの種類を作れます。`except Exception as e:` は、ほとんど全部のエラーを受け止めます。	04 3. 依頼の状態と、状態の移り変わり
`with`	使い終わったら自動で片づける書き方。`with path.open(encoding="utf-8") as f:` の中で使ったファイルは、抜ける時に自動で閉じられます。	04 5. 社員の CSV を読み込む
`or`	「または」。`a or b` は、`a` が空（`None`・`""`・0 など）なら `b` を使います。`(row.get("番号") or "").strip()` は「無ければ空の文字列にしてから空白を取る」。	04 5. 社員の CSV を読み込む

コード	意味と使い方	初めて出てくる所
`continue`	くり返しの「今回の残り」を飛ばして、次へ進む命令。	04 5. 社員のCSV を読み込む
`.open()`	`Path` のファイルを開くメソッド。`with path.open(encoding="utf-8") as f:` のように、`with` と一緒に使うと、使い終わった時に自動で閉じます。	04 5. 社員のCSV を読み込む
`.DictReader()`	`csv.DictReader(ファイル)` は、CSV を 1 行ずつ、「見出し → 値」の辞書として読む関数です。`row["従業員番号"]` のように、見出しの名前で値を取り出せます。	04 5. 社員のCSV を読み込む
`.strip()`	文字列の前後の空白や改行を取り除いたものを返すメソッド。	04 5. 社員のCSV を読み込む
`.fullmatch()`	`re.fullmatch(パターン, 文字列)` は、文字列の**全体**が正規表現（文字の形の決まり）に合うかを調べる関数です。`r"\d{6}"` は「数字がちょうど 6 つ」。合わなければ `None` です。	04 5. 社員のCSV を読み込む
`.add()`	`set`（同じ物を 2 回入れない入れ物）に 1 つ足すメソッド。すでにあれば何もしません。`no in seen` で「もう出てきたか」を確かめられます。	04 5. 社員のCSV を読み込む
`set()`	集合（同じ値は 1 つしか入らない入れ物）を作る関数。`seen = set()` で空の集合を作り、`seen.add(x)`・`x in seen` で使います。重複を見つける時に便利です。	04 5. 社員のCSV を読み込む
`r"…"`	`r"…"` は「生の文字列」で、中の `\` を特別な意味にしません。正規表現の `r"\d{6}"` のように、`\` の多い文字をそのまま書けます。	04 5. 社員のCSV を読み込む
`{値!r}`	f 文字列の `{no!r}` の `!r` は、値を `repr()` の形（プログラマー向けの形）で表示する指定です。文字なら `'012345'` のように `'` が付くので、空白や空の文字が見分けやすくなります。	04 5. 社員のCSV を読み込む
`[開始:終わり]`	`text[1:4]` は、1 番目から 4 番目の手前まで（1, 2, 3）を取り出します（スライス）。`[:3]` は最初から、`[3:]` は 3 番目から最後まで、`[::-1]` は逆さまです。	04 5. 社員のCSV を読み込む
`Path`	`Path`（`pathlib`）は、ファイルやフォルダの場所を表す型です。`Path("memo.txt").exists()`・`.read_text()`・`/` でのつなぎ（`BASE_DIR / "media"`）ができます。	04 5. 社員のCSV を読み込む
`UnicodeDecodeError`	`UnicodeDecodeError` は、ファイルを、決めた文字の符号（UTF-8 など）で読めなかった時のエラーです。Shift_JIS のファイルを UTF-8 として読んだ時などに起きます。	04 5. 社員のCSV を読み込む

コード	意味と使い方	初めて出てくる所
`.isoformat()`	日時を `2026-11-02T08:00:00+09:00` のような、決まった形（ISO 8601）の文字列にするメソッド。	04 6. 日本時間と、承認の URL の文字列
`.strftime()`	日時を、自分で決めた形の文字列にするメソッド。`%Y` は年・`%m` は月・`%d` は日・`%H` は時・`%M` は分。	04 6. 日本時間と、承認の URL の文字列
`.replace()`	`datetime` の `.replace(hour=8, minute=0)` は、一部だけ変えた新しい日時を返すメソッドです（文字列の `.replace("前", "後")` は、置きかえた新しい文字列を返す、別の意味です）。	04 6. 日本時間と、承認の URL の文字列
`.token_urlsafe()`	`secrets.token_urlsafe(32)` は、予想できないでたらめな文字列（URL に使える文字だけ）を作る関数です。1 回限りの承認の URL などに使います。	04 6. 日本時間と、承認の URL の文字列
`.sha256()`	`hashlib.sha256(バイト)` は、データの SHA-256 のハッシュ（指紋）を計算する関数です。	04 6. 日本時間と、承認の URL の文字列
`.encode()`	文字列を、バイトの並びにするメソッド（UTF-8）。`token.encode()` のように、ハッシュの計算など、バイトを求める関数に渡す前に使います。	04 6. 日本時間と、承認の URL の文字列
`.hexdigest()`	ハッシュの結果を、16 進数（0〜9 と a〜f）の文字列で返すメソッド。	04 6. 日本時間と、承認の URL の文字列
`.compare_digest()`	`hmac.compare_digest(a, b)` は、2 つの値が同じかを、かかる時間から中身を推測されない方法で比べる関数です。合言葉やハッシュを比べる時に使います。	04 6. 日本時間と、承認の URL の文字列
`datetime()`	日付と時刻を表す型（`datetime(2026, 11, 2, 8, 0)` のように作る）。`datetime.now(ZoneInfo("Asia/Tokyo"))` で、日本時間の今が取れます。	04 6. 日本時間と、承認の URL の文字列
`timedelta()`	時間の長さを表す型。`timedelta(minutes=60)` は 60 分。日時に足したり引いたりできます。	04 6. 日本時間と、承認の URL の文字列
`!=`	`a != b` は、「a と b が等しくないか」です。	04 6. 日本時間と、承認の URL の文字列

コード	意味と使い方	初めて出てくる所
`ZoneInfo`	`ZoneInfo("Asia/Tokyo")`（`zoneinfo`）は、時間帯（タイムゾーン）を表します。日時に付けると、日本の時刻として扱えます。	04 6. 日本時間と、承認の URL の文字列
`__str__`	`def __str__(self):` は、`str(物)` や `print(物)`、管理画面の一覧で表示する時の文字を決める関数です。	05 3. モデル: データの設計図
`date()`	日付を表す型。`date(2026, 11, 10)` のように作ります。	05 6. ORM: Python でデータベースを操作する
`{キー: 値 for …}`	`{u.pk: u for u in users}` は、「1 つずつから、キーと値の組を作って、辞書にする」書き方です（辞書内包表記）。	05 6. ORM: Python でデータベースを操作する
`is`	「同じ物か」を調べる書き方。`if value is None:` は「値が無ければ」。`None` と比べる時は `==` でなく `is` を使います。	06 前編 3. 設定
`False`	Python の「正しくない（いいえ）」の値。	06 前編 3. 設定
`.resolve()`	`Path(__file__).resolve()` は、ファイルの場所を、`C:\` から始まる絶対の場所にするメソッド。`.parent` で 1 つ上のフォルダになります。	06 前編 3. 設定
`.lower()`	英字を小文字にした文字列を返すメソッド。`"True".lower() == "true"` のように、大文字小文字をそろえて比べます。	06 前編 3. 設定
`.split()`	Python で、文字列を分けてリストにするメソッド。`"a b c".split()` のように何も渡さないと、空白（いくつ続いても）で分けます。	06 前編 3. 設定
`.exists()`	`Path` のファイルやフォルダがあれば `True` を返すメソッド。	06 前編 3. 設定
`.match()`	`re.match(パターン, 文字列)` は、文字列の**先頭から**正規表現に合うかを調べる関数です。合えば、見つかった物が返ります（合わなければ `None`）。	06 前編 3. 設定
`str()`	値を文字列に変える関数。`str(8080)` は `"8080"`。数と文字列は `==` で比べても等しくならないので、そろえる時に使います。	06 前編 3. 設定
`int()`	文字列などを整数にする関数。`int("25")` は 25。数でない文字だと `ValueError` です。	06 前編 3. 設定
`型 | None`	`str | None` は、「文字か、None（何も無い）」という型です。	06 前編 3. 設定
`__file__`	`__file__` は、今のファイルの場所が入った、特別な変数です。`Path(__file__).resolve().parent` で、このファイルが置いてあるフォルダが分かります。	06 前編 3. 設定

コード	意味と使い方	初めて出てくる所
`RuntimeError`	`RuntimeError` は、ほかの種類に当てはまらない、動いている時のエラーです。`raise RuntimeError("…")` で、説明を付けて止めます。	06 前編 3. 設定
`.parent`	`Path` の `.parent` は、1 つ上のフォルダです。`.parent.parent` で 2 つ上です。	06 前編 3. 設定
`.environ`	`os.environ` は、環境変数の入れ物（辞書のような物）です。`os.environ.get("名前", 無い時の値)` で読みます。	06 前編 3. 設定
`.update()`	辞書に、別の辞書の中身をまとめて入れる（上書きする）メソッド。Django の問い合わせの `.update(項目=値)` は、合う行をまとめて書きかえる、別の意味です。	06 前編 4. モデル
`**kwargs`	`def create_user(self, email, password=None, **extra):` の `**` は、「名前付きの引数を、いくつでも受け取り、辞書にまとめる」書き方です。	06 前編 4. モデル
`**辞書`	`create_user(email, **extra)` のように、呼ぶ時に辞書の前に `**` を付けると、辞書の中身を、名前付きの引数としてばらして渡します。	06 前編 4. モデル
`and`	「かつ」。両方が成り立つ時だけ `True`。左が成り立たない時は、右を調べません。	06 前編 5. 状態を変える処理を 1 か所に集める
`lambda`	その場で作る小さな関数。`lambda: send_mail(…)` は「呼ばれたらメールを送る関数」。名前を付けるほどでもない、短い処理を渡す時に使います。	06 前編 5. 状態を変える処理を 1 か所に集める
`{日時:%Y-%m-%d}`	f 文字列で日時の値に `{now:%m/%d %H:%M}` のように `%` の書式を書くと、その形の文字になります（`11/05 09:03`）。`%Y`（年）・`%m`（月）・`%d`（日）・`%H`（時）・`%M`（分）。	06 前編 5. 状態を変える処理を 1 か所に集める
`+=`	`total += n` は `total = total + n` の短い形です。	06 前編 5. 状態を変える処理を 1 か所に集める
`.dumps()`	`json.dumps(値, ensure_ascii=False)` は、Python の値を JSON の文字列にする関数です（`ensure_ascii=False` で日本語をそのまま）。	06 前編 7. 図面と添付ファイルの確かめ
`.rsplit()`	文字列を、後ろから区切るメソッド。`name.rsplit(".", 1)[-1]` は、ファイル名の最後の `.` の後ろ（拡張子）です。	06 前編 7. 図面と添付ファイルの確かめ

コード	意味と使い方	初めて出てくる所
`.startswith()`	文字列（バイト）が、その中身で始まっていれば `True` を返すメソッド。ファイルの先頭の数バイトで、本当の種類を確かめる時に使います。	06 前編 7. 図面と添付ファイルの確かめ
`isinstance()`	値が、その型かどうかを調べる関数。`isinstance(value, dict)` は「辞書か」。	06 前編 7. 図面と添付ファイルの確かめ
`any()`	並びの中に、1 つでも `True` があれば `True` を返す関数。	06 前編 7. 図面と添付ファイルの確かめ
`b"…"`	`b"…"` は、文字ではなく「バイトの並び」（`bytes`）を作る書き方です。`b"\xff\xd8\xff"` は、JPEG の画像のファイルの、最初の 3 バイトです。	06 前編 7. 図面と添付ファイルの確かめ
`[-1]`	`items[-1]` は、最後の 1 つです。`[-2]` は後ろから 2 つ目です。	06 前編 7. 図面と添付ファイルの確かめ
`A if 条件 else B`	`a if 条件 else b` は、「条件が合えば a、合わなければ b」になる式です。`ext = name.rsplit(".", 1)[-1] if "." in name else ""` 。	06 前編 7. 図面と添付ファイルの確かめ
`.read()`	ファイルの中身を全部読んで、文字列で返すメソッド。`open(パス, encoding="utf-8").read()` 。	06 前編 8. React の画面が呼ぶ API
`.seek()`	ファイルの読む位置を動かすメソッド。`f.seek(0)` で先頭に戻ります（先頭の数バイトを読んで確かめたあと、保存のために戻す）。	06 前編 8. React の画面が呼ぶ API
`dict()`	辞書を作る関数。`dict(Request.Status.choices)` は、「値 → 画面の名前」の辞書になります。	06 前編 8. React の画面が呼ぶ API
`.write()`	ファイルに文字を書くメソッド。`f.write(line + "\n")`（改行は自分で付けます）。	06 前編 9. 社員の CSV の取り込み
`async`	Python で、中で `await` を使える関数にする印（`async def handle(…):`）。`asyncio` の仕組みの上で、たくさんの接続を同時にさばけます。	09 2. SMTP の会話を見る
`await`	Python の `async` の関数の中で、時間のかかる処理が終わるまで待つ書き方。`await writer.drain()` 。	09 2. SMTP の会話を見る
`while`	Python で、条件が `True` の間くり返す書き方。`while True:` は、`break` で抜けるまでずっとくり返します。	09 2. SMTP の会話を見る

コード	意味と使い方	初めて出てくる所
`break`	Python のくり返しを、途中で抜ける命令。	09 2. SMTP の会話を見る
`pass`	「何もしない」という Python の命令。中身が要らないけれど、形の上で何か書く必要がある所に置きます。	09 2. SMTP の会話を見る
`.Path()`	`pathlib.Path(場所)` は、ファイルやフォルダの場所を表す物を作る関数です。`/` でつなげられます。	09 2. SMTP の会話を見る
`.with_name()`	`Path` の、最後の名前だけを取りかえた場所を返すメソッド。`Path(__file__).with_name("received")` は「このファイルと同じフォルダの received」。	09 2. SMTP の会話を見る
`.get_extra_info()`	`asyncio` の接続から、相手の住所などの情報を取り出すメソッド。	09 2. SMTP の会話を見る
`.drain()`	`asyncio` の接続で、書いた物を送り終わるまで待つメソッド（`await writer.drain()`）。	09 2. SMTP の会話を見る
`.readline()`	1 行だけ読むメソッド。	09 2. SMTP の会話を見る
`.decode()`	バイトの並びを、文字列に直すメソッド。`data.decode("utf-8", "replace")` は、読めない所を `�` にして直します。	09 2. SMTP の会話を見る
`.rstrip()`	文字列の後ろ（右側）から、空白や指定した文字を取り除くメソッド。`.rstrip("\r\n")` は行の終わりの改行を取ります。	09 2. SMTP の会話を見る
`.upper()`	英字を大文字にした文字列を返すメソッド。	09 2. SMTP の会話を見る
`.start_tls()`	`asyncio` の接続を、途中から暗号化（TLS）に切りかえるメソッド（STARTTLS。09 巻）。	09 2. SMTP の会話を見る
`.mkdir()`	`Path` のフォルダを作るメソッド。`exist_ok=True` で「もうあってもよい」、`parents=True` で途中のフォルダもまとめて作ります。	09 2. SMTP の会話を見る
`.write_bytes()`	`Path` のファイルに、バイトの並びをそのまま書くメソッド。	09 2. SMTP の会話を見る
`.join()`	Python で、区切り文字の方に付けて、リストの中身をつなぐメソッド。`" ".join(["a", "b"])` は `"a b"`（TypeScript とは書く順番が逆なので注意）。	09 2. SMTP の会話を見る
`.close()`	接続やファイルを閉じるメソッド。	09 2. SMTP の会話を見る

コード	意味と使い方	初めて出てくる所
`.ArgumentParser()`	`argparse.ArgumentParser()` は、コマンドの引数（`--port 2525` など）を読み取る係を作る関数です。	09 2. SMTP の会話を見る
`.parse_args()`	決めておいた引数の形で、実際に渡された引数を読み取るメソッド。	09 2. SMTP の会話を見る
`.create_default_context()`	`ssl.create_default_context()` は、暗号化（TLS）の設定を、おすすめの値で作る関数です。	09 2. SMTP の会話を見る
`.load_cert_chain()`	TLS の設定に、サーバーの証明書と秘密鍵のファイルを読み込むメソッド。	09 2. SMTP の会話を見る
`.start_server()`	`asyncio.start_server(関数, ホスト, ポート)` は、接続を待ち受けるサーバーを始める関数です。接続が来るたびに、渡した関数が動きます。	09 2. SMTP の会話を見る
`.serve_forever()`	HTTP のサーバーを、止めるまでずっと動かし続けるメソッド。	09 2. SMTP の会話を見る
`.run()`	`asyncio.run(main())` は、非同期の関数（`async def`）を動かして、終わるまで待ちます。非同期のプログラムは、ここから始まります。	09 2. SMTP の会話を見る
`:=`	`while (line := await recv()) is not None:` の `:=` は、「値を変数に入れて、その値をその場で使う」書き方です（セイウチ演算子）。	09 2. SMTP の会話を見る
`is not`	`x is not None` は、「x が None ではない」です。`None` かどうかは、`==` ではなく `is` で調べます。	09 2. SMTP の会話を見る
`KeyboardInterrupt`	`KeyboardInterrupt` は、`Ctrl` + `C` で止めた時に起きる、エラーの種類です。`except KeyboardInterrupt:` で受け止めると、止める時の後片づけができます。	09 2. SMTP の会話を見る
`.set_content()`	メール（`EmailMessage`）の本文を決めるメソッド。	09 2. SMTP の会話を見る
`.SMTP()`	`smtplib.SMTP(ホスト, ポート)` は、メールを送るためにメールのサーバー（SMTP）へつなぐ関数です。	09 2. SMTP の会話を見る
`.set_debuglevel()`	`smtp.set_debuglevel(1)` で、SMTP のやり取りを画面に表示します（練習や、届かない時の調べ）。	09 2. SMTP の会話を見る
`.ehlo()`	SMTP のサーバーに、最初のあいさつ（EHLO）を送るメソッド。	09 2. SMTP の会話を見る

コード	意味と使い方	初めて出てくる所
`.starttls()`	`smtplib` で、メールサーバーとの接続を、途中から暗号化（TLS）に切りかえるメソッド。`smtp.starttls()` のあとに送る中身は、暗号化されます。	09 2. SMTP の会話を見る
`.send_message()`	用意したメールを、SMTP のサーバーへ渡して送るメソッド。	09 2. SMTP の会話を見る
`EmailMessage`	`EmailMessage`（`email.message`）は、メール 1 通（宛先・件名・本文）を組み立てる型です。	09 2. SMTP の会話を見る
`.endswith()`	文字列が、その文字で終わっていれば `True` を返すメソッド。	09 7. アプリの設定と、送り方のコード
`.category()`	`unicodedata.category(c)` は、文字の種類を 2 文字で返す関数です。`"Cc"` は制御文字（改行・タブなどの、目に見えない特別な文字）。	10 7. 入力の確かめ: 改行を混ぜられる
`.zfill()`	文字列の前を 0 で埋めて、決めた桁にするメソッド。`"7".zfill(4)` は `"0007"`。	11 B. サンプル集
`float()`	文字列などを小数にする関数。`float("3.5")` は 3.5。数でない時は `ValueError`。	11 B. サンプル集
`round()`	数を丸める関数。`round(2.675, 2)` は小数第 2 位まで。ちょうど真ん中（`2.5`）は、偶数の方へ丸める（`round(2.5)` は 2、`round(3.5)` は 4）ので注意します。	11 B. サンプル集
`max()`	いちばん大きい物を返す関数。`max(3, 9, 4)` は 9、`max([5, 1, 8])` は 8。いちばん小さい物は `min()` です。	11 B. サンプル集
`min()`	Python で、いちばん小さい値を返す関数。`min(1, 5, 3)` は 1。	11 B. サンプル集
`sum()`	リストなどの中身を全部足した合計を返す関数。`sum([1, 2, 3])` は 6。	11 B. サンプル集
`abs()`	絶対値（マイナスを取った値）を返す関数。	11 B. サンプル集
`{値:,}`	f 文字列の `{amount:,}` の `,` は、3 けたごとに `,` を入れる指定です（`1234567` → `1,234,567`）。`{amount:,.2f}` なら、区切りと小数点以下 2 けたの両方です。	11 B. サンプル集
`{値:.桁f}`	f 文字列の `{ratio:.2f}` の `.2f` は、「小数点の下を 2 けたにした小数」で表示する指定です（`3.14159` → `3.14`）。	11 B. サンプル集
`{値:0幅}`	f 文字列の `{42:05d}` の `05` は、「5 けたになるまで、前を 0 で埋める」指定です（`00042`）。	11 B. サンプル集

コード	意味と使い方	初めて出てくる所
`{値:d}`	f 文字列の `{n:d}` の `d` は、整数として表示する指定です。	11 B. サンプル集
`{値:>幅}`	f 文字列の `{'右':>6}` の `>` と数は、「6 文字分の場所に、右寄せで入れる」指定です。	11 B. サンプル集
`{値:<幅}`	f 文字列の `{where:<16}` の `<` と数は、「16 文字分の場所に、左寄せで入れる」指定です。足りない分は右に空白が入り、後ろの列がそろいます。`>` なら右寄せ、`^` なら中央です。	11 B. サンプル集
`{値:^幅}`	f 文字列の `{'中':^6}` の `^` と数は、「6 文字分の場所の、真ん中に入れる」指定です。	11 B. サンプル集
`{値:.桁%}`	f 文字列の `{0.256:.1%}` の `%` は、100 倍して `%` を付ける指定です（`25.6%`）。`.1` は小数点の下のけたの数です。	11 B. サンプル集
`//`	Python の `7 // 2` は、割り算の答えの小数を切り捨てた整数（`3`）です。`7 / 2` は `3.5` です。	11 B. サンプル集
`**`	Python の `2 ** 10` は、2 の 10 乗（`1024`）です。	11 B. サンプル集
`%`	Python の `7 % 2` は、割った余り（`1`）です。	11 B. サンプル集
`ValueError`	`ValueError` は、「型は合っているが、値がおかしい」時のエラーです。`int("abc")` で起きます。	11 B. サンプル集
`.insert()`	リストの、決めた位置に入れるメソッド。`names.insert(0, "鈴木")` は先頭に入れます。	11 B. サンプル集
`.pop()`	リストの最後（または決めた位置）の 1 つを取り出すメソッド（リストから消える）。	11 B. サンプル集
`.items()`	辞書の「キーと値」の組を 1 つずつ取り出せる形にするメソッド。`for k, v in d.items():` で、両方を同時に使えます。	11 B. サンプル集
`.keys()`	辞書のキーだけを、1 つずつ取り出せる形で返すメソッド。`list(d.keys())` でリストにできます。	11 B. サンプル集
`.most_common()`	`Counter` の、多い順に「(値, 数)」の組を並べたリストを返すメソッド。`most_common(1)` で、いちばん多い物だけ。	11 B. サンプル集
`sorted()`	並べかえた**新しい**リストを返す関数。`sorted(並び, key=lambda r: r["amount"], reverse=True)` の `key` で「何で並べるか」、`reverse=True` で大きい順。	11 B. サンプル集
`zip()`	2 つ以上の並びを、同じ番号どうしの組にして取り出す関数。`for a, b in zip(順番, 名前):`。	11 B. サンプル集

コード	意味と使い方	初めて出てくる所
`list()`	並び（`keys()` の結果や文字列など）から、新しいリストを作る関数。`list("abc")` は `["a", "b", "c"]`。	11 B. サンプル集
`defaultdict()`	無いキーを初めて使った時に、自動で初期値（`list` なら空のリスト）を作る辞書。グループ分けに便利です。	11 B. サンプル集
`(式 for … in …)`	`sum(x * 2 for x in nums)` のように `( )` の中に書く内包表記は、値を 1 つずつ作って渡す物（ジェネレーター）です。並びを作らないので、メモリを使いません。	11 B. サンプル集
`Counter`	`Counter(並び)`（`collections`）は、同じ物がいくつずつあるかを数えます。`Counter(["a", "b", "a"])` → `{'a': 2, 'b': 1}`。	11 B. サンプル集
`@property`	クラスのメソッドの上に付けると、`()` 無しで値のように読めるようにする印。	11 B. サンプル集
`*args`	`def f(*args):` の `*args` は、「いくつでも引数を受け取り、タプル（並び）にまとめる」書き方です。	11 B. サンプル集
`*並び`	`f(*items)` のように、呼ぶ時に並びの前に `*` を付けると、中身を、1 つずつの引数としてばらして渡します。	11 B. サンプル集
`__init__`	`def __init__(self, …):` は、クラスから物を作る時（`Item("題", 100)`）に、自動で呼ばれる関数です。受け取った値を `self.title = title` のように覚えます。	11 B. サンプル集
`__repr__`	`def __repr__(self):` は、プログラマー向けの表示（`repr(物)`）の文字を決める関数です。	11 B. サンプル集
`finally`	`try … except` のあと、成功しても失敗しても最後に必ず動く所。	11 B. サンプル集
`.write_text()`	`Path` のファイルに、文字列を書くメソッド。`encoding="utf-8"` を付けます。	11 B. サンプル集
`.read_text()`	`Path` のファイルの中身を、全部文字列として読むメソッド。`encoding="utf-8"` を付けて読みます。	11 B. サンプル集
`.splitlines()`	文字列を、行ごとに分けてリストにするメソッド（改行の文字は付かない）。	11 B. サンプル集
`.stat()`	ファイルの情報（大きさ `st_size`・日時など）を取り出すメソッド。	11 B. サンプル集
`.loads()`	文字列から読み取るメソッド。`tomllib.loads(文字列)` は TOML、`json.loads(文字列)` は JSON を、Python の辞書にします。	11 B. サンプル集

コード	意味と使い方	初めて出てくる所
`.DictWriter()`	`csv.DictWriter(ファイル, fieldnames=[…])` は、辞書を 1 行ずつ CSV に書く係を作る関数です。	11 B. サンプル集
`.writeheader()`	CSV の 1 行目（見出し）を書くメソッド。	11 B. サンプル集
`.writerows()`	辞書のリストを、まとめて CSV の行として書くメソッド。	11 B. サンプル集
`.iterdir()`	`Path` のフォルダの中身を、1 つずつ取り出すメソッド（順番は決まっていないので、`sorted` で並べる）。	11 B. サンプル集
`.rmtree()`	`shutil.rmtree(フォルダ)` は、フォルダを中身ごと消す関数です。ごみ箱には入らないので、よく確かめて使います。	11 B. サンプル集
`FileNotFoundError`	`FileNotFoundError` は、無いファイルを開こうとした時のエラーです。	11 B. サンプル集
`.suffix`	`Path` の `.suffix` は、拡張子（`.txt`）です。`.name` はファイル名、`.stem` は拡張子を除いた名前です。	11 B. サンプル集
`.strptime()`	`datetime.strptime("2026/11/02 08:00", "%Y/%m/%d %H:%M")` は、決めた形の文字列から日時を読む関数です。	11 B. サンプル集
`.fromisoformat()`	`date.fromisoformat("2026-12-01")` は、`2026-12-01` の形の文字列から日付を作る関数です。	11 B. サンプル集
`.weekday()`	曜日を数で返すメソッド（月曜が 0、日曜が 6）。	11 B. サンプル集
`.astimezone()`	日時を、別の時間帯（世界標準時・日本時間など）で表したものに変えるメソッド。	11 B. サンプル集
`.days`	日時の差（`timedelta`）の `.days` は、日数です。	11 B. サンプル集
`.tzinfo`	日時の `.tzinfo` は、時間帯（タイムゾーン）の情報です。`None` なら、時間帯が付いていない日時です。	11 B. サンプル集
`.year`	日時の `.year` は年です（`.month` は月、`.day` は日）。	11 B. サンプル集

Django

コード	意味と使い方	初めて出てくる所
`.CharField()`	`models.CharField("件名", max_length=100)` は、短い文字列の列（最大の長さを必ず決める）を表すモデルの項目です。最初の `"件名"` は、管理画面などに出る名前です。	05 3. モデル: データの設計図
`.TextField()`	長い文字列（何行もの文）の列を表すモデルの項目。`blank=True` を付けると、入力が空でもよくなります。	05 3. モデル: データの設計図

コード	意味と使い方	初めて出てくる所
`.DateField()`	日付の列を表すモデルの項目。	05 3. モデル: データの設計図
`.ForeignKey()`	ほかの表の 1 行を指す列（外部キー）を表すモデルの項目。`models.ForeignKey(User, on_delete=models.PROTECT, related_name="requests")` は「ユーザーを指す。指されているユーザーは消せない（PROTECT）。ユーザーの側からは `user.requests` で依頼をたどれる」。	05 3. モデル: データの設計図
`.DateTimeField()`	日時の列を表すモデルの項目。`auto_now_add=True` で、作った時の日時を自動で入れます。	05 3. モデル: データの設計図
`.PositiveSmallIntegerField()`	0 以上の小さな整数の列を表すモデルの項目（順番などに使う）。	05 3. モデル: データの設計図
`.UniqueConstraint()`	「この組み合わせの値は、表の中で重ならない」という決まりを表す物。`fields=["request", "order"]` は「同じ依頼の中で、同じ順番は 2 つ作れない」。	05 3. モデル: データの設計図
`"-列の名前"`	Django の `ordering = ["-created_at"]` や `.order_by("-created_at")` の、名前の頭の `-` は、「大きい順（新しい順）に並べる」という意味です。`-` が無ければ、小さい順（古い順）です。	05 3. モデル: データの設計図
`.choices`	Django の `Status.choices` は、選び肢の一覧（値と表示名の組）です。`CharField(choices=Status.choices)` で、管理画面に選ぶ欄ができます。	05 3. モデル: データの設計図
`.register()`	`@admin.register(Request)` は、モデルを管理画面に登録して、画面から見たり直したりできるようにする書き方です。	05 5. 管理画面
`@admin`	`@admin.register(モデル)` を、管理画面の設定のクラスの上に付けて、そのモデルを管理画面に出します。	05 5. 管理画面
`.all()`	`Request.objects.all()` は、表の全部の行を取り出す（まだ読みに行かない「問い合わせ」を作る）メソッドです。	05 6. ORM: Python でデータベースを操作する
`.delete()`	問い合わせに合う行（または 1 つの物）を、データベースから消すメソッド。消した件数が返ります。	05 6. ORM: Python でデータベースを操作する
`.filter()`	Django の問い合わせで、条件に合う行だけに絞るメソッド。`Request.objects.filter(status="approved")`。`項目名__条件`（`title__icontains="治具"` など）で、いろいろな条件を書けます（11 巻の B-18）。	05 6. ORM: Python でデータベースを操作する

コード	意味と使い方	初めて出てくる所
`.create_user()`	Django の利用者（`User`）を作るメソッド。`User.objects.create_user("012345", password="…")`。パスワードは、そのまま保存せずに、ハッシュにしてから保存してくれます（`create()` で作ると、ハッシュにならないので使わない）。	05 6. ORM: Python でデータベースを操作する
`.create()`	Django で、新しい行を作ってデータベースに保存するメソッド。`Request.objects.create(title="…", …)` は、作ると同時に保存します（`.save()` は要らない）。	05 6. ORM: Python でデータベースを操作する
`.count()`	Django で、条件に合う行の数を数えるメソッド。`Request.objects.count()` は、全部の依頼の数。データベースに `SELECT count(*)` を送るので、全部を読み込むより速いです。	05 6. ORM: Python でデータベースを操作する
`.order_by()`	問い合わせの結果を並べかえるメソッド。`order_by("due_date")` は納期の近い順、`order_by("-due_date")` は遠い順（`-` で逆）。	05 6. ORM: Python でデータベースを操作する
`.save()`	Django のモデル（1 行分の値）を、データベースに保存するメソッド。新しい物なら行を足し、もうある物なら行を変えます。	05 6. ORM: Python でデータベースを操作する
`.get_status_display()`	`choices` を決めた項目の、画面に出す名前（`"approved"` → `"回覧完了"`）を返すメソッド。Django が `get_項目名_display()` を自動で作ります。	05 6. ORM: Python でデータベースを操作する
`.values()`	Django で、行をモデルではなく、決めた列だけの辞書で受け取るメソッド。`Request.objects.values("status").annotate(n=Count("id"))` は「状態ごとの数」を数える時の書き方です。	05 6. ORM: Python でデータベースを操作する
`.annotate()`	問い合わせの 1 行ごとに、計算した値（数など）を付け足すメソッド。`values("status").annotate(n=Count("id"))` は「状態ごとの件数」（SQL の GROUP BY）。	05 6. ORM: Python でデータベースを操作する
`.select_related()`	つながった表（外部キーの先）を、1 回の問い合わせでまとめて読むメソッド。`select_related("approver")` で、ループの中で承認者を 1 人ずつ読みに行く無駄（N+1）を防げます。	05 6. ORM: Python でデータベースを操作する
`get_user_model()`	今のプロジェクトで使っている「ユーザー」のモデルを返す関数。自分でユーザーのモデルを作った時も、これで正しく取れます。	05 6. ORM: Python でデータベースを操作する

コード	意味と使い方	初めて出てくる所
`列__条件=`	Django の ORM の `filter(created_at__gte=日時)` のように、列の名前の後ろに `__` と条件を付けると、いろいろな比べ方ができます。`__gte`（以上）・`__lte`（以下）・`__gt`（より大きい）・`__in`（並びのどれか）・`__icontains`（大文字・小文字を区別せず、含む）・`__isnull`（空か）など。	05 6. ORM: Python でデータベースを操作する
`外の表__列=`	Django の ORM の `filter(requester__dept="製造")` のように、`__` でつなぐと、つながった別の表（外部キーの先）の列で、しぼり込めます。	05 6. ORM: Python でデータベースを操作する
`Count`	Django の `Count("id")` は、ORM の集計で、数を数えます。`.annotate(n=Count("id"))` で、グループごとの件数が付きます。	05 6. ORM: Python でデータベースを操作する
`.objects`	Django の `Request.objects` は、その表の行を取り出す係（マネージャー）です。`.all()`（全部）・`.filter(…)`（しぼる）・`.get(…)`（1 つだけ）・`.create(…)`（作る）などを付けて使います。	05 6. ORM: Python でデータベースを操作する
`.pk`	Django の `.pk` は、その行の主キー（ふつうは `id`）です。	05 6. ORM: Python でデータベースを操作する
`.DoesNotExist`	`Request.DoesNotExist` は、`.get(…)` で、合う行が 1 つも無かった時のエラーです。`except Request.DoesNotExist:` で受け止めます。	05 6. ORM: Python でデータベースを操作する
`render()`	Django で、テンプレート（HTML のひな形）に値を入れて、答え（HTML）を作る関数。`render(request, "テンプレート名", {"名前": 値})`。	05 7. HTML の画面
`@login_required`	関数の画面（ビュー）の上に付けると、ログインしていない人をログインの画面へ移す印（デコレーター）です。	05 7. HTML の画面
`.user`	Django の `request.user` は、今ログインしている人（していなければ「だれでもない人」）です。`request.user.is_authenticated` で、ログインしているかが分かります。	05 7. HTML の画面
`path()`	URL と、それに答える画面（ビュー）を結び付ける関数。`path("requests/<int:pk>/", views.detail)` の `<int:pk>` は、URL の中の数を `pk` という名前で受け取ります。	05 7. HTML の画面
`include()`	URL の決まりの中で、ほかのファイル（アプリ）の URL の決まりを、まとめて取り込む関数。`path("api/", include("requests_app.urls"))` のように使います。	05 7. HTML の画面

コード	意味と使い方	初めて出てくる所
`.urls`	`admin.site.urls` は、Django の管理画面の URL の一覧です。`path("admin/", admin.site.urls)` で、`/admin/` に管理画面を出します。	05 7. HTML の画面
`.site`	`admin.site` は、Django の管理画面そのものです。`admin.site.register(モデル)` で、モデルを管理画面に出します。	05 7. HTML の画面
`{% load %}`	Django のテンプレートの `{% load static %}` は、`{% static %}` などの道具を使えるようにする命令です。テンプレートの最初に書きます。	05 7. HTML の画面
`{% static %}`	`{% static 'style.css' %}` は、静的なファイル（CSS・画像）の URL（`/static/style.css`）を作る命令です。	05 7. HTML の画面
`{% for %}`	Django のテンプレートの `{% for r in items %} … {% endfor %}` は、並びの 1 つずつについて、間の HTML をくり返します。	05 7. HTML の画面
`{% empty %}`	`{% for %}` の中の `{% empty %}` の下は、並びが空の時だけ表示されます。「依頼はまだありません」のような文に使います。	05 7. HTML の画面
`{% endfor %}`	`{% endfor %}` は、`{% for %}` のくり返しの終わりの印です。	05 7. HTML の画面
`{{ 値 }}`	Django のテンプレートの `{{ r.title }}` は、値をそこに表示します。中の `<` などは自動で `<` に変わる（エスケープされる）ので、入力された文字がタグとして動くことはありません。	05 7. HTML の画面
`{{ 値|default }}`	`{{ r.note|default:"なし" }}` の `|default:"なし"` は、値が空の時に、代わりに表示する文字です（`|` の後ろは「フィルター」という、値の変え方です）。	05 7. HTML の画面
`{{ 値|date }}`	`{{ r.created_at|date:"Y-m-d H:i" }}` の `|date` は、日時を、その形の文字にするフィルターです。`Y`（年）・`m`（月）・`d`（日）・`H`（時）・`i`（分）。	05 7. HTML の画面
`.assertEqual()`	テストで、2 つの値が同じかを確かめるメソッド。`self.assertEqual(res.status_code, 403)`。	05 9. 自動のテスト
`.json()`	テストで受け取った答え（レスポンス）の本文を、JSON として読んで、Python のリストや辞書にするメソッド。`self.assertEqual(res.json(), [])`。	05 9. 自動のテスト
`.post()`	テストの係（`self.client`）で、POST のリクエストを送るメソッド。`self.client.post("/api/requests/", {…})` で、サーバーを動かさずに、API を試せます。	05 9. 自動のテスト

コード	意味と使い方	初めて出てくる所
`.assertIn()`	テストで、値が入れ物の中にあるかを確かめるメソッド。`self.assertIn("title", res.json())`。	05 9. 自動のテスト
`.status_code`	テストの答えの `res.status_code` は、HTTP の状態コード（`200`・`403` など）です。	05 9. 自動のテスト
`.set_password()`	ユーザーのパスワードを、元に戻せない形（ハッシュ）にしてから設定するメソッド。パスワードをそのまま保存しないために、必ずこれを使います。	06 前編 4. モデル
`.set_unusable_password()`	そのユーザーを、パスワードではログインできないようにするメソッド（別の方法でログインする人や、作ったばかりの人に使う）。	06 前編 4. モデル
`.EmailField()`	メールアドレスの列を表すモデルの項目。入力がメールアドレスの形かを確かめてくれます。	06 前編 4. モデル
`.BooleanField()`	はい・いいえ（`True`・`False`）の列を表すモデルの項目。	06 前編 4. モデル
`AbstractBaseUser`	Django の `AbstractBaseUser` は、ログインする人（ユーザー）のモデルを、自分で作る時の元になる型です。パスワードの保存や確かめの仕組みが入っています。	06 前編 4. モデル
`BaseUserManager`	Django の `BaseUserManager` は、ユーザーを作る係（`create_user`・`create_superuser`）を書く時の、元になる型です。	06 前編 4. モデル
`PermissionsMixin`	Django の `PermissionsMixin` は、ユーザーのモデルに、管理者かどうか・グループ・権限の項目を足す部品です。	06 前編 4. モデル
`.JSONField()`	JSON（図面のデータなど）をそのまま入れる列を表すモデルの項目。PostgreSQL では、中を検索することもできます。	06 前編 4. モデル
`.FileField()`	上げられたファイルの置き場所を入れる列を表すモデルの項目。`upload_to="attachments/%Y/%m/"` で、年・月ごとのフォルダに保存します。	06 前編 4. モデル
`.PositiveIntegerField()`	0 以上の整数の列を表すモデルの項目。	06 前編 4. モデル
`.now()`	`timezone.now()` は、今の日時を、時差付きで返します（Django の設定に合わせた形）。Python の `datetime.now()` は時差が付かないので、Django ではこちらを使います。	06 前編 5. 状態を変える処理を 1 か所に集める
`.select_for_update()`	問い合わせで読んだ行に、ほかの人が同時に書きかえられないよう鍵をかけるメソッド（`transaction.atomic()` の中で使う）。2 人が同時に承認しても、順番に処理されます。	06 前編 5. 状態を変える処理を 1 か所に集める

コード	意味と使い方	初めて出てくる所
`.first()`	問い合わせの結果の最初の 1 件を返すメソッド。無ければ `None` です（`get()` と違い、エラーになりません）。	06 前編 5. 状態を変える処理を 1 か所に集める
`.on_commit()`	`transaction.on_commit(関数)` は、データベースへの保存が確定（コミット）してから関数を動かす書き方。保存が取り消されたのにメールだけ送られる、という事故を防ぎます。	06 前編 5. 状態を変える処理を 1 か所に集める
`.localtime()`	`timezone.localtime()` は、今の時刻を、設定した時間帯（日本時間）で返す関数です。	06 前編 5. 状態を変える処理を 1 か所に集める
`render_to_string()`	テンプレート（文の型）に値を入れて、文字列（メールの本文など）を作る関数。	06 前編 5. 状態を変える処理を 1 か所に集める
`send_mail()`	メールを送る関数。`send_mail(件名, 本文, 差出人, [宛先])`。送り先のサーバー（SMTP）は設定で決めます（09 巻）。	06 前編 5. 状態を変える処理を 1 か所に集める
`@transaction`	`@transaction.atomic` を関数の上に付けると、その関数の中のデータベースの変更が「全部成功か、全部取り消し」になります。	06 前編 5. 状態を変える処理を 1 か所に集める
`require_http_methods()`	画面（ビュー）が受け付ける HTTP のメソッドを決める物。`@require_http_methods(["GET", "POST"])` 以外で来たら、405 で断ります。	06 前編 6. 1 回限りの承認の URL
`@require_http_methods`	`@require_http_methods(["GET", "POST"])` を画面の関数の上に付けて、使えるメソッドを絞ります。	06 前編 6. 1 回限りの承認の URL
`.method`	Django の `request.method` は、要求の種類（`"GET"`・`"POST"`）です。	06 前編 6. 1 回限りの承認の URL
`.POST`	Django の `request.POST` は、フォームで送られた値の入れ物です。`request.POST.get("decision")` で読みます。	06 前編 6. 1 回限りの承認の URL
`{% extends %}`	Django のテンプレートの `{% extends "workflow/base.html" %}` は、「土台のテンプレートを受け継ぎ、`{% block %}` の所だけを書きかえる」という宣言です。ページの共通の外枠を、1 か所にまとめられます。	06 前編 6. 1 回限りの承認の URL

コード	意味と使い方	初めて出てくる所
`{% block %}`	`{% block title %} … {% endblock %}` は、土台のテンプレートの中で「ここは、ページごとに書きかえる」場所です。受け継いだテンプレートで、同じ名前の `block` を書くと、その中身に置きかわります。	06 前編 6. 1 回限りの承認の URL
`{% endblock %}`	`{% endblock %}` は、`{% block %}` の終わりの印です。	06 前編 6. 1 回限りの承認の URL
`{% if %}`	Django のテンプレートの `{% if 条件 %} … {% endif %}` は、条件が合う時だけ、間の HTML を表示します。`{% else %}` も書けます。	06 前編 6. 1 回限りの承認の URL
`{% endif %}`	`{% endif %}` は、`{% if %}` の終わりの印です。	06 前編 6. 1 回限りの承認の URL
`{% csrf_token %}`	`{% csrf_token %}` は、フォームの中に書く、CSRF（ほかのサイトからのなりすましの送信）を防ぐための、見えない合言葉の欄を作る命令です。Django は、POST のフォームにこれが無いと、403 で断ります。	06 前編 6. 1 回限りの承認の URL
`{{ 値|linebreaksbr }}`	`{{ r.detail|linebreaksbr }}` の `|linebreaksbr` は、文の中の改行を、HTML の改行（` `）にして表示するフィルターです。	06 前編 6. 1 回限りの承認の URL
`ValidationError`	Django の `ValidationError` は、入力の値が決まりに合わない時に起こすエラーです。フォームや DRF が受け止めて、利用者にメッセージを返します。	06 前編 7. 図面と添付ファイルの確かめ
`.prefetch_related()`	「たくさん」の側のつながり（1 つの依頼の、何人もの承認者など）を、まとめて読んでおくメソッド。N+1 の問い合わせを防ぎます。	06 前編 8. React の画面が呼ぶ API
`.distinct()`	問い合わせの結果から、重なった行を消すメソッド。JOIN で同じ依頼が何度も出る時に使います。	06 前編 8. React の画面が呼ぶ API
`.exclude()`	問い合わせから、条件に合う行を除くメソッド（`filter` の逆）。	06 前編 8. React の画面が呼ぶ API
`get_object_or_404()`	1 件を取り出し、無ければ 404（見つかりません）の答えにする関数。	06 前編 8. React の画面が呼ぶ API
`DjangoValidationError`	`from django.core.exceptions import ValidationError as DjangoValidationError` で、Django の `ValidationError` を、DRF の同じ名前の物と取り違えないように、別の名前で読み込んでいます。	06 前編 8. React の画面が呼ぶ API

コード	意味と使い方	初めて出てくる所
`Q`	Django の `Q(…)` は、ORM の条件を、`|`（または）・`&`（かつ）・`~`（ではない）で組み合わせる道具です。`filter(Q(requester=u) | Q(approver=u))`。	06 前編 8. React の画面が呼ぶ API
`FileResponse`	Django の `FileResponse(開いたファイル)` は、ファイルを、少しずつ送る答えです。添付ファイルのダウンロードに使います。	06 前編 8. React の画面が呼ぶ API
`.FILES`	Django の `request.FILES` は、送られたファイルの入れ物です。`request.FILES.get("file")` で 1 つ取り出します。	06 前編 8. React の画面が呼ぶ API
`<int:名前>`	Django の URL の `"api/requests/<int:pk>/"` の `<int:pk>` は、「ここに来た数を、`pk` という名前で、ビューの関数に渡す」書き方です。`int` は数だけ受け付けます（`str`・`slug`・`uuid` もあります）。	06 前編 8. React の画面が呼ぶ API
`.atomic()`	`with transaction.atomic():` の中のデータベースの変更を、「全部成功か、全部取り消し」にする書き方。	06 前編 9. 社員の CSV の取り込み
`.update_or_create()`	条件に合う行があれば書きかえ、無ければ作るメソッド。`(物, 新しく作ったか)` の組が返ります。	06 前編 9. 社員の CSV の取り込み
`validate_email()`	メールアドレスの形として正しいかを調べ、正しくなければ `ValidationError` を起こす関数。	06 前編 9. 社員の CSV の取り込み
`BaseCommand`	Django の `BaseCommand` は、`python manage.py 名前` で動かす、自分のコマンドを作る時の元になる型です。`handle(self, …)` に処理を書きます。	06 前編 9. 社員の CSV の取り込み
`CommandError`	Django の `CommandError` は、自分のコマンドの中で、失敗を知らせるエラーです。	06 前編 9. 社員の CSV の取り込み
`.stderr`	`self.stderr.write(…)` は、エラーの文を出します。	06 前編 9. 社員の CSV の取り込み
`.stdout`	Django のコマンド（`BaseCommand`）の `self.stdout.write(…)` は、画面に文を出します（`print` の代わり）。	06 前編 9. 社員の CSV の取り込み
`.localdate()`	`timezone.localdate()` は、今日の日付を、設定した時間帯（日本時間）で返す関数です。	06 前編 10. 毎朝 8:00 の通知

コード	意味と使い方	初めて出てくる所
`.assertRaisesMessage()`	テストで、「この処理は、このエラー（文）を起こすはず」を確かめるメソッド。`with self.assertRaisesMessage(エラーの型, "文"):` の中で処理を呼びます。	06 前編 11. 自動のテスト
`method_decorator()`	関数用の印（デコレーター）を、クラスの画面（ビュー）のメソッドにも付けられるようにする物。	06 後編 3. サーバーとの窓口
`@method_decorator`	`@method_decorator(ensure_csrf_cookie, name="dispatch")` をクラスの上に付けて、そのクラスの画面に印を付けます。	06 後編 3. サーバーとの窓口
`@ensure_csrf_cookie`	その画面を返す時に、CSRF の札（クッキー）を必ず付ける印。React の画面が、あとで送るフォームに札を付けられるようにします（10 巻の 5 章）。	06 後編 3. サーバーとの窓口
`HttpResponse`	Django の `HttpResponse("文字")` は、文字（HTML など）をそのまま返す、いちばん基本の答えです。	06 後編 3. サーバーとの窓口
`JsonResponse`	Django の `JsonResponse({"status": "ok"})` は、辞書を JSON にして返す答えです。	06 後編 3. サーバーとの窓口
`TemplateView`	Django の `TemplateView` は、決まったテンプレート（HTML）を表示するだけのビューです。`template_name = "index.html"` を書くだけで使えます。	06 後編 3. サーバーとの窓口
`.as_view()`	クラスで作った画面（ビュー）を、URL の決まりに登録できる形にするメソッド。	06 後編 3. サーバーとの窓口
`re_path()`	正規表現で URL の決まりを書く関数。`re_path(r"^(?!api/|admin/).*$", …)` は「`api/`・`admin/` で始まらない URL は全部」（React の画面を返す）。	06 後編 3. サーバーとの窓口
`.bulk_create()`	いくつもの行を、1 回の SQL でまとめて作るメソッド。	11 B. サンプル集
`.values_list()`	問い合わせの結果を、タプル（値の組）の並びで返すメソッド。`flat=True` を付けると、1 つの列の値だけの並びになります。	11 B. サンプル集
`.aggregate()`	問い合わせの全体を、1 つの値（最小・最大・合計など）にまとめるメソッド。`aggregate(last=Max("due_date"))`。	11 B. サンプル集
`.get_or_create()`	条件に合う行があれば取り出し、無ければ作るメソッド。`(物, 作ったか)` の組が返ります。	11 B. サンプル集
`F`	Django の `F("amount")` は、ORM の中で「その行の、その列の値」を表します。`update(round=F("round") + 1)` は、データベースの中で 1 足すので、同時に 2 人が直してもずれません。	11 B. サンプル集

コード	意味と使い方	初めて出てくる所
`Max`	Django の `Max("amount")` は、ORM の集計で、いちばん大きい値を求めます。	11 B. サンプル集
`Min`	Django の `Min("amount")` は、いちばん小さい値を求めます。	11 B. サンプル集

Django REST framework

コード	意味と使い方	初めて出てくる所
`DefaultRouter`	DRF の `DefaultRouter` は、ViewSet（一覧・詳細・作成・変更・削除をまとめた物）から、URL を自動で作る係です。	05 7. HTML の画面
`.ValidationError()`	Django REST framework で、入力のまちがいを知らせるエラー。`raise serializers.ValidationError("件名を入れてください")` と書くと、その文が `400` の答えに入って返ります。	05 8. JSON の API
`.force_authenticate()`	テストで、決めたユーザーとしてログインしたことにして API を呼ぶためのメソッド（DRF のテスト用のクライアント）。	05 9. 自動のテスト
`APITestCase`	DRF の `APITestCase` は、API のテストを書くための元の型です。`self.client.get(…)` で要求を送り、答えを確かめます。	05 9. 自動のテスト
`.is_valid()`	送られてきたデータが、シリアライザー（入力の確かめ係）の決まりに合うかを調べるメソッド。`raise_exception=True` で、合わない時は 400 の答えにします。	06 前編 8. React の画面が呼ぶ API
`api_view()`	関数を、DRF の API の画面にする物。`@api_view(["GET", "POST"])` のように使います。	06 前編 8. React の画面が呼ぶ API
`parser_classes()`	API が受け付ける送り方（JSON・ファイル付きのフォームなど）を決める物。	06 前編 8. React の画面が呼ぶ API
`@api_view`	`@api_view(["GET", "POST"])` を関数の上に付けて、API にします。決めたメソッド以外は 405 で断られます。	06 前編 8. React の画面が呼ぶ API
`@parser_classes`	`@parser_classes([MultiPartParser])` を付けて、ファイル付きのフォーム（multipart）を受け付けるようにします。	06 前編 8. React の画面が呼ぶ API
`PermissionDenied`	DRF の `PermissionDenied` は、「権限が無い」というエラーです。起こすと、403 の答えになります。	06 前編 8. React の画面が呼ぶ API
`MultiPartParser`	DRF の `MultiPartParser` は、ファイルの付いたフォーム（`multipart/form-data`）を読む係です。	06 前編 8. React の画面が呼ぶ API

コード	意味と使い方	初めて出てくる所
`Response`	DRF の `Response(data)` は、API の答えです。辞書や並びを、JSON にして返します。	06 前編 8. React の画面が呼ぶ API
`.query_params`	DRF の `request.query_params` は、URL の `?mine=1` の部分の値です。	06 前編 8. React の画面が呼ぶ API
`.data`	DRF の `serializer.data` は、モデルの値を、JSON にできる形（辞書）にした物です。`request.data` は、届いた中身（JSON やフォーム）です。	06 前編 8. React の画面が呼ぶ API
`.validated_data`	DRF の `serializer.validated_data` は、確かめ（`is_valid()`）を通った後の、安全な値です。	06 前編 8. React の画面が呼ぶ API

練習アプリで作る物（Python）

コード	意味と使い方	初めて出てくる所
`token_in()`	テストの中で作る小さな関数。送られたメールの本文から、承認の URL の文字列（トークン）を取り出します。	06 前編 11. 自動のテスト

画面に出す

コード	意味と使い方	初めて出てくる所
`{}`	`println!("{}", 値)` の `{}` は、「ここに値を入れる」場所の印です。後ろに `,` で並べた値が、左から順に入ります（`println!("{}と{}", 1, 2)` → `1と2`）。`{}` は、人が読む形（Display）で表示します。	07 2. 依頼の状態を enum で書く
`{:?}`	`{:?}` は、値を「プログラマー向けの形」（Debug）で表示する指定です。`Some("main.rs")` のように、`Option` の `Some` や、文字列の `"` まで、そのまま見せます。`{}` では表示できない型（`Vec`・`Option` など）も、`{:?}` なら表示できます。`{path:?}` のように名前と一緒にも書けます。自分で作った構造体は、`#[derive(Debug)]` を付けると使えます。	07 2. 依頼の状態を enum で書く
`println!`	かっこの中の文字を画面に出して、最後に改行するマクロ。`println!("Hello, world!");` は `Hello, world!` と表示します。`{}` を書くと、後ろに並べた値がそこに入ります（例: `println!("{}点", 80);` → `80点`）。名前の最後の `!` は「マクロ」（コードを作り出す特別な仕組み）の印で、ふつうの関数と見分けるために付いています。	07 4. 所有権と借用

コード	意味と使い方	初めて出てくる所
`{名前}`	`println!` や `format!` の文字の中に `{os}` のように `{ }` で変数の名前を書くと、その変数の中身がそこに入ります（`let os = "windows"; println!("OS: {os}");` → `OS: windows`）。`println!("OS: {}", os)` と書くのと同じ結果です。`{ }` の中に書けるのは名前だけで、`{a + 1}` や `{p.x}` のような式は書けません（その時は `{}` と、後ろに並べる値を使います）。`{name:<22}` のように、名前の後ろに `:` と書式を付けることもできます。	07 4. 所有権と借用
`{:0幅}`	`{:02}` のように、幅の前に `0` を付けると、「この幅になるまで、前を 0 で埋める」指定になります。`{:02}` で `7` は `07`、`{:06}` で `42` は `000042`。`{:016x}` は「16 けたになるまで 0 で埋めた 16 進数」です。時刻（`09:05`）や番号（`000042`）の表示に使います。	08 前編 6. 状態を変える処理
`eprintln!`	`println!` のエラー版。「標準エラー出力」というエラー用の出口に書きます。画面ではふつうの出力と同じに見えますが、`> ファイル` で結果をファイルに書き出した時も、エラーは画面に残ります。失敗の知らせを出す時に使います。	08 前編 6. 状態を変える処理
`{:#}`	`{:#}` は、人が読む形（`{}`）の「もう 1 つの形」で表示する指定です。`anyhow` のエラーを `{:#}` で出すと、「書き足した説明: 元のエラー」のように、原因までを 1 行につないで表示します（`{}` だと、いちばん外側の説明だけです）。	08 前編 11. ここまでの部品を 1 つにまとめる
`{:.桁}`	`{:.1}` の `.` と数は、「小数点の下を何けた出すか」の指定です。`{:.1}` で `3.14159` は `3.1`、`{:.2}` で `3.14`、`{:.0}` なら `3`。けたを切りそろえて（丸めて）表示します。`{:.2?}` のように Debug（`?`）と組み合わせると、かかった時間（`Duration`）を `1.23ms` のように短く出せます。	08 後編 2. 画面の部品と、XSS を防ぐ仕組み
`{:X}`	`{:X}` は、整数を大文字の 16 進数（0〜9 と A〜F）で表示する指定です。`255` は `FF`、`'A' as u32`（65）は `41`。文字の番号や色の値を見る時に使います。	08 後編 8. 添付ファイル

変数

コード	意味と使い方	初めて出てくる所
`static`	プログラムが動いている間、ずっとある値。`&'static str` は「プログラムの中に書いた、ずっと消えない文字列」を借りている型です（`"回覧中"` のような文字列はこの型）。	07 2. 依頼の状態を enum で書く

コード	意味と使い方	初めて出てくる所
`mut`	「あとで変えてよい」という印。`let mut total = 0;` と書くと、`total += 10;` のように値を変えられます。付けないと変えられません（変えようとすると、コンパイルのエラーで教えてくれます）。	07 2. 依頼の状態を enum で書く
`let`	値に名前を付けて覚えておく（変数を作る）書き方。`let age = 20;` で、`age` という名前で 20 を覚えます。最初は「変えられない」変数です。あとで変えたい時は `let mut age = 20;` のように `mut` を付けます。	07 2. 依頼の状態を enum で書く
`const`	ずっと変わらない値（定数）に名前を付ける書き方。`const MAX_TITLE_CHARS: usize = 80;`。名前は大文字と `_` で書き、型（`usize` など）を必ず書きます。プログラムのどこからでも使えます。	08 前編 4. 表
`let _ =`	`let _ = fs::remove_file("count.txt");` の `let _ =` は、「結果は受け取るが、使わずに捨てる」という書き方です。`Result` を返す関数の結果を使わないと「使っていない」と警告が出るので、わざと捨てる時に書きます。ここでは「ファイルが無くて失敗しても、気にしない」という意味です。	08 前編 8. 社員の CSV と、管理のコマンド

数

コード	意味と使い方	初めて出てくる所
`u32`	0 以上の整数の型（32 ビット。0〜約 42 億）。数を数える・番号を付ける時に使います。	07 2. 依頼の状態を enum で書く
`.max()`	2 つの数のうち、大きい方を返すメソッド。`page.max(1)` は「`page` と 1 の大きい方」なので、0 やマイナスが来ても 1 になります（下限を決める時の書き方）。イテレーターに使うと、いちばん大きい物を `Option` で返します。	07 5. Topcoat の最小のアプリ
`u16`	0〜65535 の整数の型（16 ビット）。ポート番号（`8080` など）はこの範囲に収まるので、`u16` で表します。	08 前編 3. 設定を読む
`i32`	マイナスもある整数の型（32 ビット。約 −21 億〜21 億）。`i` は integer（整数）の頭文字です。型を書かない整数は、ふつう `i32` になります。	08 前編 4. 表
`i64`	マイナスもある整数の型（64 ビット。約 ±922 京）。データベースの整数の列や、番号（id）によく使います。	08 前編 4. 表

コード	意味と使い方	初めて出てくる所
`i16`	マイナスもある整数の型（16 ビット。−32768〜32767）。小さな数で足りる項目（何回目か、など）に使います。PostgreSQL の `SMALLINT` の列と合います。	08 前編 4. 表
`as`	型を変える書き方。`x as f64` は整数を小数に、`300 as u8` は 300 を u8 に変えます。入りきらない分は**黙って切り捨てられる**ので（300 → 44）、範囲が分からない時は `try_from` を使います。	08 前編 6. 状態を変える処理
`u8::from()`	ほかの型の値を `u8`（0〜255 の数）に変える関数。`u8::from(now.month())` は、`Month::November` を 11 にします（必ず入りきる変換なので、`try_from` ではなく `from`）。	08 前編 6. 状態を変える処理
`usize`	0 以上の整数の型で、大きさはパソコンに合わせて決まります（64 ビットのパソコンなら `u64` と同じ）。並びの長さ（`.len()`）や、何番目か（`v[i]` の `i`）は、いつも `usize` です。	08 前編 6. 状態を変える処理
`u8`	0〜255 の整数の型（8 ビット＝1 バイト）。`u` は unsigned（マイナスが無い）の意味です。ファイルや通信の中身（バイトの並び `Vec<u8>`）は、この型の並びです。	08 前編 6. 状態を変える処理
`10u8`	数のすぐ後ろに型の名前を付けると、その型の数になります。`10u8` は `u8` 型の 10、`0u64` は `u64` 型の 0 です。型を書かないと、整数は `i32` と決まります（使い方から別の型に決まることもあります）。	08 前編 6. 状態を変える処理
`+=`	`total += n;` は `total = total + n;` を短く書いた形です。`total` は `let mut` で作った（変えられる）変数でないといけません。文字列（`String`）でも `s += "続き";` で書き足せます。	08 前編 8. 社員の CSV と、管理のコマンド
`.fract()`	小数の、小数点より下の部分を返すメソッド。`2.5_f64.fract()` は `0.5`。`v.fract() == 0.0` で「整数ちょうどか」を確かめています。	08 後編 2. 画面の部品と、XSS を防ぐ仕組み
`f64`	小数を入れる型（64 ビット）。`3.14` のような数です。小数は、`0.1 + 0.2` が `0.30000000000000004` になるような、わずかな誤差が出るので、お金の計算には使いません（整数の「円」で数えます）。	08 後編 2. 画面の部品と、XSS を防ぐ仕組み
`1.5`	`1.08` のように `.` の付いた数は小数で、型を書かなければ `f64` になります。整数（`i32` など）と小数は、そのままでは足したり掛けたりできないので、`as f64` などで型をそろえます。	08 後編 2. 画面の部品と、XSS を防ぐ仕組み

文字

コード	意味と使い方	初めて出てくる所
`'a'`	`'='` のように 1 文字を `'`（シングルクォート）で囲むと、`char`（1 文字）の値になります。何文字でも入る文字列（`"…"`）とは別の物です。`split_once('=')` は「最初の `=` で 2 つに分ける」。	08 前編 3. 設定を読む
`.is_ascii_digit()`	その文字（バイト）が、半角の数字（`0` 〜 `9`）かを `true` / `false` で返すメソッド。	08 前編 8. 社員の CSV と、管理のコマンド
`char`	1 文字を入れる型です。`'a'`・`'あ'`・`'😀'` のように `'` で囲んで書きます。日本語の文字も絵文字も、1 つの `char` です（大きさはいつも 4 バイト）。	08 後編 5. 入力フォーム
`.is_control()`	その文字が、制御文字（改行・タブなど、目に見えない特別な文字）かを返すメソッド。ファイルの名前から取り除くのに使っています。	08 後編 8. 添付ファイル
`.is_ascii_alphanumeric()`	その文字が、ASCII（半角）の英字か数字（`A` 〜 `Z`・`a` 〜 `z`・`0` 〜 `9`）なら `true` を返すメソッド。`'あ'` は `false` です。	08 後編 8. 添付ファイル
`.is_ascii_hexdigit()`	その文字が 16 進数の数字（`0` 〜 `9`・`a` 〜 `f`・`A` 〜 `F`）なら `true` を返すメソッド。	10 13. テスト
`.is_ascii_lowercase()`	その文字が、半角の英小文字なら `true` を返すメソッド。	10 13. テスト

文字列

コード	意味と使い方	初めて出てくる所
`write!`	`println!` と同じ書き方で、画面ではなく、決めた書き込み先に書くマクロ。`Display` を実装する時の `write!(f, "…")` は、「表示する文を `f` に書く」という意味です。	07 2. 依頼の状態を enum で書く
`.to_string()`	値を、自分の物の文字列（`String`）に変えるメソッド。`"abc".to_string()` や `42.to_string()` のように使います。	07 2. 依頼の状態を enum で書く
`String`	自分の物として持つ、変えられる文字列の型。後ろに足す（`push_str`）・書きかえる、ができます。`"…"` と書いた文字列は `&str`（借りている読むだけの文字列）で、関数の引数は `&str` にしておくと、どちらも渡せます。	07 2. 依頼の状態を enum で書く
`.as_str()`	`String`（持ち主のいる文字列）を、借りた `&str` として見せるメソッド。`&str` を受け取る関数に渡す時などに使います。	07 4. 所有権と借用

コード	意味と使い方	初めて出てくる所
`String::from()`	`"…"`（`&str`）から、自分の物の文字列（`String`）を作る関数。`String::from("こんにちは")`。`"こんにちは".to_string()` と同じです。	07 4. 所有権と借用
`format!`	`println!` と同じ書き方で、画面に出さずに文字列（`String`）を作るマクロ。`let msg = format!("{}さん", name);` のように、値を埋め込んだ文を作る時に使います。	07 5. Topcoat の最小のアプリ
`.trim()`	文字列の前後の空白や改行を取り除いたものを返すメソッド。入力された文字の、うっかり入った空白を取る時によく使います。	07 5. Topcoat の最小のアプリ
`.is_empty()`	空（長さが 0）なら `true` を返すメソッド。文字列にも `Vec` にも使えます。`title.trim().is_empty()` は「空白を取ったら何も残らないか」を調べます。	07 5. Topcoat の最小のアプリ
`.as_deref()`	`Option<String>` を、中身を借りて `Option<&str>` に変えるメソッド。`env::args().nth(1).as_deref() == Some("reset")` のように、「引数が `reset` かどうか」を文字列と比べる時に使います。	08 前編 3. 設定を読む
`.parse()`	文字列を、数などの別の型に変えるメソッド。`"42".parse::<i32>()` や、`let n: i32 = "42".parse().unwrap();` のように、何に変えるかを型で伝えます。変えられない文字（`"abc"` など）もあるので、結果は `Result` です。	08 前編 3. 設定を読む
`.replace()`	文字列の中の、ある部分を全部置きかえた、新しい文字列を返すメソッド。`"a-b".replace('-', "/")` は `"a/b"`。元の文字列は変わりません。	08 前編 4. 表
`.as_bytes()`	文字列を、中のバイト（コンピューターが持っている 0〜255 の数の並び）として見るメソッド。ネットやファイルに書く関数がバイトを求める時に使います。	08 前編 5. ログインとセッション
`.contains()`	含まれていれば `true` を返すメソッド。文字列なら `"report.csv".contains("2026")`、範囲なら `(1..=80).contains(&length)`（1〜80 に入っているか）、`Vec` なら `v.contains(&x)` です。	08 前編 6. 状態を変える処理
`hex::encode()`	バイトの並びを、`0`〜`9`・`a`〜`f` の文字（16 進数）に変える関数。1 バイトが 2 文字になります。	08 前編 6. 状態を変える処理
`\`	文字列の中で、行の終わりに `\` を書くと、「次の行へ続く」という意味になります。改行と、次の行の頭の空白は、文字列に入りません。長い文字列を、見やすく何行かに分けて書く時に使います。	08 前編 6. 状態を変える処理

コード	意味と使い方	初めて出てくる所
`.starts_with()`	その文字（文字列）で始まっていれば `true` を返すメソッド。`"report.csv".starts_with("report")` は `true`。終わりを調べるのは `.ends_with()`、含むかは `.contains()` です。	08 前編 8. 社員の CSV と、管理のコマンド
`.push_str()`	`String` の後ろに文字列を足すメソッド。`s.push_str("さん");`。`s` は書きかえるので、`let mut s` のように `mut` が要ります。	08 前編 8. 社員の CSV と、管理のコマンド
`.trim_end_matches()`	文字列の後ろから、決めた文字（文字列）をくり返し取り除いたものを返すメソッド。`name.trim_end_matches(".rs")` は、`.rs` を取ります。	08 前編 8. 社員の CSV と、管理のコマンド
`.chars()`	文字列を、1 文字ずつ（`char`）取り出せる形にするメソッド。`s.chars().count()` で文字の数、`s.chars().take(60).collect::<String>()` で先頭の 60 文字。日本語も 1 文字ずつ正しく扱えます（`len()` はバイト数）。	08 前編 8. 社員の CSV と、管理のコマンド
`String::new()`	空の `String` を作る関数。あとで中身を足していく時（`read_line(&mut s)` で 1 行読み込む など）に使います。	08 前編 8. 社員の CSV と、管理のコマンド
`.strip_prefix()`	文字列が、決めた文字で始まっていれば、それを取り除いた残りを `Some(残り)` で返すメソッド。始まっていなければ `None` です。	08 前編 8. 社員の CSV と、管理のコマンド
`.bytes()`	文字列を、1 バイトずつ取り出せる形にするメソッド。`s.bytes().all(|b| b.is_ascii_digit())` で「全部、半角の数字か」を調べます。	08 前編 8. 社員の CSV と、管理のコマンド
`.split_once()`	文字列を、最初に見つかった区切りの所で 2 つに分けるメソッド。`"title=買い物".split_once('=')` は `Some(("title", "買い物"))`。区切りが無い時は `None` です。	08 前編 8. 社員の CSV と、管理のコマンド
`.ends_with()`	その文字（文字列）で終わっていれば `true` を返すメソッド。`name.ends_with(".db")`。	08 前編 8. 社員の CSV と、管理のコマンド
`b"…"`	`b"…"` は、文字列ではなく「バイトの並び」（`&[u8; N]`）を作る書き方です。中には英数字と記号（ASCII）だけを書けます。通信やファイルの中身を、バイトのまま扱う時に使います。	08 前編 8. 社員の CSV と、管理のコマンド
`.split()`	文字列を、区切り文字のたびに分けるメソッド。`"a,b,c".split(',')` は `a`・`b`・`c` の 3 つ。結果は 1 つずつ取り出せる形（イテレーター）なので、`Vec` にしたい時は `.collect()` を付けます。	08 前編 13. 自動のテスト

コード	意味と使い方	初めて出てくる所
`.lines()`	文字列を、1 行ずつに分けて取り出せる形にするメソッド。行の終わりの改行（`\n` や `\r\n`）は含みません。`text.lines().count()` で行の数が分かります。	08 前編 13. 自動のテスト
`r#"…"#`	`r#"…"#` も「生の文字列」で、`"#` が来るまでを、`\` も `"` もそのまま文字にします。`r#"{"title": "牛乳"}"#` のように、`"` の多い JSON や、何行もの SQL を書く時に便利です。	08 後編 6. 図面
`.rsplit()`	文字列を、後ろから順に区切り文字で分けるメソッド。`line.rsplit(' ').next()` は、最後の単語です。	08 後編 8. 添付ファイル
`.rsplit_once()`	文字列を、後ろから探して最初に見つかった区切りで、2 つに分けるメソッド。`"a.b.pdf".rsplit_once('.')` は `Some(("a.b", "pdf"))`。拡張子を取り出すのに使います。	08 後編 8. 添付ファイル
`.to_ascii_lowercase()`	半角の英字を、全部小文字にした文字列を返すメソッド。`"PDF"` は `"pdf"`。	08 後編 8. 添付ファイル
`r"…"`	`r"…"` は「生の文字列」で、中の `\` を特別な意味にしません。`r"C:\Users\you"` や、正規表現の `r"\d+"` のように `\` の多い文字を、そのまま書けます。中に `"` を入れたい時は `r#"…"#` を使います。	08 後編 8. 添付ファイル
`String::from_utf8()`	バイトの並び（`Vec<u8>`）を、文字列（`String`）に変える関数。UTF-8 の文字として読めない時は `Err` を返します。	10 13. テスト

Vec（並び）

コード	意味と使い方	初めて出てくる所
`Vec::new()`	空の `Vec` を作る関数。あとで `push` で足すので、`let mut v = Vec::new();` のように `mut` を付けます。中身の型は、足す物から Rust が推し量ります。	07 2. 依頼の状態を enum で書く
`Vec`	同じ種類の値を、順番に並べて持つ入れ物（ベクタ）。`Vec<i32>` は「整数の並び」、`Vec<String>` は「文字列の並び」です。`vec![…]` か `Vec::new()` で作り、`push` で足し、`len()` で数を数え、`v[0]` や `v.get(0)` で取り出します。	07 2. 依頼の状態を enum で書く

コード	意味と使い方	初めて出てくる所
`vec!`	`Vec`（ベクタ：同じ種類の値を、順番に並べて持つ入れ物）を、中身を書いて作るマクロ。`vec![80, 95, 70]` は、80・95・70 の 3 つが入った並びです。`scores[0]` で最初の 80 を取り出せます（番号は 0 から数えるので、3 つなら使える番号は 0・1・2）。範囲の外の番号（`scores[10]` など）を使うと、プログラムは止まります（パニック）。あとから `push` で足したり、`len()` で数を数えたりできます。	07 4. 所有権と借用
`.push()`	後ろに 1 つ足すメソッド。`Vec` なら `v.push(10);` で 10 を足し、`String` なら `s.push('!');` で 1 文字足します。`mut` の付いた変数に使います。	07 4. 所有権と借用
`&[T]`	`&[User]` のような `&[型]` は、「並び（`Vec` や配列）の全部か一部を、借りて見る」型です（スライス）。関数の引数を `&[User]` にすると、`Vec<User>` も配列も渡せます。	07 4. 所有権と借用
`.insert()`	`Vec` の、決めた位置に入れるメソッド。`v.insert(0, x)` は先頭に入れます（後ろの物は 1 つずつずれる）。`HashMap` の `.insert(キー, 値)` は「入れる（上書き）」です。	07 5. Topcoat の最小のアプリ
`[値; 個数]`	`[0; 5]` は「0 を 5 個並べた配列」（`[0, 0, 0, 0, 0]`）を作る書き方です。型の所に書く `[u8; 32]` は、「`u8` が 32 個の配列」という型です。	08 前編 4. 表
`[a, b]`	`[ "Cargo.toml", "src/main.rs" ]` のように `[ ]` の中に `,` で区切って値を並べると、「配列」（決まった数の値の並び）ができます。`for x in 配列 { … }` で 1 つずつ取り出せます。あとから数を増やしたい時は、`vec![…]` で作る `Vec` を使います。	08 前編 4. 表
`.len()`	中身の数（長さ）を返すメソッド。`Vec` なら入っている個数、`String`・`&str` ならバイト数です。日本語は 1 文字が 3 バイトなので、文字の数を数える時は `.chars().count()` を使います。	08 前編 6. 状態を変える処理
`.to_vec()`	並び（スライス）の中身を写して、新しい `Vec` を作るメソッド。	08 前編 6. 状態を変える処理
`.sort()`	`Vec` の中身を、小さい順（文字なら辞書の順）に並べかえるメソッド。新しい `Vec` は作らず、その `Vec` 自身を並べかえるので、`mut` が要ります。	08 前編 6. 状態を変える処理
`.dedup()`	`Vec` の中で、隣どうしで同じ値が続く所を 1 つにまとめるメソッド。先に `sort` しておくと、全部の重複が消えます。	08 前編 6. 状態を変える処理
`.first()`	並びの最初の 1 つを `Option` で返すメソッド（空なら `None`）。最後の 1 つは `.last()` です。	08 前編 8. 社員の CSV と、管理のコマンド

コード	意味と使い方	初めて出てくる所
`.get()`	番号やキーで中身を取り出すメソッド。`v.get(10)` は、範囲の外なら止まらずに `None` を返します（`v[10]` はプログラムが止まる）。文字列の `s.get(0..3)` は、文字の区切りでない位置なら `None` です。`HashMap` の `map.get("キー")` も、無ければ `None` です。	08 前編 8. 社員の CSV と、管理のコマンド
`.last()`	並びの最後の 1 つを `Option` で返すメソッド（空なら `None`）。最初の 1 つは `.first()`。	08 前編 13. 自動のテスト
`.extend()`	`Vec` の後ろに、別の並びの中身をまとめて足すメソッド。`v.extend(extra);`。	08 後編 5. 入力フォーム
`.sort_by_key()`	「何で並べるか」を関数で決めて、`Vec` を並べかえるメソッド。`notes.sort_by_key(|note| note.id)` は、番号の小さい順にします。	08 後編 9. 集計と、動いているかの確かめ

HashMap（表）

コード	意味と使い方	初めて出てくる所
`std::collections::HashSet::new()`	空の `HashSet`（同じ物を 2 回入れない入れ物）を作る関数。`.insert(値)` は、初めてなら `true`、もうあれば `false` を返すので、「同じ社員番号が 2 回出てきた」ことを見つけられます。	08 前編 8. 社員の CSV と、管理のコマンド
`HashSet`	同じ値は 1 つしか入らない集まり（値だけの `HashMap`）。重複を消す・「もう見たか」を覚える時に使います。`set.insert(x)` は、新しく入った時だけ `true` を返します。	08 前編 8. 社員の CSV と、管理のコマンド

範囲

コード	意味と使い方	初めて出てくる所
`a..b`	`0..5` は「0 から 5 の手前まで」（0, 1, 2, 3, 4）の範囲です。`for i in 0..5` で 5 回くり返します。5 も含めたい時は `0..=5` と書きます。`&s[0..3]` のように、文字列や並びの一部を取り出す時にも使い、`..3`（最初から）・`2..`（最後まで）と片方を省けます。	08 前編 8. 社員の CSV と、管理のコマンド

条件

コード	意味と使い方	初めて出てくる所
`if`	条件によって処理を分ける書き方。`if 条件 { 合う時 } else { 合わない時 }`。条件は `true`・`false` になる式（`age >= 20` など）で、`( )` で囲む必要はありませんが、`{ }` は必ず要ります。	07 4. 所有権と借用
`if let`	`if let Some(x) = 値 { … }` は、「値が `Some` の形なら、中身を `x` に入れて `{ }` の中を動かす」という書き方です。`match` で 1 つの形の時だけ書きたい時の、短い形です。`else { … }` も付けられ、`if let Ok(n) = s.parse::<i32>()` のように `Result` にも使えます。	07 4. 所有権と借用
`==`	`a == b` は「a と b が等しいか」を調べ、`true` か `false` になります。`=`（1 つ）は「値を入れる」なので、比べる時は必ず 2 つ書きます。	07 4. 所有権と借用
`!`	`!valid` のように値の前に付けた `!` は、「〜ではない」（true と false を入れかえる）です。`if !path.exists()` は「無かったら」。`println!` の `!` はマクロの印で、これとは別物です。	07 5. Topcoat の最小のアプリ
`bool`	`true`（はい）か `false`（いいえ）の、どちらかだけが入る型です。`if` の条件や、`5 > 3` のように比べた結果は `bool` です。	08 前編 3. 設定を読む
`&&`	`a && b` は「a と b の両方が true なら true」（かつ）です。左が false なら、右は調べません。	08 前編 3. 設定を読む
`else`	`if` の条件が合わなかった時の処理を書く所。`else if 条件 { … }` で、条件をいくつも続けられます。	08 前編 4. 表
`||`	`a || b` は「a か b の、どちらかが true なら true」（または）です。`if n == 0 || line == "\r\n"` は「0 か、空の行なら」。左が true なら、右は調べません。	08 前編 5. ログインとセッション
`!=`	`a != b` は「a と b が等しくないか」を調べます。等しくなければ `true` です。	08 前編 6. 状態を変える処理

くり返し

コード	意味と使い方	初めて出てくる所
`for`	並びや範囲から 1 つずつ取り出して、くり返す書き方。`for n in 1..=100 { … }` は、`n` に 1, 2, …, 100 を順に入れて、中を 100 回動かします。`for x in &v` で、`Vec` の中身を借りて回せます。	07 2. 依頼の状態を enum で書く

コード	意味と使い方	初めて出てくる所
`in`	`for 名前 in 並び` の一部で、「この並びの中から」という意味です。	07 4. 所有権と借用
`while`	条件が `true` の間、くり返す書き方。`while 条件 { … }`。条件が最初から `false` なら、1 回も動きません。	08 前編 8. 社員の CSV と、管理のコマンド
`while let`	`while let Some(c) = cause { … }` は、「`Some` の形である間、中身を `c` に入れてくり返す」書き方です。`None` になったら止まります。`while let Some(x) = stack.pop()` のように、並びから 1 つずつ取り出す時によく使います。	08 前編 8. 社員の CSV と、管理のコマンド
`continue`	くり返しの「今回の残り」を飛ばして、次の回へ進む命令。`for` や `while` の中で、条件に合わない物を飛ばす時に使います。	08 前編 8. 社員の CSV と、管理のコマンド
`loop`	終わりのないくり返し。`loop { … }` の中で `break` を書いた所で抜けます。サーバーが「次の接続を待つ」のをずっと続ける時などに使います。	08 前編 9. 毎朝 8:00 の通知

場合分け

コード	意味と使い方	初めて出てくる所
`match`	値の形に合わせて、処理を分ける書き方。`match 値 { 形1 => 処理1, 形2 => 処理2 }` と書き、上から順に合う物を探します。全部の場合を書かないとコンパイルが通らないので、書き忘れを防げます。`_ =>` は「それ以外全部」です。	07 2. 依頼の状態を enum で書く
`=>`	`match` の中で「この形なら、→ この処理」をつなぐ矢印。`Some(name) => println!("{name}"),` は「値があったら、その名前を表示」という意味です。	07 2. 依頼の状態を enum で書く
`_ =>`	`match` の最後の `_ =>` は、「ここまでのどれにも当てはまらない、その他すべて」を受け止める腕です。`_` は「何でもよい（名前を付けない）」という印です。`match` は、ありうる値を全部受け止めないとエラーになるので、最後によく書きます。	07 2. 依頼の状態を enum で書く
`A | B =>`	`match` の腕の `A | B =>` は、「A か B のどちらかの形なら」という意味で、同じ処理をまとめて書けます。`1 | 2 | 3 => "小さい"` のように使います。	07 2. 依頼の状態を enum で書く
`_`	`(0, _)` や `Err(_)` の `_` は、「ここの値は何でもよい・使わない」という印です。名前を付けないので、「使っていない」という警告も出ません。	08 前編 3. 設定を読む

コード	意味と使い方	初めて出てくる所
`パターン if 条件 =>`	`match` の腕に `None if scheme == "https" =>` のように `if 条件` を付けると、「形が合って、しかも条件も合う時だけ」その腕を選びます（マッチガード）。条件に合わなければ、次の腕を調べます。	08 前編 3. 設定を読む
`matches!`	値が、ある形（パターン）に合うかを `true`・`false` で返すマクロ。`matches!(filter, Filter::Active | Filter::Done)` のように、`match` を 1 行で書けます。	08 前編 4. 表
`..`	`Point { x, .. }` や `(first, ..)` の `..` は、「残りの項目は何でもよい（見ない）」という印です。項目の多い構造体から、使う物だけを取り出す時に書きます。	08 前編 5. ログインとセッション

Option

コード	意味と使い方	初めて出てくる所
`.is_some()`	`Option` が `Some`（値がある）なら `true` を返すメソッド。値そのものは要らず、あるかどうかだけ知りたい時に使います。	07 4. 所有権と借用
`Option`	「値があるかもしれないし、無いかもしれない」を表す型。中身は `Some(値)` か `None`。`Option<u32>` は「整数があるかもしれない」。取り出す時は `match`・`if let`・`unwrap_or` などを使います。	07 4. 所有権と借用
`Some`	`Option`（値が「ある」か「無い」か）の、「ある」の方。`Some(値)` の形で中身を包みます。`match` で `Some(name) => { … }` と書くと、中身を `name` という名前で取り出して使えます。「無い」の方は `None` です。	07 4. 所有権と借用
`.is_none()`	`Option` が `None`（値が無い）なら `true` を返すメソッド。	08 前編 3. 設定を読む
`None`	`Option` の「値が無い」側。`Some(値)`（値がある）か `None`（無い）のどちらかになります。Rust には「null」が無い代わりに、無いかもしれない値は必ず `Option` で表し、`match` などで両方の場合を書きます。	08 前編 3. 設定を読む
`let … else`	`let Some(x) = 値 else { return …; };` は、「`Some` の形なら中身を `x` に入れて先へ進み、そうでなければ `else` の中で抜ける」という書き方です。`else` の中は、必ず `return` や `continue` などで抜けます。思ったとおりでない時に早めに抜けて、残りの処理を平らに書けます。	08 前編 5. ログインとセッション

コード	意味と使い方	初めて出てくる所
`.is_some_and()`	`Option` に値があり、しかもその値が条件に合う時だけ `true` を返すメソッド。`cookie.is_some_and(|c| c == "admin=yes")`。	08 前編 6. 状態を変える処理
`.get_or_insert()`	`Option` が `None` の時だけ、渡した値を入れるメソッド（もう `Some` なら何もしない）。`first.get_or_insert(step_id)` は「最初の 1 つだけを覚えておく」書き方です。	08 前編 6. 状態を変える処理
`.cloned()`	借りている値を、`.clone()` で複製した値にするメソッド。`Option<&String>` を `Option<String>` に、イテレーターなら借りた物の並びを複製の並びにします。	08 前編 8. 社員の CSV と、管理のコマンド
`.then_some()`	`bool` が `true` なら `Some(値)`、`false` なら `None` を返すメソッド。`on_about.then_some("page")`。Topcoat の `view!` では、属性に `None` を入れると、その属性ごと書かれません。	08 後編 5. 入力フォーム

エラー処理

コード	意味と使い方	初めて出てくる所
`.unwrap()`	`Option` や `Result` から中身を取り出すメソッド。中身が無い（失敗した）時は、その場でプログラムが止まります（パニック）。練習や「絶対に失敗しない」と分かっている所だけで使い、本番のコードでは `?` や `unwrap_or` を使います。	07 2. 依頼の状態を enum で書く
`.is_err()`	`Result` が `Err`（失敗）なら `true` を返すメソッド。	07 2. 依頼の状態を enum で書く
`Result`	「うまくいった（`Ok(値)`）」か「失敗した（`Err(エラー)`）」かのどちらかを入れる型。失敗するかもしれない関数は、`Result` を返します。受け取った側は、`match` で両方の場合を書くか、`?`（失敗ならその場で呼び出し元へ返す）で受け取ります。`Result<Request, String>` は「うまくいけば `Request`、失敗なら説明の文字列」です。	07 2. 依頼の状態を enum で書く
`Error`	エラーを表す型が守る決まりごと（トレイト）。`use std::error::Error;` と書くと、`.source()`（元になったエラー）などを使えます。`Box<dyn Error>` は「どんなエラーでも入る箱」です。	07 2. 依頼の状態を enum で書く
`Ok`	`Result` の「うまくいった」の方。`Ok(値)` の形で結果を包みます。`Ok(())` は「うまくいった。返す値は特に無い（`()` は空っぽの値）」という意味で、`Result<()>` を返す関数の最後によく書きます。	07 2. 依頼の状態を enum で書く

コード	意味と使い方	初めて出てくる所
`Err`	`Result` の「失敗した」の方。`Err(エラー)` の形で、何が起きたかを包みます。`match` で `Err(e) => …` と書くと、エラーを `e` で受け取れます。`?` を付けると、`Err` の時はその場で呼び出し元へ返します。	07 2. 依頼の状態を enum で書く
`?`	`Result` や `Option` の後ろに付ける記号。「成功なら中身を取り出して先へ進む。失敗なら、その場で関数を終えて、エラーを呼び出し元へ返す」という意味です。`let text = fs::read_to_string("a.txt")?;` のように使います。`?` を使える関数は、戻り値が `Result`（または `Option`）の関数だけです。	07 2. 依頼の状態を enum で書く
`.unwrap_or()`	`Option` や `Result` から中身を取り出し、無い（失敗した）時は、代わりに決めた値を使うメソッド。`name.unwrap_or("名無し".to_string())` のように使います。止まらない安全な取り出し方です。	07 5. Topcoat の最小のアプリ
`.ok()`	`Result` を `Option` に変えるメソッド。`Ok(値)` は `Some(値)` に、`Err(…)` は `None` になります（エラーの中身は捨てる）。`"42".parse().ok()` のように、「失敗したら無かったことにする」時に使います。	08 前編 3. 設定を読む
`.ok_or_else()`	`Option` を `Result` に変えるメソッド。`None` の時に、渡した関数でエラーを作ります（必要な時だけ作る `ok_or` の版）。	08 前編 3. 設定を読む
`.unwrap_or_else()`	`Option` や `Result` から中身を取り出し、無い（失敗した）時は、渡した関数を動かして代わりの値を作るメソッド。`unwrap_or` と違い、代わりの値を作る処理が、必要な時だけ動きます。	08 前編 3. 設定を読む
`.map_err()`	`Result` のエラーの方だけを、別の物に変えるメソッド。`.map_err(|_| ConfigError::BadPort)` で、元のエラーを自分のエラーの型に変えます。成功の値はそのままです。	08 前編 3. 設定を読む
`.unwrap_or_default()`	`Option` や `Result` から中身を取り出し、無い（失敗した）時は、その型の「空の値」（文字列なら `""`、数なら 0）を使うメソッド。	08 前編 4. 表
`.expect()`	`Option` や `Result` から中身を取り出すメソッド。中身が無い（失敗した）時は、かっこの中の文を表示して、その場でプログラムが止まります（パニック）。`unwrap()` と同じですが、止まった理由が分かる文を付けられるので、`unwrap()` よりこちらがおすすめです。	08 前編 5. ログインとセッション
`.is_ok()`	`Result` が `Ok`（成功）なら `true` を返すメソッド。中身は要らず、成功したかだけ知りたい時に使います。	08 前編 5. ログインとセッション
`bail!`	`anyhow` のマクロ。その場でエラーを作って、関数から返します。`bail!("ノートが 1 件もありません");` は `return Err(anyhow!("…"));` の短い書き方です。	08 前編 8. 社員の CSV と、管理のコマンド

コード	意味と使い方	初めて出てくる所
`.with_context()`	`anyhow` のメソッド。失敗した時のエラーに、「何をしていて失敗したか」の説明を書き足します。`.with_context(|| format!("{path} を読めませんでした"))?`。ログを見た人が、原因をすぐ分かるようになります。	08 前編 8. 社員の CSV と、管理のコマンド
`.context()`	`anyhow` の部品で、エラーに「何をしていた時のエラーか」の説明を足すメソッド。`fs::read_to_string(path).context("設定ファイルを読めません")?`。説明を `format!` で作る時は、必要な時だけ作る `.with_context(|| …)` を使います。	08 前編 8. 社員の CSV と、管理のコマンド
`.source()`	エラーの「元になったエラー」を返すメソッド。`while let Some(e) = err.source()` のようにたどると、原因をさかのぼれます。	08 前編 8. 社員の CSV と、管理のコマンド
`Context`	`anyhow::Context` は、`with_context`・`context` メソッドを使えるようにする機能（トレイト）。`use anyhow::Context;` と書いておくと、`Result` や `Option` に説明を付けられます。	08 前編 8. 社員の CSV と、管理のコマンド
`panic!`	その場でプログラム（今のスレッド）を止めるマクロ。`panic!("起きるはずのないことが起きました")`。直しようのない、プログラムのまちがいを知らせる時に使います。ふつうの失敗には `Result` を使います。	08 前編 13. 自動のテスト
`.downcast_ref()`	いろいろな種類が入るエラー（`anyhow::Error` など）の中身が、指定した型なら、それを借りて `Some` で返すメソッド。「このエラーは〇〇の種類か」を調べる時に使います。	08 後編 1. 画面の並び
`.and_then()`	`Option` や `Result` に中身があれば、それを使って、次の `Option`・`Result` を作るメソッド（無ければ、そのまま `None`・`Err`）。失敗するかもしれない処理を、続けてつなぐ時に使います。	08 後編 7. 依頼の詳しい画面と、ボタンの操作
`.err()`	`Result` から、エラーの方だけを `Option` で取り出すメソッド。`Err(e)` は `Some(e)` に、`Ok(…)` は `None` になります（`.ok()` の逆）。	08 後編 8. 添付ファイル

関数

コード	意味と使い方	初めて出てくる所
`fn`	関数（ひとまとまりの処理に名前を付けた物）を作る書き方。`fn main() { … }` の `main` は、プログラムを動かすと最初に動く特別な関数です。`fn add(a: i32, b: i32) -> i32 { a + b }` は「整数を 2 つ受け取って、整数を返す関数 `add`」。`{ }` の中が中身で、最後の `;` の無い式が戻り値になります。	07 2. 依頼の状態を enum で書く

コード	意味と使い方	初めて出てくる所
`->`	`fn main() -> std::io::Result<()>` の `->` は、「この関数が返す値の型」を書く印です。`fn add(a: i32, b: i32) -> i32` なら「`i32` を返す」。`->` が無い関数は、何も返しません（`()` という「空」を返します）。	07 2. 依頼の状態を enum で書く
`|x|`（クロージャ）	その場で作る小さな関数（クロージャ）。`|x| x * 2` は「`x` を受け取って 2 倍を返す関数」です。`| |` の間が引数です。並べかえの基準やエラーの時の処理などを、ほかの関数に渡す時によく使います。	07 4. 所有権と借用
`return`	関数を、その場で終えて値を返す命令。`return Err(…);` のように、途中で抜ける時に使います。Rust では、関数の最後の `;` の無い式が自動で戻り値になるので、最後の行では `return` を書かないのがふつうです。	08 前編 3. 設定を読む
`move`	クロージャが使う外の変数を、借りるのではなく「持っていく」印。`thread::spawn(move || { … })` のように、別のスレッドに値を渡す時に必要です。	08 前編 5. ログインとセッション

型とメソッド

コード	意味と使い方	初めて出てくる所
`enum`	「いくつかの種類のうち、どれか 1 つ」を表す型（列挙型）。`enum Filter { All, Active, Done }` のように種類を並べ、`match` で種類ごとに処理を分けます。種類ごとに違うデータを持たせることもできます。	07 2. 依頼の状態を enum で書く
`impl`	2 つの使い方があります。① `impl 型 { … }` で、その型にメソッド（関数）を付ける。② `impl View` のように型の場所に書くと、「`View` の機能を持つ何かの型」という意味になります（戻り値の型を細かく書かずに済む）。	07 2. 依頼の状態を enum で書く
`self`	メソッドの中で「このもの自身」を表す名前。`impl` の中の `fn label(&self)` の `&self` は「自分を借りて読むだけ」、`&mut self` は「自分を書きかえる」という意味です。`use` の中の `self`（例: `use std::io::{self, Write};`）は「`io` そのもの」を表します。	07 2. 依頼の状態を enum で書く
`struct`	いくつかの値をまとめて、新しい型を作る書き方（構造体）。`struct Note { id: u32, title: String }` で、「番号と題名を持つノート」という型ができます。`Note { id: 1, title: "買い物".to_string() }` で 1 つ作ります。	07 2. 依頼の状態を enum で書く

コード	意味と使い方	初めて出てくる所
`#[derive]`	型に、よく使う機能を自動で付ける印（属性）。`#[derive(Debug, Clone, PartialEq)]` のように、付けたい機能の名前を並べます（例: `Debug` → `{:?}` で表示できる、`Clone` → `.clone()` できる）。	07 2. 依頼の状態を enum で書く
`Debug`	`{:?}` で中身を表示できる機能（トレイト）。`#[derive(Debug)]` を付けた型は、`println!("{:?}", 値)` で中身を確かめられます。	07 2. 依頼の状態を enum で書く
`Clone`	`.clone()` で値を丸ごと複製できる機能（トレイト）。`#[derive(Clone)]` で付けます。	07 2. 依頼の状態を enum で書く
`Copy`	代入や関数に渡した時に、移る（ムーブ）のではなく、自動で写される機能（トレイト）。数や `bool` のような小さな値が持っています。`#[derive(Clone, Copy)]` で自分の型にも付けられます（中身が全部 `Copy` の時だけ）。	07 2. 依頼の状態を enum で書く
`PartialEq`	`==` と `!=` で比べられる機能（トレイト）。`#[derive(PartialEq)]` で付けます。	07 2. 依頼の状態を enum で書く
`Eq`	`==` の比べ方が、いつも筋が通っている（自分と比べると必ず等しい）ことを表す印（トレイト）。`HashMap` のキーにする型などに要ります。`PartialEq` と一緒に `#[derive]` で付けます。	07 2. 依頼の状態を enum で書く
`Display`	`{}` で表示できる機能（トレイト）。利用者に見せる形を自分で決めたい時に、`impl fmt::Display for 型 { fn fmt(…) { … } }` と書いて実装します（`Debug` は開発者向けの表示）。	07 2. 依頼の状態を enum で書く
`Formatter`	`Display` や `Debug` の実装の中で受け取る「書き込み先」。`write!(f, "…")` で、そこに表示する文を書きます。	07 2. 依頼の状態を enum で書く
`type`	型に別の名前を付ける書き方（型の別名）。`type DynRepo = Arc<dyn NoteRepository + Send + Sync>;` のように、長い型を短く呼べます。トレイトの中の `type Err = IdError;` は「このトレイトで使うエラーの型は `IdError` 」と決める書き方です。	07 5. Topcoat の最小のアプリ
`.build()`	設定を少しずつ足していく「組み立て係」（ビルダー）の最後に呼んで、でき上がった物を作るメソッド。`Router::builder()…build()` の形で、ルーターなどを作る時に使います。	07 5. Topcoat の最小のアプリ
`.into()`	値を、受け取る側が求める型に変えるメソッド（`From` の実装を使う）。`let s: String = "abc".into();` のように、変換先の型が分かる所で使います。	07 5. Topcoat の最小のアプリ

コード	意味と使い方	初めて出てくる所
`::<型>`	`"42".parse::<i32>()` の `::<i32>` は、「どの型にするか」をその場で指定する書き方です（形が魚に見えるので turbofish と呼ばれます）。`let n: i32 = "42".parse().unwrap();` のように、受け取る変数の型で決めても同じです。`collect::<Vec<_>>()` もよく使います。	07 5. Topcoat の最小のアプリ
`impl トレイト`	`-> Result<impl View>` の `impl View` は、「`View` という決まり（トレイト）を満たす、何かの型」という意味です。本当の型の名前は長くて書けないので、「`View` として使える物を返す」とだけ書きます。引数の `x: impl Display` なら、「表示できる物なら何でも受け取る」です。	07 5. Topcoat の最小のアプリ
`<_>`	`Vec<_>` の `_` は、「中の型は、Rust が使い方から決めて」という印です。`.collect::<Vec<_>>()` は「`Vec` に集めて。中身の型は任せる」という意味です。	07 5. Topcoat の最小のアプリ
`.0`	`pair.0`・`pair.1` のように、タプル（`(1, "a")` のような、いくつかの値の組）は、`.` と番号（0 から数える）で中の値を取り出します。`struct Meters(f64);` のような、項目に名前の無い構造体でも `.0` を使います。	08 前編 3. 設定を読む
`macro_rules!`	自分でマクロ（同じ形のコードを、まとめて作り出す仕組み）を作る書き方。`macro_rules! 名前 { (受け取る形) => { 作るコード } }`。ここでは、似た enum とその変換を、まとめて作る `text_enum!` を作っています。`$name:ident` は「名前を 1 つ受け取る」、`$( … ),*` は「くり返し受け取る」という印です。	08 前編 4. 表
`stringify!`	渡したコードを、そのまま文字列にするマクロ。`stringify!(Status)` は `"Status"`。エラーの文に、型の名前を入れる時に使っています。	08 前編 4. 表
`Self`	`impl` の中で、「今、関数を付けている型そのもの」を表す名前。`fn new(…) -> Self` は「この型の値を返す」という意味で、型の名前を何度も書かずに済みます。	08 前編 4. 表
`TryFrom`	「ある型から作れるが、失敗することもある」変換の機能（トレイト）。`impl TryFrom<CreateNoteInput> for NewNote` を書くと、`NewNote::try_from(input)` で、入力を確かめながら変換でき、だめなら `Err` を返します。	08 前編 4. 表
`Self`	`impl` の中で、「今、関数を付けている型そのもの」を表す名前（`-> Self`・`Self { … }` など）。	08 前編 4. 表
`From`	「ある型から作れる」機能（トレイト）。`impl From<RepoError> for AppError` を書くと、`RepoError` を `AppError` に変えられます。`?` は、エラーを返す時にこの変換を自動で使います。	08 前編 6. 状態を変える処理

コード	意味と使い方	初めて出てくる所
`Into`	「ある型に変えられる」機能（トレイト。`From` の逆向き）。`body: impl Into<String>` は「`String` に変えられる物なら何でも（`&str` でも `String` でも）受け取る」という引数で、中で `.into()` を呼んで変えます。	08 前編 6. 状態を変える処理
`<T>`	`fn longest<'a>(…)` や `struct Wrapper<T>` の、名前のすぐ後ろの `< >` は、「型（やライフタイム）を、使う時に決める」ための入れ物（ジェネリクス）です。`T` は「何かの型」の仮の名前で、使う時に `i32` や `String` などに決まります。	08 前編 6. 状態を変える処理
`Default`	「初期値（空の値）を作れる」機能（トレイト）。`#[derive(Default)]` を付けると、数は 0、文字列は `""`、`bool` は `false` の初期値を作れます。	08 前編 6. 状態を変える処理
`..値`	`Note { title: …, ..note }` の `..note` は、「書かなかった残りの項目は、`note` と同じ値にする」という書き方です（構造体の更新）。`..Default::default()` なら、残りを初期値にします。`..` の後ろの物は、いつも最後に書きます。	08 後編 5. 入力フォーム

モジュール

コード	意味と使い方	初めて出てくる所
`use`	ほかの場所にある道具に、短い名前で呼べるようにする宣言。`use std::env;` と書くと、`std::env::args()` を `env::args()` と短く書けます。`std` は Rust に最初から付いている道具箱（標準ライブラリ）です。	07 2. 依頼の状態を enum で書く
`pub`	「外から使ってよい」（公開）という印。`pub fn title_length(…)` は、ほかのファイル（モジュール）からも呼べる関数になります。付けないと、同じモジュールの中からしか使えません。	07 2. 依頼の状態を enum で書く
`mod`	コードをまとめる箱（モジュール）を作る書き方。`mod tests { … }` のように、関係する関数をまとめます。`mod 名前;` と書くと、別のファイル（`名前.rs`）の中身を読み込みます。	07 2. 依頼の状態を enum で書く
`super`	「1 つ外側のモジュール」を表す名前。テストの中の `use super::*;` は、「外側（テストする本体）にある物を全部使えるようにする」という決まり文句です。	07 2. 依頼の状態を enum で書く
`::*`	`use super::*;` の `*` は、「その中の物を全部、短い名前で呼べるようにする」という意味です。テストの `mod tests` の中で、外側（`super`、1 つ上）の関数を全部使えるようにする時によく書きます。	07 2. 依頼の状態を enum で書く

コード	意味と使い方	初めて出てくる所
`::`	`std::env` の `::` は、「〜の中の」を表す区切りです。`std::env::args()` は「標準ライブラリ（`std`）の中の `env` の中の `args` 関数」という意味です。型の関数（`String::new()`）や、enum の種類（`Ordering::Less`）を指す時にも使います。	07 2. 依頼の状態を enum で書く
`crate`	「このプロジェクト（クレート）のいちばん上」を表す言葉。`use crate::note::Note;` は「このプロジェクトの `note` モジュールの `Note` を使う」という意味です。	07 5. Topcoat の最小のアプリ
`pub(crate)`	`pub(crate)` は、「このクレート（このプロジェクト）の中からは使えるが、外からは使えない」という公開の範囲です。`pub` だけなら、どこからでも使えます。	08 後編 7. 依頼の詳しい画面と、ボタンの操作

所有権と借用

コード	意味と使い方	初めて出てくる所
`&mut`	「書きかえてよい」という約束で値を貸す書き方。`read_to_string(&mut text)` は、関数に `text` を貸して、中身を書き込んでもらいます。貸す元の変数は `let mut text` のように `mut` が付いている必要があります。	07 2. 依頼の状態を enum で書く
`&`	値の前の `&` は「貸す（借用）」の印です。`fs::write(&relative, …)` は、`relative` を手放さずに、見せるだけ貸します。型に付けた `&str`・`&Vec<i32>` は「借りた値」の型です。	07 2. 依頼の状態を enum で書く
`'static`	`&'static str` の `'static` は、「プログラムが動いている間ずっと有効」という意味のライフタイムの印です。`"文字"` のように、コードに直接書いた文字列は `&'static str` です。	07 2. 依頼の状態を enum で書く
`'_`	`Slot<'_>` の `'_` は、「何かを借りているが、その期間（ライフタイム）は Rust に任せる」という印です。名前（`'a`）を付けずに済みます。	07 2. 依頼の状態を enum で書く
`.clone()`	値を丸ごと複製するメソッド。`let b = a.clone();` で、`a` と `b` の両方を使えるようになります。`clone` しないで `let b = a;` と書くと、中身が `b` に移り（所有権の移動）、`a` はもう使えません。	07 5. Topcoat の最小のアプリ
`.as_ref()`	中身を借りた形に変えるメソッド。`Option<String>` を `Option<&String>` にするなど、中身を移さずに覗きたい時に使います。	08 前編 6. 状態を変える処理

コード	意味と使い方	初めて出てくる所
`*`	`*price` のように、借りた値（参照）の前の `*` は「借りている先の、値そのもの」を表します。`*count += 1` は、借りた先の数を 1 増やします。かけ算の `*` とは別物です。	08 前編 6. 状態を変える処理

イテレーター

コード	意味と使い方	初めて出てくる所
`.iter()`	並び（`Vec` など）の中身を、借りて 1 つずつ取り出せる形（イテレーター）にするメソッド。`for x in v.iter()` や、`v.iter().map(…)`・`.filter(…)` のように、加工してから使う時の入り口です。	07 4. 所有権と借用
`.filter()`	条件に合う物だけを残すメソッド。`v.iter().filter(|n| **n % 2 == 0)` は偶数だけ。`|n|` の中の `n` は借りた値なので、`*` が付くことがあります。	07 4. 所有権と借用
`.map()`	中身を 1 つずつ変換するメソッド。イテレーターなら `v.iter().map(|x| x * 2)` で全部を 2 倍に、`Option` なら `Some` の中身だけを変換します（`None` は `None` のまま）。`Result` なら `Ok` の中身だけを変換します。	07 4. 所有権と借用
`.collect()`	1 つずつ取り出せる形（イテレーター）の中身を、`Vec` や `String` などの入れ物に集めるメソッド。何に集めるかを、`let v: Vec<&str> = ….collect();` のように変数の型で教えます。	07 4. 所有権と借用
`.enumerate()`	並びを 1 つずつ取り出す時に、「何番目か」の番号を一緒に付けるメソッド。`for (i, x) in list.iter().enumerate()` で、`i` に 0, 1, 2…、`x` に中身が入ります。	08 前編 6. 状態を変える処理
`.position()`	並びの中で、条件に合う最初の物が「何番目か」を `Option` で返すメソッド。`notes.iter().position(|n| n.id == id)`。無ければ `None` です。	08 前編 8. 社員の CSV と、管理のコマンド
`.any()`	並びの中に、条件に合う物が 1 つでもあれば `true` を返すメソッド。全部が合うかは `.all()` で調べます。	08 前編 8. 社員の CSV と、管理のコマンド
`.count()`	イテレーター（1 つずつ取り出せる物）の中身を最後まで数えて、その数を返すメソッド。`env::args().skip(1).count()` は「渡された引数の数」、`s.chars().count()` は「文字の数」です。	08 前編 8. 社員の CSV と、管理のコマンド

コード	意味と使い方	初めて出てくる所
`.all()`	イテレーターの中身が、全部条件に合うかを `true` / `false` で返すメソッド。`v.iter().all(|n| *n > 0)` は「全部 0 より大きいか」。1 つでも合うかは `.any()` です。	08 前編 8. 社員の CSV と、管理のコマンド
`Iterator`	「1 つずつ取り出せる物」（イテレーター）の機能（トレイト）。`map`・`filter`・`sum` などのメソッドは、全部この機能に付いています。`impl Iterator<Item = &Note>` は「`&Note` を 1 つずつ出す何か」を返す、という戻り値の書き方です。	08 前編 10. 画面から呼ぶ、読むだけの SQL
`Item`	イテレーターが 1 つずつ取り出す物の型（関連型）。`IntoIterator<Item = T>` は「`T` を 1 つずつ取り出せる物なら何でもよい」という意味です。	08 前編 10. 画面から呼ぶ、読むだけの SQL
`.skip()`	並びの先頭から、決めた数だけ飛ばすメソッド。`env::args().skip(1)` は、0 番目（プログラム自身）を飛ばします。	08 前編 11. ここまでの部品を 1 つにまとめる
`.nth()`	並びの中から「n 番目」を 1 つ取り出すメソッド。番号は 0 から数えます。`env::args().nth(1)` は、コマンドの引数の 1 番目（0 番目はプログラム自身の場所）。無いかもしれないので、結果は `Some(値)` か `None`（`Option`）で返ります。	08 前編 13. 自動のテスト
`.next()`	イテレーター（1 つずつ取り出せる物）から、次の 1 つを取り出すメソッド。無くなると `None` を返すので、結果は `Option` です。	08 前編 13. 自動のテスト
`.take()`	イテレーターの先頭から、決めた数だけを取り出すメソッド。`body.chars().take(60)` は先頭の 60 文字。`.collect::<String>()` で文字列に戻せます。	08 後編 8. 添付ファイル
`.filter_map()`	1 つずつ変換して、`None` になった物は捨て、`Some` の中身だけを残すメソッド。`.filter_map(|line| line.split_once('='))` は、`=` を含む行だけを「名前と値の組」にします。	08 後編 8. 添付ファイル
`.into_iter()`	入れ物の中身を、借りずに「もらって」1 つずつ取り出す形にするメソッド。使ったあと、元の入れ物は使えなくなります（`.iter()` は借りるだけ）。	08 後編 9. 集計と、動いているかの確かめ

ファイルとフォルダ

コード	意味と使い方	初めて出てくる所
`PathBuf::from()`	文字列から、ファイルやフォルダの場所（`PathBuf`）を作る関数。`PathBuf::from("files")`。あとで `.join("名前")` で、下の場所をつなげられます。	08 前編 3. 設定を読む
`PathBuf`	ファイルやフォルダの場所（パス）を、自分の物として持つ型（`String` のパス版）。`path.join("a")` で場所をつなげ、`.push("b")` で足せます。借りるだけの形は `&Path` です。	08 前編 3. 設定を読む
`.display()`	ファイルやフォルダの場所（`Path`・`PathBuf`）を、`println!` で表示できる形にするメソッド。場所は、そのままでは `{}` で出せないので、`path.display()` を通して出します。	08 前編 8. 社員の CSV と、管理のコマンド
`std::fs::read()`	ファイルの中身を全部、バイトの並び（`Vec<u8>`）として読む関数。文字として読む時は `fs::read_to_string` です。	08 前編 8. 社員の CSV と、管理のコマンド
`include_bytes!`	ファイルの中身を、プログラムを作る時に、バイトの並びとして中に埋め込むマクロ。`include_bytes!("../sample_data/employees.csv")`。テストで、決まったファイルを使う時に便利です。	08 前編 8. 社員の CSV と、管理のコマンド
`.has_headers()`	CSV の 1 行目が見出し（列の名前）かどうかを決めるメソッド。`true` なら、1 行目は中身として数えません。	08 前編 8. 社員の CSV と、管理のコマンド
`.from_reader()`	決めた読み方で、渡したデータ（バイトの並びやファイル）から CSV を読む係を作るメソッド。	08 前編 8. 社員の CSV と、管理のコマンド
`.join()`	場所（パス）に、フォルダやファイルの名前をつなげた新しい場所を作るメソッド。`dir.join("notes.txt")` は、`dir` の中の `notes.txt` を表します。区切りの `\` は自動で入ります。（文字列の配列に使う `.join(", ")` は、「区切り文字でつなげて 1 つの文字列にする」別の意味です。）	08 前編 8. 社員の CSV と、管理のコマンド
`.records()`	CSV の行を、1 行ずつ取り出せる形にするメソッド。`for (i, record) in reader.records().enumerate()` で、何行目かと中身を同時に使えます。	08 前編 8. 社員の CSV と、管理のコマンド
`csv::ReaderBuilder::new()`	`csv` の部品で、CSV の読み方を組み立て始める関数。`.has_headers(true)`（1 行目は見出し）などを決めて、`.from_reader(バイト)` で読む係を作ります。	08 前編 8. 社員の CSV と、管理のコマンド

コード	意味と使い方	初めて出てくる所
`ReaderBuilder`	`csv` の、CSV の読み方を組み立てる型。	08 前編 8. 社員の CSV と、管理のコマンド
`.exists()`	その場所に、ファイルかフォルダがあれば `true` を返すメソッド。`Path::new("memo.txt").exists()`。	08 前編 13. 自動のテスト
`std::env::temp_dir()`	OS の、一時的なファイルを置くフォルダの場所を返す関数（Windows なら `C:\Users\you\AppData\Local\Temp`）。	08 前編 13. 自動のテスト
`std::fs::read_dir()`	フォルダの中の一覧を返す関数（`fs::read_dir` と同じ）。	08 前編 13. 自動のテスト
`std::fs::remove_dir_all()`	フォルダを、中身ごと全部消す関数。ごみ箱を通さずに消えるので、場所をよく確かめて使います。	08 前編 13. 自動のテスト
`.file_name()`	場所（パス）から、最後の部分（ファイル名やフォルダ名）だけを取り出すメソッド。`C:\data\memo.txt` なら `memo.txt`。無いこともあるので `Option` で返ります。	08 後編 5. 入力フォーム
`tokio::fs::create_dir_all()`	フォルダを、途中のフォルダもまとめて作る関数（非同期。`.await` で待つ）。もうあればそのままです。	08 後編 8. 添付ファイル
`tokio::fs::write()`	ファイルに中身を書く関数（非同期）。ファイルが無ければ作り、あれば上書きします。	08 後編 8. 添付ファイル
`tokio::fs::remove_file()`	ファイルを消す関数（非同期）。	08 後編 8. 添付ファイル
`Path`	ファイルやフォルダの場所を表す型（借りている形）。`.join()`・`.file_name()`・`.extension()`・`.exists()` などで扱います。自分の物として持つ形は `PathBuf` です。	08 後編 8. 添付ファイル

入出力

コード	意味と使い方	初めて出てくる所
`.read_line()`	1 行だけ読んで、文字列に書き足すメソッド。読んだ行の最後には改行が付いたままなので、`.trim_end()` などで取ってから使います。	08 前編 8. 社員の CSV と、管理のコマンド
`std::io::stdin()`	キーボードからの入力（標準入力）を表す物を返す関数。`std::io::stdin().lock().read_line(&mut line)?` で、1 行読めます。	08 前編 8. 社員の CSV と、管理のコマンド

コード	意味と使い方	初めて出てくる所
`BufRead`	「1 行ずつ読める物」が持っている機能（トレイト）。`read_line` や `lines` を使うには、`use std::io::BufRead;` が要ります。	08 前編 8. 社員の CSV と、管理のコマンド

プロセス

コード	意味と使い方	初めて出てくる所
`env::var()`	環境変数（プログラムの外から渡す設定の値）を読む関数。`env::var("GREETING")`。設定されていないこともあるので、`Result` で返ります。`unwrap_or_else(|_| "既定値".to_string())` で、無い時の値を決めるのがふつうです。	08 前編 3. 設定を読む
`std::env::args()`	プログラムに渡された引数を取り出す関数（`env::args()` と同じ）。	08 前編 11. ここまでの部品を 1 つにまとめる
`ExitCode`	`main` が返す、終了コードの型。`fn main() -> ExitCode` にして、`ExitCode::SUCCESS`（0）か `ExitCode::FAILURE`（1）を返します。`process::exit` と違い、片づけをしてから終われます。	08 前編 11. ここまでの部品を 1 つにまとめる
`SUCCESS`	`ExitCode::SUCCESS` は、終了コード 0（成功）です。	08 前編 11. ここまでの部品を 1 つにまとめる
`FAILURE`	`ExitCode::FAILURE` は、終了コード 1（失敗）です。	08 前編 11. ここまでの部品を 1 つにまとめる

設定

コード	意味と使い方	初めて出てくる所
`dotenvy::dotenv()`	`.env` ファイルに書いた「名前=値」を、環境変数として読み込む関数（`dotenvy` の部品）。開発のパソコンで、設定をファイルにまとめておく時に使います。`.env` は Git に入れません。	08 前編 8. 社員の CSV と、管理のコマンド

時間

コード	意味と使い方	初めて出てくる所
`offset!`	`time` の部品で、時差（`UtcOffset`）を書くマクロ。`offset!(+9)` は「世界標準時より 9 時間進んでいる」＝日本時間です。プログラムを作る時に確かめられるので、まちがった書き方はエラーになります。	08 前編 4. 表
`format_description!`	`time` の部品で、日時を文字にする形（書式）を書くマクロ。`format_description!("[year]-[month]-[day] [hour]:[minute]")` は「2026-11-02 08:00」の形です。`.format(…)` に渡して使います。	08 前編 4. 表
`.to_offset()`	日時（`OffsetDateTime`）を、決めた時差の日時に変えるメソッド。`t.to_offset(JST)` は日本時間にします。指す瞬間は同じで、表し方だけが変わります。	08 前編 4. 表
`.format()`	日時を、決めた形（`format_description!`）の文字列にするメソッド。結果は `Result` なので、`.unwrap_or_default()`（失敗したら空の文字）などで受け取ります。	08 前編 4. 表
`.date()`	日時（`OffsetDateTime`）から、日付（`Date`。年・月・日）だけを取り出すメソッド。	08 前編 4. 表
`OffsetDateTime::now_utc()`	今の日時を、世界標準時（UTC）で返す関数。日本時間にする時は `.to_offset(JST)` をつなげます。	08 前編 4. 表
`Date`	`time` の、日付（年・月・日）を表す型。データベースの `DATE` の列と行き来できます。	08 前編 4. 表
`OffsetDateTime`	`time` の、時差付きの日時を表す型。データベースの `TIMESTAMPTZ` の列と行き来できます。	08 前編 4. 表
`UtcOffset`	`time` の、時差（世界標準時から何時間ずれているか）を表す型。`offset!(+9)` で日本の時差を作れます。	08 前編 4. 表
`Duration::from_secs()`	秒から、時間の長さ（`Duration`）を作る関数。`Duration::from_secs(10)` は 10 秒。	08 前編 5. ログインとセッション
`OffsetDateTime::from()`	ほかの形の日時（`SystemTime` など）から、`OffsetDateTime` を作る関数。Topcoat のセッションの期限（`SystemTime`）を、データベースに入れる形に変えています。	08 前編 5. ログインとセッション
`Duration`	時間の長さを表す型。`Duration::from_secs(5)` で 5 秒。`{:?}` で表示すると `5s` や `200ms` のように単位つきで出ます。	08 前編 5. ログインとセッション
`.month()`	日時から「月」を取り出すメソッド。`Month::November` のような enum で返るので、数にする時は `u8::from(now.month())` とします。	08 前編 6. 状態を変える処理

コード	意味と使い方	初めて出てくる所
`.day()`	日時から「日」（1〜31）を取り出すメソッド。	08 前編 6. 状態を変える処理
`.hour()`	日時から「時」（0〜23）を取り出すメソッド。	08 前編 6. 状態を変える処理
`.minute()`	日時から「分」（0〜59）を取り出すメソッド。	08 前編 6. 状態を変える処理
`time::OffsetDateTime::now_utc()`	今の日時を、世界標準時（UTC）で返す関数（`OffsetDateTime::now_utc()` と同じ）。	08 前編 8. 社員の CSV と、管理のコマンド
`time!`	`time` の部品で、時刻（`Time`）を書くマクロ。`time!(8:00)` は 8 時 0 分。まちがった時刻（`25:00` など）は、プログラムを作る時にエラーになります。	08 前編 9. 毎朝 8:00 の通知
`.time()`	日時（`OffsetDateTime`）から、時刻（`Time`。時・分・秒）だけを取り出すメソッド。`now.time() < RUN_AT` は「今の時刻が、決めた時刻（8:00）より前か」です。	08 前編 9. 毎朝 8:00 の通知
`time::Duration::days()`	`time` の、日数ぶんの長さ（`Duration`）を作る関数。`今日 + time::Duration::days(3)` は「3 日後」です。	08 前編 9. 毎朝 8:00 の通知
`Time`	`time` の、時刻（時・分・秒）を表す型。`time!(8:00)` で「8 時 0 分」を作れます。	08 前編 9. 毎朝 8:00 の通知
`std::time::Duration::from_secs()`	秒数ぶんの長さ（`Duration`）を作る関数。`Duration::from_secs(5)` は 5 秒です。	08 前編 11. ここまでの部品を 1 つにまとめる
`date!`	`time` の部品で、日付（`Date`）を書くマクロ。`date!(2026 - 11 - 30)` は 2026 年 11 月 30 日。テストで、決まった日付を作る時に使います。	08 前編 13. 自動のテスト
`datetime!`	`time` の部品で、時差付きの日時を書くマクロ。`datetime!(2026-11-02 07:59 +9)` は「日本時間の 2026 年 11 月 2 日 7 時 59 分」。テストで「8 時より前」などの時刻を作るのに使います。	08 前編 13. 自動のテスト
`Date::parse()`	文字列を、決めた形（`format_description!`）で読んで、`Date` にする関数。形に合わない文字（`2026/13/40` など）だと `Err` を返します。	08 後編 5. 入力フォーム

スレッドと共有

コード	意味と使い方	初めて出てくる所
`.lock()`	`Mutex` の鍵を取って、中身を使えるようにするメソッド。返ってきた物（ガード）を通して中身を読み書きし、ガードが片づけられると鍵は自動で返されます。ほかのスレッドが鍵を持っている間は、返されるまで待ちます。	07 5. Topcoat の最小のアプリ
`Mutex::new()`	値を、鍵付きの入れ物（ `Mutex` ）に入れる関数。使う時は `.lock()` で鍵を取ります。	07 5. Topcoat の最小のアプリ
`Mutex`	中身を、1 度に 1 つのスレッドだけが使えるようにする鍵付きの入れ物。`.lock()` で鍵を取ってから使います。	07 5. Topcoat の最小のアプリ
`LazyLock::new()`	初めて使われた時に 1 回だけ作る値を、作り方の関数と一緒に用意する関数。	08 前編 5. ログインとセッション
`LazyLock`	初めて使われた時に 1 回だけ作られ、その後はずっと同じ値を使う入れ物。`static DUMMY: LazyLock<String> = LazyLock::new(|| { … });` のように、作るのに手間がかかる物を、プログラム全体で共有する時に使います。	08 前編 5. ログインとセッション
`Arc::new()`	値を、いくつものスレッドで一緒に持てる `Arc` に入れる関数。`Arc::new(Mutex::new(0))` は「鍵付きの 0 を、みんなで持つ」。	08 前編 8. 社員の CSV と、管理のコマンド
`Arc`	いくつものスレッドから、1 つの値を一緒に持てるようにする型（ `Rc` のスレッド版）。書きかえもしたい時は、`Arc<Mutex<T>>` の形にします。	08 前編 8. 社員の CSV と、管理のコマンド

非同期

コード	意味と使い方	初めて出てくる所
`async`	「待つ所で、ほかの仕事に順番をゆずれる関数」にする印。`async fn root(…)` のように付けます。呼ぶ側は `.await` で結果を待ちます。Web サーバーが、たくさんの人を同時にさばけるのは、この仕組みのおかげです。	07 5. Topcoat の最小のアプリ
`await`	`async` の関数（Future）が終わるのを待って、結果を受け取る書き方。`sleep(…).await` のように、後ろに `.await` を付けます。待っている間は、ほかの仕事に順番をゆずります。`.await` を付けないと、その仕事は始まりません。	08 前編 5. ログインとセッション

コード	意味と使い方	初めて出てくる所
`tokio::task::spawn_blocking()`	時間のかかる重い計算や、待ってしまう処理（パスワードのハッシュなど）を、非同期の仕事を回すスレッドとは別のスレッドで動かす関数。ほかの仕事を止めないために使います。	08 前編 5. ログインとセッション
`tokio::spawn()`	非同期の仕事を、別のタスクとしてすぐ動かし始める関数。`tokio::spawn(async move { … })`。返ってくる物（`JoinHandle`）を `.await` すると、結果を受け取れます。	08 前編 9. 毎朝 8:00 の通知
`tokio::time::sleep()`	決めた時間だけ待つ、非同期の関数。`tokio::time::sleep(Duration::from_millis(300)).await`。待っている間、ほかの仕事は止まりません（`std::thread::sleep` はスレッドごと止めてしまうので、`async` の中では使いません）。	08 前編 9. 毎朝 8:00 の通知
`#[tokio]`	`#[tokio::main]` は、`main` を `async` にして、tokio（非同期の仕事を回す仕組み）の上で動かす印（属性）。`#[tokio::test]` は、テストを `async` で書けるようにする印です。	08 前編 11. ここまでの部品を 1 つにまとめる
`join!`	`tokio::join!(a, b)` は、いくつかの非同期の仕事を同時に進めて、全部が終わるのを待つマクロ。かかる時間は、いちばん長い物の分だけです。	08 前編 13. 自動のテスト

HTTP

コード	意味と使い方	初めて出てくる所
`StatusCode`	HTTP の状態コード（`200 OK`・`404 Not Found` など）を表す型。`StatusCode::OK`・`StatusCode::NOT_FOUND` のように、名前で書けます。	08 前編 7. 1 回限りの承認の URL
`Response`	答え（レスポンス）の型。Topcoat では `topcoat::router::response::Response`。ふつうのページは `view!` で作りますが、ファイルのダウンロードのように、見出しや本文を自分で決めたい時に直接作ります。	08 前編 7. 1 回限りの承認の URL
`OK`	`StatusCode::OK` は、HTTP の答えの番号 200（成功）です。	08 前編 7. 1 回限りの承認の URL
`GONE`	状態コード 410（Gone: もう無い）。`StatusCode::GONE`。ここでは、承認の URL の期限が切れた・もう使えない時に返しています（404 の「初めから無い」と区別する）。	08 前編 7. 1 回限りの承認の URL

コード	意味と使い方	初めて出てくる所
`CONFLICT`	`StatusCode::CONFLICT` は、HTTP の答えの番号 409（「ぶつかった」）。同じメールアドレスで登録しようとした時などに返します。	08 前編 8. 社員の CSV と、管理のコマンド
`.headers()`	答え（レスポンス）の見出し（ヘッダー）の一覧を返すメソッド。`.headers().get("set-cookie")` のように、名前で 1 つ取り出せます。	08 前編 8. 社員の CSV と、管理のコマンド
`UNAUTHORIZED`	`StatusCode::UNAUTHORIZED` は、HTTP の答えの番号 401（「ログインしていない」）です。	08 後編 3. ログインの画面
`.header()`	答えやリクエストに、見出し（ヘッダー）を 1 つ足すメソッド。`.header("Content-Type", "application/pdf")` は「中身は PDF」。	08 後編 8. 添付ファイル
`.body()`	リクエストやレスポンスの本文を取り出す（決める）メソッド。テストで `Request::builder()….body(Body::from("…"))` のように本文を入れます。	08 後編 8. 添付ファイル
`Response::builder()`	答え（レスポンス）を、少しずつ決めて組み立て始める関数。`.header(名前, 値)` で見出しを足し、`.body(…)` で本文を入れて出来上がりです。	08 後編 8. 添付ファイル
`Body::from()`	文字列やバイトの並びから、本文（`Body`）を作る関数。	08 後編 8. 添付ファイル
`Body`	リクエストや答えの本文を表す型（`topcoat::router::Body`）。	08 後編 8. 添付ファイル

認証

コード	意味と使い方	初めて出てくる所
`Argon2::default()`	パスワードを安全に保存するための計算（Argon2id）を、おすすめの設定で用意する書き方（`argon2` の部品）。この計算は、わざと重く作られていて、パスワードを総当たりで当てるのを難しくします。	08 前編 5. ログインとセッション
`PasswordHash::new()`	保存してあるハッシュの文字列を読み取って、確かめに使える形にする関数。形が違えばエラーです。	08 前編 5. ログインとセッション
`Argon2`	パスワードをハッシュにする方法の 1 つ（Argon2）を使うための型。`Argon2::default()` で、おすすめの設定のものを作ります。わざと時間とメモリを使うので、総当たりで破られにくくなります。	08 前編 5. ログインとセッション

コード	意味と使い方	初めて出てくる所
`PasswordHasher`	パスワードからハッシュ（保存用の値）を作る機能（トレイト。`password_hash` の部品）。`argon2.hash_password(…)` を使うには、これを `use` します。	08 前編 5. ログインとセッション
`PasswordVerifier`	入力されたパスワードが、保存してあるハッシュと合うかを確かめる機能（トレイト）。`argon2.verify_password(…)` を使うには、これを `use` します。	08 前編 5. ログインとセッション
`PasswordHash`	保存してあるハッシュの文字列（ `$argon2id$v=19$…` ）を読み取った物の型。`PasswordHash::new(&保存した文字列)` で作り、確かめに使います。	08 前編 5. ログインとセッション
`getrandom::fill()`	OS が用意する、予想できない乱数で、バイトの並びを埋める関数。`getrandom::fill(&mut bytes)` 。承認の URL の文字列（トークン）のように、当てられてはいけない値を作るのに使います。	08 前編 6. 状態を変える処理
`Sha256::digest()`	データから、SHA-256 というハッシュ（32 バイトの指紋）を計算する関数（ `sha2` の部品）。	08 前編 6. 状態を変える処理
`Digest`	ハッシュの計算（ `digest` ・ `update` ・ `finalize` ）を使えるようにする機能（トレイト。`sha2` などの部品）。	08 前編 6. 状態を変える処理
`Sha256`	SHA-256 というハッシュの計算をする型。データの「指紋」を作るのに使います（パスワードの保存には、わざと重い Argon2 を使います）。	08 前編 6. 状態を変える処理

テスト

コード	意味と使い方	初めて出てくる所
`assert_eq!`	「2 つの値が同じか」を確かめるマクロ。`assert_eq!(実際の値, 期待する値);` 。違う時はテストが失敗し、`left` （実際）と `right` （期待）の値を表示します。	07 2. 依頼の状態を enum で書く
`assert!`	「条件が `true` か」を確かめるマクロ。`assert!(is_valid(&title));` 。`false` ならテストが失敗します。	07 2. 依頼の状態を enum で書く
`#[cfg]`	「この条件の時だけ、この部分を翻訳する」という印（属性）。`#[cfg(test)]` を付けた部分は、`cargo test` の時だけ翻訳され、ふつうの実行ファイルには入りません。	07 2. 依頼の状態を enum で書く
`#[test]`	「この関数はテストです」という印（属性）。`cargo test` が、この印の付いた関数を全部探して動かします。	07 2. 依頼の状態を enum で書く

コード	意味と使い方	初めて出てくる所
`.unwrap_err()`	`Result` が `Err` の時に、そのエラーを取り出すメソッド。`Ok` だった時は止まります。テストで「ちゃんと失敗するか」を確かめる時に使います。	07 2. 依頼の状態を enum で書く
`#[cfg(test)]`	`#[cfg(test)]` を付けた所（ふつうは `mod tests { … }`）は、`cargo test` の時だけ翻訳されます。`cargo run` や本番の実行ファイルには入りません。テストのコードは、この中に書きます。	07 2. 依頼の状態を enum で書く

データベース

コード	意味と使い方	初めて出てくる所
`#[sqlx]`	`sqlx` に、列の読み方を伝える印。`#[sqlx(try_from = "String")]` は「この列は、まず文字として読み、`TryFrom` で（ここでは enum に）変える」という意味です。	08 前編 4. 表
`FromRow`	`sqlx` の部品で、SQL の結果の 1 行を、struct に読み込めるようにする印。`#[derive(sqlx::FromRow)]` を付けると、列の名前と同じ名前の項目に、自動で入れてくれます。	08 前編 4. 表
`Json`	`sqlx` の `Json<型>`（`sqlx::types::Json`）は、`JSONB` の列を、Rust の struct として読み書きするための包みです。`Option<Json<Drawing>>` は「図面が無ければ `None`」。（Topcoat にも、答えを JSON で返すための `Json` があり、名前は同じでも別の物です。）	08 前編 4. 表
`#[sqlx(try_from)]`	`#[sqlx(try_from = "String")]` は、データベースから読んだ値を、いったん `String` として受け取り、`TryFrom` でこの型に変える印です。決まった文字（状態の名前など）だけを受け付ける型で、データベースの列を読む時に使います。	08 前編 4. 表
`.bind()`	SQL の `$1`・`$2` … に入れる値を、順に渡すメソッド。値は SQL の文とは別に送られるので、どんな文字が入っていても SQL として読まれません（SQL インジェクションを防ぐ）。	08 前編 5. ログインとセッション
`.fetch_optional()`	SQL を送って、結果を 0 行か 1 行で受け取るメソッド。行が無ければ `None` です（`Option` で返る）。	08 前編 5. ログインとセッション
`.execute()`	値を入れたり変えたりする SQL の文を、1 つ実行するメソッド（libsql）。`conn.execute("UPDATE … WHERE id = ?1", params![id]).await?` のように、値は `params!` で渡します。変わった行の数が返ります。	08 前編 5. ログインとセッション

コード	意味と使い方	初めて出てくる所
`sqlx::query_as()`	`sqlx` で、結果の行を、決めた型（`FromRow` の struct や、`(i64, String)` のような組）で受け取る SQL を用意する関数。	08 前編 5. ログインとセッション
`sqlx::query()`	`sqlx` で、SQL の文を用意する関数。`sqlx::query("UPDATE … WHERE id = $1").bind(id).execute(pool).await?` のように、`$1`・`$2` に入れる値を `.bind()` で渡して、`.execute()`・`.fetch_…()` で送ります。	08 前編 5. ログインとセッション
`.fetch_one()`	SQL を送って、結果を 1 行受け取るメソッド。行が無いとエラーになるので、必ず 1 行ある時（`INSERT … RETURNING id` など）に使います。	08 前編 6. 状態を変える処理
`.commit()`	トランザクションの中の変更を、まとめて確定するメソッド。`tx.commit().await?`。`commit` しないまま `tx` が無くなると、変更は全部取り消されます（ロールバック）。	08 前編 6. 状態を変える処理
`.begin()`	トランザクション（いくつもの SQL を、全部するか全部しないかの 1 まとまりにする仕組み）を始めるメソッド。`let mut tx = pool.begin().await?;`。	08 前編 6. 状態を変える処理
`.fetch_all()`	SQL を送って、結果の行を全部 `Vec` で受け取るメソッド。	08 前編 6. 状態を変える処理
`.rows_affected()`	`.execute()` の結果から、変わった行の数を返すメソッド。0 なら「条件に合う行が無かった（もう誰かが変えた）」と分かります。	08 前編 6. 状態を変える処理
`sqlx::query_scalar()`	`sqlx` で、結果が 1 つの値だけ（`count(*)` など）の SQL を用意する関数。`let found: i64 = sqlx::query_scalar("SELECT count(*) …")…` のように、値をそのまま受け取れます。	08 前編 6. 状態を変える処理
`PgConnection`	`sqlx` の、PostgreSQL への接続 1 本の型。トランザクション（`Transaction`）の中の接続も、`&mut *tx` でこの型として渡せます。	08 前編 6. 状態を変える処理
`Postgres`	`sqlx` で、「相手のデータベースは PostgreSQL」を表す型。`Transaction<'_, Postgres>` のように、型の中で使います。	08 前編 6. 状態を変える処理
`Transaction`	`sqlx` の、トランザクションの型。`pool.begin()` で始まり、`.commit()` で確定します。	08 前編 6. 状態を変える処理
`AssertSqlSafe`	`sqlx` に「この組み立てた SQL の文は、決まった文字だけでできていると確かめた」と伝える包み。`sqlx::query(sqlx::AssertSqlSafe(sql))`。外から来た値は、必ず `$1` と `.bind()` で別に渡します（そうしないと SQL インジェクションの穴になる）。	08 前編 6. 状態を変える処理

コード	意味と使い方	初めて出てくる所
`PgPool`	`sqlx` の、PostgreSQL への接続の集まり（プール）の型。何本かの接続を用意しておき、頼まれるたびに貸し出します。複製（ `.clone()` ）しても、同じ集まりを指します。	08 前編 6. 状態を変える処理
`.acquire()`	接続の集まり（プール）から、接続を 1 本借りるメソッド。トランザクションを使わずに、同じ接続で続けて SQL を送りたい時に使います。	08 前編 7. 1 回限りの承認の URL
`sqlx::PgPool::connect_lazy()`	PostgreSQL へのプールを、まだつながずに作る関数。最初に使う時に、はじめてつなぎます。	08 前編 8. 社員の CSV と、管理のコマンド
`migrate!`	`sqlx` の部品で、 `migrations` フォルダの SQL のファイル（テーブルを作る・変える手順）を、プログラムの中に取り込むマクロ。 `sqlx::migrate!("./migrations").run(pool).await` で、まだ流していない物を順に流します。	08 前編 11. ここまでの部品を 1 つにまとめる
`.max_connections()`	プールに用意する接続の、いちばん多い数を決めるメソッド。	08 前編 11. ここまでの部品を 1 つにまとめる
`.acquire_timeout()`	接続を借りる時に、空くのを待つ長さの上限を決めるメソッド。これを過ぎるとエラーにします（ずっと待ち続けない）。	08 前編 11. ここまでの部品を 1 つにまとめる
`PgPoolOptions::new()`	`PgPoolOptions` を、決まりの設定で作る関数。 `.max_connections(10)` ・ `.acquire_timeout(…)` をつなげて決め、最後に `.connect(URL)` でプールを作ります。	08 前編 11. ここまでの部品を 1 つにまとめる
`PgPoolOptions`	`sqlx` の、PostgreSQL の接続の集まり（プール）の設定を組み立てる型。	08 前編 11. ここまでの部品を 1 つにまとめる
`MigrateError`	`sqlx` の、マイグレーションに失敗した時のエラーの型。	08 前編 11. ここまでの部品を 1 つにまとめる
`Text`	`Value::Text(s)` は、データベースの値が文字列であることを表す種類です。	08 前編 13. 自動のテスト

SQL

コード	意味と使い方	初めて出てくる所
`CREATE`	`CREATE ROLE`（ユーザーを作る）・`CREATE DATABASE`（データベースを作る）・`CREATE TABLE`（表を作る）のように、何かを新しく作る SQL の命令です。	06 前編 2. 開発の準備
`ROLE`	PostgreSQL のユーザー（ログインする名前と、持つ権限のまとまり）。`CREATE ROLE webapp_dev_app LOGIN CREATEDB;` は「ログインでき、データベースを作れるユーザー」を作ります。	06 前編 2. 開発の準備
`LOGIN`	`CREATE ROLE … LOGIN` で、そのユーザーがログインできるようにします。	06 前編 2. 開発の準備
`CREATEDB`	`CREATE ROLE … CREATEDB` で、そのユーザーがデータベースを作れるようにします（Django のテストが、テスト用のデータベースを作るのに要ります）。	06 前編 2. 開発の準備
`DATABASE`	`CREATE DATABASE 名前` で、データベースを作ります。	06 前編 2. 開発の準備
`OWNER`	`OWNER ユーザー` で、そのデータベースの持ち主を決めます。持ち主は、中の表を自由に作れます。	06 前編 2. 開発の準備
`ENCODING`	`ENCODING 'UTF8'` で、データベースの文字コードを UTF-8 にします（日本語を正しく入れるため）。	06 前編 2. 開発の準備
`TEMPLATE`	`TEMPLATE template0` で、何も手を加えていない元の形から、データベースを作ります（文字コードを指定する時に必要）。	06 前編 2. 開発の準備
`'文字'`	SQL では、文字の値を `'…'`（シングルクォート）で囲みます（`'2026-11-05'`・`'山田'`）。`"…"`（ダブルクォート）は表や列の名前に使う物なので、文字の値には使いません。中に `'` を入れる時は `''` と 2 つ書きます。	06 前編 2. 開発の準備
`--`	SQL では、`--` から行の終わりまでがコメントです。	06 前編 2. 開発の準備
`;`	SQL の `;` は、1 つの命令（文）の終わりの印です。psql では、`;` を打つまで命令が送られません（`;` を忘れると、続きを待つ `practice-#` の表示になります）。	06 前編 2. 開発の準備
`PASSWORD`	`CREATE ROLE 名前 LOGIN CREATEDB PASSWORD '…'` の `PASSWORD` は、そのロール（データベースの利用者）がログインする時のパスワードを決める部分です。`LOGIN` はログインしてよい、`CREATEDB` はデータベースを作ってよい（テストで使う）という意味です。	08 前編 2. 開発の準備

コード	意味と使い方	初めて出てくる所
`TABLE`	SQL で「表」を表す言葉。`CREATE TABLE`（表を作る）・`DROP TABLE`（表を消す）のように使います。	08 前編 4. 表
`BIGSERIAL`	PostgreSQL の、行を足すたびに 1, 2, 3… と自動で番号を振る列の型（大きな整数）。`id BIGSERIAL PRIMARY KEY` のように、行の番号に使います。	08 前編 4. 表
`PRIMARY`	`PRIMARY KEY`（主キー）の一部。主キーは、表の中で 1 行を必ず見分けられる列で、同じ値が 2 回入りません。ふつうは `id` 列にします。	08 前編 4. 表
`KEY`	`PRIMARY KEY`（主キー）・`FOREIGN KEY`（よその表とつなぐ列）などの「キー」。	08 前編 4. 表
`TEXT`	SQL の列の型の 1 つで、文字を入れる列。	08 前編 4. 表
`NOT`	SQL で「〜ではない」。`NOT NULL` は「この列は空（NULL）を許さない」という決まりです。	08 前編 4. 表
`NULL`	SQL で「値が無い」ことを表す特別な値。0 や空の文字列とは違います。	08 前編 4. 表
`UNIQUE`	「この列の値は、表の中で重ならない」という決まり。`email TEXT NOT NULL UNIQUE` は、同じメールアドレスを 2 回登録できなくします。	08 前編 4. 表
`DEFAULT`	列に値を入れなかった時に使う、決めておいた値（既定値）。`done INTEGER NOT NULL DEFAULT 0` は「指定が無ければ 0」。	08 前編 4. 表
`CHECK`	「この列に入れてよい値の条件」を決める決まり。`CHECK (length(title) BETWEEN 1 AND 80)` は、題名を 1〜80 文字に限ります。合わない値を入れようとすると、データベースが断ります。	08 前編 4. 表
`IN`	SQL で「このどれか」。`WHERE status IN ('queued', 'running')` は、状態がどちらかの行だけです。`CHECK (pinned IN (0, 1))` は、0 か 1 しか入れられない決まりです。	08 前編 4. 表
`BOOLEAN`	`TRUE`（はい）か `FALSE`（いいえ）を入れる列の型。`is_active BOOLEAN NOT NULL DEFAULT TRUE` は「使えるか。決まりは はい」。	08 前編 4. 表
`TRUE`	SQL の「はい」（真）の値。逆は `FALSE` です。	08 前編 4. 表
`BYTEA`	PostgreSQL の、バイトの並び（文字でないデータ）を入れる列の型。ここでは、承認の URL の文字列のハッシュ（SHA-256 の 32 バイト）を入れています。	08 前編 4. 表
`BIGINT`	大きな整数（約 ±900 京まで）を入れる列の型。`BIGSERIAL` の番号を指す列（`REFERENCES employees (id)`）は、同じ大きさの `BIGINT` にします。Rust では `i64` で受け取ります。	08 前編 4. 表

コード	意味と使い方	初めて出てくる所
`REFERENCES`	「この列の値は、あの表の、あの列に必ずある値」という決まり（外部キー）。`user_id INTEGER REFERENCES users(id)` で、存在しないユーザーの番号を入れられなくなります。	08 前編 4. 表
`ON`	SQL の「〜の時」「〜で」。`ON DELETE CASCADE` は「消された時は」、`JOIN … ON a.id = b.user_id` は「この条件で横につなぐ」、`CREATE INDEX … ON 表 (列)` は「この表の、この列に」という意味です。	08 前編 4. 表
`DELETE`	`DELETE FROM 表 WHERE 条件` で、表の行を消す SQL の命令。`WHERE` を忘れると、全部の行が消えます。`ON DELETE CASCADE` の中では「消された時」という意味です。	08 前編 4. 表
`CASCADE`	`ON DELETE CASCADE` は、「つながった元の行（ユーザー）が消されたら、こちらの行（そのユーザーのノート）も一緒に消す」という決まりです。	08 前編 4. 表
`TIMESTAMPTZ`	PostgreSQL の、時差（タイムゾーン）付きの日時の型。中では世界標準時（UTC）で持ち、表示する時に日本時間などに変えます。Rust では `time::OffsetDateTime` で受け取ります。	08 前編 4. 表
`BETWEEN`	「〜から〜の間」。`CHECK (score BETWEEN 0 AND 100)` は「0 以上 100 以下だけ許す」という決まりです。	08 前編 4. 表
`AND`	SQL で「かつ」。`WHERE a = 1 AND b = 2` は両方に合う行だけ。`BETWEEN 0 AND 100` の一部としても使います。	08 前編 4. 表
`DATE`	日付（年・月・日。時刻は無し）を入れる列の型。期限（`due_date`）などに使います。Rust では `time::Date` で受け取ります。	08 前編 4. 表
`JSONB`	PostgreSQL の、JSON を入れる列の型。中で扱いやすい形に変えて持つので、速く探せます。ここでは図面（四角や線の並び）を入れています。Rust では `sqlx::types::Json<型>` で受け取ります。	08 前編 4. 表
`SMALLINT`	小さな整数（−32768〜32767）を入れる列の型。何回目か（`round`）・何番目か（`step_order`）などに使います。Rust では `i16` で受け取ります。	08 前編 4. 表
`INDEX`	`CREATE INDEX 名前 ON 表 (列)` で、表に索引（本の索引のような、速く探すための仕組み）を作る SQL の命令。よく条件や並べかえに使う列に作ります。	08 前編 4. 表
`INTEGER`	SQL の列の型の 1 つで、整数を入れる列。	08 前編 4. 表
`char_length()`	PostgreSQL の `char_length(title)` は、文字の数を数える関数です。	08 前編 4. 表

コード	意味と使い方	初めて出てくる所
`now()`	PostgreSQL の `now()` は、今の日時を返す関数です。`DEFAULT now()` で、行を作った日時を自動で入れられます。	08 前編 4. 表
`ALL`	`UNION ALL` は、2 つの結果を、重なりも消さずにそのままつなげる書き方です。	08 前編 4. 表
`SELECT`	表から行を取り出す SQL の命令。`SELECT 列, … FROM 表 WHERE 条件 ORDER BY 列` の形でよく使います。`SELECT *` は全部の列です。	08 前編 5. ログインとセッション
`FROM`	`SELECT … FROM 表` の一部で、「どの表から」取り出すかを書きます。	08 前編 5. ログインとセッション
`JOIN`	2 つの表を、条件で横につないで取り出す書き方。`FROM notes JOIN users ON users.id = notes.user_id` は、ノートと、その持ち主を 1 行に並べます。	08 前編 5. ログインとセッション
`WHERE`	`SELECT … FROM 表 WHERE 条件` の、条件を書く所。`WHERE user_id = 1` は「持ち主が 1 番の行だけ」。`UPDATE`・`DELETE` で `WHERE` を忘れると、全部の行が変わる（消える）ので注意します。	08 前編 5. ログインとセッション
`$1`	PostgreSQL の SQL の `$1`・`$2` は、「ここに 1 番目・2 番目の値を入れる」という、あとから値を渡す場所（プレースホルダー）です。値は SQL とは別に送られるので、入力が SQL の命令として読まれることがありません。	08 前編 5. ログインとセッション
`INSERT`	`INSERT INTO 表 (列, …) VALUES (値, …)` で、表に新しい行を足す SQL の命令。	08 前編 5. ログインとセッション
`INTO`	`INSERT INTO 表` の一部で、「この表へ」入れる、という意味。	08 前編 5. ログインとセッション
`VALUES`	`INSERT INTO 表 (列, …) VALUES (値, …)` の、入れる値を書く所。	08 前編 5. ログインとセッション
`表.列`	SQL の `n.title`・`users.email` のように `表.列` と書くと、「どの表の列か」をはっきりさせられます。いくつもの表をつなぐ（`JOIN`）時は、同じ名前の列（`id` など）があるので、こう書きます。`notes AS n` のように、表に短い別名を付けて使います。	08 前編 5. ログインとセッション
`UPDATE`	`UPDATE 表 SET 列 = 値 WHERE 条件` で、行の中身を書きかえる SQL の命令。`WHERE` を忘れると、全部の行が変わります。	08 前編 5. ログインとセッション
`SET`	`UPDATE` の中で、「どの列を、どの値にするか」を書く所。	08 前編 5. ログインとセッション

コード	意味と使い方	初めて出てくる所
`AS`	SQL で、列や表に別の名前を付ける書き方。`SELECT COUNT(*) AS notes` は、結果の列の名前を `notes` にします。`FROM users AS u` は、表を `u` と短く呼べるようにします。	08 前編 6. 状態を変える処理
`FOR`	`SELECT … FOR UPDATE` は、読んだ行に鍵をかける書き方です。同じ行を、別の人が同時に変えようとしても、こちらのトランザクションが終わるまで待たせるので、「2 人が同時に承認して 2 回進む」ことを防げます。	08 前編 6. 状態を変える処理
`OF`	`FOR UPDATE OF s, r` は、つないだ表（`JOIN`）のうち、鍵をかける表を名前（ここでは `s` と `r`）で選ぶ書き方です。	08 前編 6. 状態を変える処理
`ANY`	PostgreSQL の `id = ANY($1)` は、「`id` が、渡した並び（`$1`）の中のどれかと同じ」という条件です。いくつもの番号を、1 回の SQL で調べられます。	08 前編 6. 状態を変える処理
`COUNT(*)`	SQL の `COUNT(*)` は、行の数を数えます。`GROUP BY status` と組み合わせると、`status` ごとの件数になります。	08 前編 6. 状態を変える処理
`count()`	SQL の `count(*)` は、行の数を数える関数です（`COUNT(*)` と同じで、大文字・小文字はどちらでもかまいません）。	08 前編 6. 状態を変える処理
`RETURNING`	`INSERT … RETURNING id` で、入れた（変えた）行の値（作られた番号など）を、その場で返してもらう書き方。	08 前編 6. 状態を変える処理
`ORDER`	`ORDER BY 列` で、取り出した行を並べる書き方。`DESC` を付けると大きい順（新しい順）です。	08 前編 6. 状態を変える処理
`BY`	`ORDER BY`（並べる）・`GROUP BY`（まとめる）の一部で、「〜で」という意味です。	08 前編 6. 状態を変える処理
`LIMIT`	`LIMIT 件数` で、取り出す行の数の上限を決める書き方。`ORDER BY … LIMIT 20` は「並べて、先頭の 20 件だけ」。	08 前編 6. 状態を変える処理
`DO`	`INSERT … ON CONFLICT (列) DO UPDATE SET …` は、「同じ値の行がもうあれば、足す代わりに、その行を変える」という書き方です（アップサート）。何もしない時は `DO NOTHING` です。	08 前編 8. 社員の CSV と、管理のコマンド
`EXCLUDED`	`ON CONFLICT … DO UPDATE` の中で、「足そうとした（はじかれた）方の行」を指す名前。`SET name = EXCLUDED.name` は「今の行の名前を、新しく入れようとした名前に変える」です。	08 前編 8. 社員の CSV と、管理のコマンド
`LEFT`	`LEFT JOIN` は、左の表の行は、相手が無くても全部残す JOIN。「ノートが 0 件の人も一覧に出す」時に使います。	08 前編 9. 毎朝 8:00 の通知

コード	意味と使い方	初めて出てくる所
`NOTHING`	`ON CONFLICT DO NOTHING` は、「入れようとした行が、決まり（`UNIQUE` など）とぶつかったら、何もしない」という書き方です。同じ仕事を 2 回始めないようにする時などに使います。	08 前編 9. 毎朝 8:00 の通知
`OR`	SQL で「または」。`WHERE a = 1 OR b = 2` は、どちらかに合う行です。`AND` と混ぜる時は、`( )` で囲んで順番をはっきりさせます。	08 前編 10. 画面から呼ぶ、読むだけの SQL
`EXISTS`	SQL で「ある」。`IF NOT EXISTS` は「無ければ」という意味です。	08 前編 10. 画面から呼ぶ、読むだけの SQL
`DESC`	`ORDER BY 列 DESC` で、大きい順（新しい順）に並べます（小さい順は `ASC`）。	08 前編 10. 画面から呼ぶ、読むだけの SQL
`GROUP`	`GROUP BY 列` で、同じ値の行をまとめて、まとめごとに数える・足す時の書き方。`SELECT user_id, COUNT(*) FROM notes GROUP BY user_id` は、人ごとのノートの数です。	08 前編 10. 画面から呼ぶ、読むだけの SQL
`IS`	SQL で NULL かどうかを調べる時の書き方。`WHERE x IS NULL`（無い）・`WHERE x IS NOT NULL`（ある）。NULL は `= NULL` では比べられません。	08 前編 13. 自動のテスト
`PREPARE`	PostgreSQL で、値を入れる場所（`$1`）付きの SQL を、名前を付けて用意する命令。値は文と別に渡るので、SQL インジェクションが起きません。	10 8. SQL インジェクション
`EXECUTE`	`PREPARE` で用意した SQL に、値を渡して実行する命令。	10 8. SQL インジェクション

JSON

コード	意味と使い方	初めて出てくる所
`Deserialize`	「JSON などから作れる」機能（トレイト。`serde` の部品）。`#[derive(Deserialize)]` を付けた型は、`serde_json::from_str` で JSON から作れます。逆向き（値 → JSON）は `Serialize` です。	07 5. Topcoat の最小のアプリ
`Serialize`	「JSON などにできる」機能（トレイト。`serde` の部品）。`#[derive(Serialize)]` を付けた型は、`serde_json::to_string` で JSON にできます。	08 後編 1. 画面の並び

コード	意味と使い方	初めて出てくる所
`serde_json::to_string()`	Rust の値を、JSON の文字列（1 行）に変える関数。値の型に `Serialize` が付いている必要があります。失敗することもあるので `Result` が返ります。	08 後編 5. 入力フォーム
`#[serde]`	`serde` の印（属性）。JSON にする時の細かい決まりを付けます。`#[serde(rename_all = "camelCase")]`（名前の書き方を変える）・`#[serde(default)]`（無い時は初期値）・`#[serde(skip_serializing_if = "Option::is_none")]`（空なら書かない）などがあります。	08 後編 6. 図面
`serde_json::from_str()`	JSON の文字列を読んで、決めた型（struct など）に変える関数。形が合わないと `Err` を返します。	08 後編 6. 図面
`#[serde(rename_all)]`	`#[serde(rename_all = "camelCase")]` は、項目の名前を、JSON では別の書き方にする印です。`camelCase` なら、Rust の `width_mm` が JSON では `widthMm` になります。`snake_case`・`kebab-case`・`UPPERCASE` などもあります。	08 後編 6. 図面
`#[serde(deny_unknown_fields)]`	`serde` の印。JSON に、構造体に無い名前の項目が入っていたら、読み込みをエラーにします。項目の名前の書き間違い（`titel` など）を見逃さないために付けます。	08 後編 6. 図面
`#[serde(tag)]`	`#[serde(tag = "kind")]` は、enum を JSON にする時に、種類の名前を `"kind": "line"` のような項目として、中身と一緒に入れる指定です。読む時も、その項目を見て、どの種類かを決めます。	08 後編 6. 図面

Topcoat

コード	意味と使い方	初めて出てくる所
`topcoat`	Topcoat の道具のコマンド（`cargo install topcoat-cli` で入る）。`topcoat dev`（コードを直すと自動で作り直して読み直す開発用の起動）・`topcoat asset bundle`（CSS などの束を作る）などがあります。	07 1. 準備
`topcoat --version`	`topcoat --version` は、topcoat CLI（Topcoat の道具）の版を表示します。アプリで使う Topcoat の版と同じかを確かめます。	07 1. 準備
`topcoat new --minimal`	`topcoat new 名前 --minimal` の `--minimal` は、いちばん小さなひな形（1 ページだけ）でプロジェクトを作る指定です。	07 5. Topcoat の最小のアプリ

コード	意味と使い方	初めて出てくる所
`module_router!`	Topcoat のマクロ。Rust のモジュール（ファイル）の並びを、そのまま URL の並びにしたルーター（URL ごとに、どの関数に答えさせるかの名簿）を作ります。	07 5. Topcoat の最小のアプリ
`view!`	Topcoat のマクロ。中に HTML をほぼそのままの形で書いて、画面（`View`）を作ります。文字は `"…"` で囲み、Rust の値は `(値)` で埋め込みます。埋め込んだ文字の `<` などは、自動で安全な形に置きかえられます。	07 5. Topcoat の最小のアプリ
`asset!`	Topcoat のマクロ。CSS や画像などのファイル（アセット）を「このファイルを配る」と宣言し、ブラウザーから読める URL を作ります。`asset!("styles.css")` を、`<link>` の `href` などに入れて使います。	07 5. Topcoat の最小のアプリ
`href!`	Topcoat のマクロ。ページの関数から、その URL を作ります。`href!(id::show, id::Id(note.id))` は「このノートの詳細ページの URL」。URL を文字で書かないので、打ちまちがいがコンパイルの時に見つかります。	07 5. Topcoat の最小のアプリ
`#[layout]`	Topcoat の印（属性）。この関数を「どのページにも共通の外枠（HTML の骨組み・メニュー）」にします。外枠の中の、ページが入る場所を `slot` で受け取ります。	07 5. Topcoat の最小のアプリ
`#[page]`	Topcoat の印（属性）。この関数を、1 つの URL に答える「ページ」にします。`#[page("/")]` なら `/` に、`#[page(POST)]` なら送信（POST）に答えます。	07 5. Topcoat の最小のアプリ
`.assets()`	ルーターに、配るアセット（CSS など）の束を登録するメソッド。`.assets(AssetBundle::load().unwrap())`。	07 5. Topcoat の最小のアプリ
`.app_context()`	アプリ全体で共有する物（データベースの接続など）をルーターに登録するメソッド。ページの関数からは、`cx` を通して取り出せます。	07 5. Topcoat の最小のアプリ
`.resolve()`	`href!` で作った URL の部品を、今のリクエスト（`cx`）に合わせて、本物の URL の文字列にするメソッド。	07 5. Topcoat の最小のアプリ
`AssetBundle::load()`	`topcoat asset bundle` で作った CSS などの束（アセット）を読み込む関数。`.assets(AssetBundle::load().unwrap())` のように、ルーターに渡して配れるようにします。	07 5. Topcoat の最小のアプリ
`topcoat::dev::script()`	`topcoat dev` で動かしている時だけ、「コードを直したら、ブラウザーを自動で読み直す」ための小さなスクリプトを、ページに入れる関数。`cargo run` や本番では何も出しません。	07 5. Topcoat の最小のアプリ
`AssetBundle`	Topcoat の型。`topcoat asset bundle` で作った「アセットの束」（名前にハッシュが付いたファイルと目録）を、`AssetBundle::load()` で読み込んで、配れるようにします。	07 5. Topcoat の最小のアプリ

コード	意味と使い方	初めて出てくる所
`RouterBuilderAssetExt`	Topcoat のトレイト（機能のまとまり）。`use` しておくと、ルーターを組み立てる時に、アセット（CSS など）を配る設定のメソッドが使えるようになります。	07 5. Topcoat の最小のアプリ
`Cx`	Topcoat の「今のリクエストの情報（URL・クッキー・セッションなど）が入った箱」。ページの関数の引数 `cx: &Cx` で受け取り、`href!(…).resolve(cx)` などで使います。	07 5. Topcoat の最小のアプリ
`Router`	URL とメソッド（GET・POST）ごとに、どの関数に答えさせるかを決めた名簿（ルーター）。Topcoat のサーバーは、来た頼み（リクエスト）をこの名簿で振り分けます。	07 5. Topcoat の最小のアプリ
`Slot`	`#[layout]` の関数で受け取る「ページが入る場所」。レイアウトの HTML の中で `{slot}` と書いた所に、各ページの中身が入ります。	07 5. Topcoat の最小のアプリ
`Form`	Topcoat の、送られてきたフォームの中身を受け取る型。`Form(input): Form<NoteForm>` と書くと、フォームの項目が `NoteForm` の形に読み取られて `input` に入ります。	07 5. Topcoat の最小のアプリ
`View`	Topcoat の「画面（HTML になる物）」の型（トレイト）。ページの関数は `Result<impl View>`（HTML になる何かを返す。失敗することもある）を返します。	07 5. Topcoat の最小のアプリ
`POST`	`#[page(POST)]` の `POST` は、「フォームの送信（POST）に答えるページ」にする指定。何も書かなければ、ふつうの表示（GET）に答えます。	07 5. Topcoat の最小のアプリ
`topcoat asset bundle --release`	`topcoat asset bundle --release` は、CSS などのアセットの束を、本番用（`target\release\assets`）に作ります。`cargo build --release` と組にして使います。	07 6. async と、会社のサーバーへの持ち込み
`#[memoize]`	Topcoat の印（属性）。付けた関数の結果を、**1 つの要求（リクエスト）の間だけ**覚えておきます。レイアウト・ページ・部品のどこから呼んでも、データベースを 1 回しか読まずに済みます。	08 前編 5. ログインとセッション
`.token_store()`	セッションの札（トークン）を、どこに置くかを決めるメソッド。ここでは、HTTP のページでもログインが続くように、自分で用意した `HttpCookieTokenStore`（クッキーに置く係）を渡しています。	08 前編 5. ログインとセッション
`.lifetime()`	セッションの長さ（何も操作しないと切れるまでの時間）を決めるメソッド。`.lifetime(IDLE)`（`IDLE` は 60 分）。	08 前編 5. ログインとセッション
`SessionConfig::builder()`	Topcoat のセッションの設定を、少しずつ決めて組み立て始める関数。`.token_store(…)`・`.lifetime(…)` をつなげて、最後に `.build()` で出来上がりです。	08 前編 5. ログインとセッション

コード	意味と使い方	初めて出てくる所
`session::start()`	Topcoat のセッションを新しく始める（ログインする）関数。ログインのたびに新しい札を作るので、前から知られている札を使わせない（セッション固定の攻撃を防ぐ）ことができます。	08 前編 5. ログインとセッション
`session::stop()`	Topcoat のセッションを終わらせる（ログアウトする）関数。消した札のハッシュを返すので、データベースの記録も消せます。	08 前編 5. ログインとセッション
`session::token_hash()`	Topcoat の関数。今のリクエストのセッションの印（トークン）の、ハッシュを返します。データベースの `sessions` 表に、ハッシュで記録しておき、ログイン中かを調べるのに使います。	08 前編 5. ログインとセッション
`session::refresh()`	今のセッションの期限を、先へ延ばす関数。使うたびに呼ぶと、「何も操作しないと 60 分で切れる」動きになります。セッションが無ければ `None` です。	08 前編 5. ログインとセッション
`SessionConfig`	Topcoat のセッションの設定（クッキーの名前・期限など）の型。	08 前編 5. ログインとセッション
`#[memoize(as_ref)]`	`#[memoize(as_ref)]` の `as_ref` は、覚えておいた結果を、複製せずに「借りる形」で返す指定です。`Option<User>` を返す関数なら、呼んだ側には `Option<&User>` が返ります。付けない `#[memoize]` は `&Option<User>` を返します。	08 前編 5. ログインとセッション
`module_param!`	Topcoat のマクロ。モジュールの区切りを「URL の変わる部品」にします。`mod id` の中に `module_param!(id: u64, error = not_found);` と書くと、URL が `/notes/{id}` になり、数として読めない時は 404 になります。	08 前編 7. 1 回限りの承認の URL
`path_param!`	URL の中の値（`/attachments/{file_id}` の `file_id` など）を受け取る型を作るマクロ。`path_param!(pub file_id: i64, error = not_found);` は、`FileId` という型を作り、数でない時は「見つかりません」にします。`href!(…, FileId(5))` で URL を作る時にも使います。	08 前編 7. 1 回限りの承認の URL
`#[route]`	Topcoat の印（属性）。画面（HTML）ではなく、JSON やただの文字などを返す「ルート」にします。`#[route(GET)]` のように、メソッドを必ず書きます。人が見る画面は `#[page]`、プログラムが読む答えは `#[route]` と使い分けます。	08 前編 7. 1 回限りの承認の URL
`.ok_or_not_found()`	Topcoat のメソッド。`Option` が `None`（無い）の時に、404 Not Found のエラーにします。`… .ok_or_not_found()?` と書くと、無い時はそこで 404 を返します（`RouterErrorExt` を `use` して使う）。	08 前編 7. 1 回限りの承認の URL

コード	意味と使い方	初めて出てくる所
`RouterErrorExt`	`.ok_or_not_found()` などを使えるようにする機能（トレイト）。	08 前編 7. 1 回限りの承認の URL
`Token`	`topcoat::session::Token` は、セッションの印（でたらめな長い文字列）の型。ブラウザーのクッキーに入れ、サーバーには、そのハッシュだけを保存します。	08 前編 7. 1 回限りの承認の URL
`FileId`	`path_param!(pub file_id: i64 …)` が作る型。URL の中の、添付ファイルの番号を表します。`FileId(a.id)` で URL を作り、`*path_param::<FileId>(cx)?` で受け取ります。	08 前編 7. 1 回限りの承認の URL
`Cx::new()`	`Cx`（Topcoat の、共有の物を取り出す入れ物）を、画面のリクエストの外で作る関数。毎朝の処理やコマンドで、メールの設定などを使うために作ります。	08 前編 8. 社員の CSV と、管理のコマンド
`AppContext::new()`	空の `AppContext` を作る関数。`.insert(物)` で入れていきます。	08 前編 8. 社員の CSV と、管理のコマンド
`topcoat::start()`	組み立てたルーターを受け取って、Web サーバーを動かし始める関数。`topcoat::start(router).await` は、止めるまでリクエストを待ち続けます。	08 前編 8. 社員の CSV と、管理のコマンド
`AppContext`	アプリで共有する物（データベースや設定など）を、型ごとに入れておく入れ物。	08 前編 8. 社員の CSV と、管理のコマンド
`CxTestBuilder::new()`	`CxTestBuilder` を作る関数。`.app_context(物)` で入れたい物を足し、`.build()` で `Cx` にします。サーバーを動かさずに、アプリの関数を直接試せます。	08 前編 13. 自動のテスト
`CxTestBuilder`	テストで、`Cx`（共有の物の入れ物）を組み立てる型。	08 前編 13. 自動のテスト
`#[query_params]`	Topcoat の印（属性）。`?q=rust&page=2` のようなクエリ（URL の `?` の後ろ）を読むための構造体に付けます。項目は `Option<T>` にしておき、無い・空の時は `None` になります。	08 後編 1. 画面の並び
`.cookies()`	ルーターに、クッキーを使う仕組みを登録するメソッド（`module_router!().cookies()`）。セッションを使う時も、先にこれを登録します。	08 後編 1. 画面の並び
`.sessions()`	ルーターに、セッション（ログインの状態を覚える仕組み）を登録するメソッド。`.sessions(SessionConfig::default())`。先に `.cookies()` も登録します。	08 後編 1. 画面の並び

コード	意味と使い方	初めて出てくる所
`.layer()`	ルーターに、すべてのリクエストの前後で動く仕組み（レイヤー。受け取る大きさの上限など）を足すメソッド。	08 後編 1. 画面の並び
`.at()`	`BodyLimit` などのレイヤーを、決めた URL の下だけで効かせるメソッド。`.at("/requests")` は `/requests` から始まる URL だけ。	08 後編 1. 画面の並び
`.query()`	`href!(…)` で作った URL に、`?mine=1` のような問い合わせの部分を足すメソッド。`href!(home).query(MineOnly { mine: "1" })`。	08 後編 1. 画面の並び
`BodyLimit::max()`	上限の大きさ（バイト）を決めて `BodyLimit` を作る関数。`BodyLimit::max(60 * 1024 * 1024)` は 60MB です。	08 後編 1. 画面の並び
`RouterBuilderCookieExt`	ルーターに `.cookies()`（クッキーを使う仕組み）を登録できるようにする機能（トレイト）。	08 後編 1. 画面の並び
`RouterBuilderMailExt`	ルーターに `.mail(…)`（メールの設定を登録する）を足すための、Topcoat のトレイト。`use` しておくと `.mail(mail)` を呼べます。	08 後編 1. 画面の並び
`BodyLimit`	受け取るリクエストの本文の大きさに、上限を決めるレイヤー。上限を超えると `413`（大きすぎる）を返します。	08 後編 1. 画面の並び
`NotFoundError`	Topcoat の「404 Not Found（見つかりません）」のエラーの型。	08 後編 1. 画面の並び
`RouterBuilderSessionExt`	ルーターに `.sessions(…)`（セッションの仕組み）を登録できるようにする機能（トレイト）。セッションはクッキーを使うので、先に `.cookies()` も登録します。	08 後編 1. 画面の並び
`Id`	URL の一部（`/requests/3` の `3` など）を、型として受け取るための入れ物。練習アプリでは `Id(番号)` の形で、URL の中の依頼の番号を表します。`href!(detail, Id(5))` で URL を作り、ページの関数で受け取ります。	08 後編 1. 画面の並び
`#[component]`	Topcoat の印（属性）。画面の一部を作る、何度も使える部品（コンポーネント）にします。`view!` の中で `panel(title: "一覧", …)` のように、名前付きの引数で呼べます。	08 後編 2. 画面の部品と、XSS を防ぐ仕組み
`SeeOther`	「303 See Other」（別のページへ移って）という答えを表す型。フォームを送ったあと、一覧や詳細のページへ移す時（PRG）に使います。	08 後編 3. ログインの画面
`.next_field()`	`Multipart` から、次のフォームの項目を 1 つ受け取るメソッド。無くなると `None` です。	08 後編 5. 入力フォーム
`.name()`	フォームの項目の名前（`<input name="…">` の `name`）を返すメソッド。	08 後編 5. 入力フォーム

コード	意味と使い方	初めて出てくる所
`Multipart`	ファイル付きのフォーム（`multipart/form-data`）を、1 項目ずつ読むための型。`while let Some(field) = multipart.next_field().await?` で、項目を順に受け取ります。	08 後編 5. 入力フォーム
`.bcc()`	メールの BCC（ほかの受け取り手には見えない宛先）を取り出すメソッド（テストで確かめる）。	10 13. テスト
`.formatted()`	メールを、実際に送る形（ヘッダーと本文の文字の並び）にするメソッド。	10 13. テスト
`Cx::default()`	テストなどで、空のリクエストの情報の箱（`Cx`）を作る書き方。	10 13. テスト

練習アプリで作る物（Rust）

コード	意味と使い方	初めて出てくる所
`.css_class()`	状態（`Status`）から、画面の札の CSS の class（`badge-circulating` など）を返すメソッド。	07 5. Topcoat の最小のアプリ
`.add()`	`Store` に、新しい依頼を 1 件足すメソッド。	07 5. Topcoat の最小のアプリ
`Store::sample()`	見本の依頼をいくつか入れた `Store` を作る関数。	07 5. Topcoat の最小のアプリ
`Store`	依頼の一覧を、メモリーの中に持っておく struct。少し後の所で作ります。	07 5. Topcoat の最小のアプリ
`text_enum!`	この巻で `macro_rules!` を使って作るマクロ。`text_enum!(Status { Circulating => ("circulating", "回覧中"), … })` と書くと、enum と、データベースの文字との行き来（`as_str`）、画面に出す日本語（`label`）、文字から enum に戻す変換（`TryFrom<String>`）を、まとめて作ります。	08 前編 4. 表
`Decision`	承認する人 1 人ごとの答え（待ち・承認・差し戻し・取り消し）を表す enum。`text_enum!` で作ります。	08 前編 4. 表
`Waiting`	`Decision` の値の 1 つで、「まだ答えていない（待ち）」。	08 前編 4. 表
`Cancelled`	`Decision` の値の 1 つで、「依頼が取り下げられたので、答えなくてよくなった（取り消し）」。	08 前編 4. 表
`Role`	社員の役割（一般・担当者・管理者）を表す enum。`text_enum!` で作ります。担当者と管理者は、全部の依頼を見たり、担当を決めたりできます。	08 前編 4. 表

コード	意味と使い方	初めて出てくる所
`Member`	`Role` の値の 1 つで、ふつうの社員（一般）。自分の依頼を出し、頼まれた承認をします。	08 前編 4. 表
`Worker`	`Role` の値の 1 つで、依頼を受けて作業する担当者。	08 前編 4. 表
`Admin`	`Role` の値の 1 つで、管理者。全部の依頼を見て、担当を決められます。	08 前編 4. 表
`Drawing`	依頼に付ける図面（四角や線の並び）を表す struct。後編で作り、`JSONB` の列に入れます。	08 前編 4. 表
`HttpCookieTokenStore`	セッションの札を、`Secure` の付かないクッキーに置く係（`src/http_cookie.rs`）。Topcoat の決まりのクッキーは HTTPS のページでしか送られないので、社内の HTTP のページでもログインが続くように、onprem の 04 で確かめた物をそのまま使います。	08 前編 5. ログインとセッション
`.send_all()`	`Outbox` にためたメールを、まとめて送るメソッド。トランザクションを `commit` したあとに呼びます。	08 前編 6. 状態を変える処理
`upload::save()`	送られてきた添付ファイルを、ファイルを置くフォルダに保存する関数。後編で作ります。	08 前編 6. 状態を変える処理
`upload::remove()`	保存した添付ファイルを消す関数。データベースへの記録に失敗した時に、保存したファイルを片づけるのに使います。後編で作ります。	08 前編 6. 状態を変える処理
`Outbox::new()`	空の `Outbox`（送るメールの箱）を作る関数。	08 前編 6. 状態を変える処理
`Outbox`	送るメールを、すぐには送らずに、ためておく箱（`src/mail.rs`）。データベースの変更を `commit` で確定してから、`.send_all()` でまとめて送ります。確定に失敗したのにメールだけ届く、ということを防げます。	08 前編 6. 状態を変える処理
`SavedFile`	保存した添付ファイルの情報（保存した名前・元の名前・大きさなど）をまとめた struct。後編で作ります。	08 前編 6. 状態を変える処理
`UploadedFile`	フォームから送られてきた添付ファイル（名前・種類・中身のバイト）をまとめた struct。後編で作ります。	08 前編 6. 状態を変える処理
`queries::find_request()`	依頼を 1 件、承認の流れや添付も含めて読む関数。無ければ `None` です。	08 前編 7. 1 回限りの承認の URL
`queries::find_attachment()`	添付ファイルの記録を、番号で 1 件読む関数。無ければ `None` です。	08 前編 7. 1 回限りの承認の URL

コード	意味と使い方	初めて出てくる所
`files::download()`	添付ファイルを読んで、ダウンロードさせる答え（レスポンス）を作る関数。後編で作ります。	08 前編 7. 1 回限りの承認の URL
`FullRequest`	依頼 1 件の全部（依頼そのもの・承認の流れ・添付）をまとめた struct。	08 前編 7. 1 回限りの承認の URL
`jobs::daily_reminders()`	毎朝の処理（期限の近い依頼や、止まっている承認の知らせ）を 1 回する関数。	08 前編 8. 社員の CSV と、管理のコマンド
`jobs::spawn_daily()`	毎朝の処理を、サーバーの裏で動かし続ける関数。	08 前編 8. 社員の CSV と、管理のコマンド
`employees::read_csv()`	社員の CSV を読んで、1 行ずつ確かめる関数。まちがいがあれば、何行目の何がおかしいかを返します。	08 前編 8. 社員の CSV と、管理のコマンド
`employees::import()`	読んだ社員を、データベースに入れる（あれば変える）関数。作った数と変えた数を返します。	08 前編 8. 社員の CSV と、管理のコマンド
`AppState`	アプリ全体で共有する物（データベースのプールと設定）をまとめた struct。`.app_context(…)` でルーターに渡し、どのページからも `state(cx)` で使えるようにします。	08 前編 8. 社員の CSV と、管理のコマンド
`Shape`	図面の部品（四角 `Rect`・線 `Line` など）を表す enum。後編で作ります。	08 後編 2. 画面の部品と、XSS を防ぐ仕組み
`Rect`	`Shape` の値の 1 つで、四角（左上の位置 `x`・`y` と、幅 `width`・高さ `height`）。	08 後編 2. 画面の部品と、XSS を防ぐ仕組み
`Line`	`Shape` の値の 1 つで、線（始まり `x1`・`y1` と、終わり `x2`・`y2`）。	08 後編 2. 画面の部品と、XSS を防ぐ仕組み
`Msg`	画面に 1 回だけ出すお知らせ（「出しました」など）を、URL の `?msg=…` で受け渡すための struct。後編で作ります。	08 後編 5. 入力フォーム

メール

コード	意味と使い方	初めて出てくる所
`mail!`	Topcoat の部品で、送るメールを組み立てるマクロ。`mail! { from: …, to: …, subject: …, text: … }`。宛先の形がまちがっているとエラーを返すので、`?` を付けます。	08 前編 6. 状態を変える処理
`.transport()`	メールを実際に送る係（トランスポート）を決めるメソッド。ファイルに書き出す `FileTransport`・SMTP で送る `SmtpTransport`・メモリーにためる `MemoryTransport` などを渡します。	08 前編 6. 状態を変える処理
`.port()`	つなぐ先のポート番号（SMTP なら 25・587 など）を決めるメソッド。	08 前編 6. 状態を変える処理
`MailConfig::builder()`	`MailConfig` を組み立て始める関数。`.transport(送る係)` をつなげて、`.build()` で出来上がりです。	08 前編 6. 状態を変える処理
`FileTransport::new()`	`FileTransport` を、書き出すフォルダを渡して作る関数。	08 前編 6. 状態を変える処理
`SmtpTransport::starttls()`	STARTTLS（つないでから暗号化に切りかえる方法）で送る `SmtpTransport` を作り始める関数。`.port(587)` などをつなげて、`.build()` で出来上がりです。	08 前編 6. 状態を変える処理
`SmtpTransport::unencrypted()`	暗号化しないで送る `SmtpTransport` を作り始める関数。社内の、外へ出ない中継サーバー（リレー）に渡す時などに使います。	08 前編 6. 状態を変える処理
`FileTransport`	メールを送らずに、フォルダの中のファイルに書き出す係。開発中に、送られたはずのメールの中身を確かめるのに使います。	08 前編 6. 状態を変える処理
`MailConfig`	Topcoat の、メールの送り方（どこへ・どうやって送るか）の設定の型。	08 前編 6. 状態を変える処理
`SmtpTransport`	SMTP（メールを送る決まり）で、メールサーバーに渡して送る係。	08 前編 6. 状態を変える処理
`SmtpError`	Topcoat の、SMTP の設定や送信で起きたエラーの型。	08 前編 6. 状態を変える処理
`.sent()`	`MemoryTransport` にたまった、送ったメールの一覧を返すメソッド。	08 前編 13. 自動のテスト
`.text()`	メールの本文（文字の方）を返すメソッド（`TextBody` で返る）。後編のアップロードでは、フォームの項目（`field.text()`）の中身を文字列で読むのにも使います。	08 前編 13. 自動のテスト

コード	意味と使い方	初めて出てくる所
`.to()`	メールの宛先の一覧を返すメソッド。`m.to()[0].address()` で、1 人目の宛先のメールアドレスを取り出せます。	08 前編 13. 自動のテスト
`.address()`	宛先から、メールアドレスの部分を取り出すメソッド。	08 前編 13. 自動のテスト
`.subject()`	メールの件名を返すメソッド。	08 前編 13. 自動のテスト
`MemoryTransport::new()`	空の `MemoryTransport` を作る関数。複製（`.clone()`）しても同じ入れ物を指すので、片方をアプリに渡し、もう片方で中身を見られます。	08 前編 13. 自動のテスト
`MemoryTransport`	メールを送らずに、メモリーの中にためておく係。テストで、送ったはずのメールの宛先や本文を確かめるのに使います。	08 前編 13. 自動のテスト
`TextBody`	メールの文字の本文を表す型。`TextBody::Text(文字列)` で中身を取り出せます。	08 前編 13. 自動のテスト